

LÊ THANH BÌNH

CÔNG NGHỆ THÔNG

BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC PHENIKAA



ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NỀN TẢNG WEB THƯ VIỆN DÀNH CHO
THIẾT BỊ THÔNG MINH

Sinh viên: Lê Thanh Bình

Mã số sinh viên: 22010495

Khóa: Khóa 16

Ngành: Hệ thống thông tin

Hệ: Chính quy

Giảng viên hướng dẫn: TS. Đặng Thị Thúy An

Hà Nội – Năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



ĐỒ ÁN TỐT NGHIỆP
XÂY DỰNG NỀN TẢNG WEB THƯ VIỆN DÀNH CHO
THIẾT BỊ THÔNG MINH

Sinh viên: Lê Thanh Bình

Mã số sinh viên: 22010495

Khóa: Khóa 16

Ngành: Hệ thống thông tin

Hệ: Chính quy

Giảng viên hướng dẫn: TS. Đặng Thị Thúy An

Hà Nội – Năm 2026

TÓM TẮT

Đồ án tập trung vào việc xây dựng một nền tảng tra cứu và nghiên cứu thông tin thiết bị công nghệ. Hệ thống được phát triển nhằm giải quyết tình trạng thông số thiết bị phân tán trên nhiều nguồn, cách đặt tên không đồng nhất và khó kiểm chứng nguồn dữ liệu. SpecHub hướng tới xây dựng một kho dữ liệu có cấu trúc, giúp người dùng dễ dàng tìm kiếm, tra cứu, so sánh và đánh giá các thiết bị trên cùng một nền tảng.

Các chức năng chính của hệ thống bao gồm quản lý danh mục thiết bị và linh kiện phân cứng, tìm kiếm và lọc sản phẩm, xem thông tin chi tiết, so sánh thiết bị, chấm điểm, khuyến nghị theo nhu cầu, quản lý danh sách yêu thích, theo dõi lịch sử giá và cảnh báo giá. Bên cạnh đó, SpecHub tích hợp hệ thống Wiki và trợ lý trí tuệ nhân tạo sử dụng phương pháp **Retrieval-Augmented Generation (RAG)** nhằm khai thác dữ liệu trong kho tri thức và cung cấp câu trả lời có nguồn tham chiếu.

Về mặt công nghệ, hệ thống được tổ chức theo mô hình **monorepo**, sử dụng Next.js và React để phát triển giao diện, NestJS kết hợp Fastify cho phía máy chủ, Prisma để truy cập dữ liệu và PostgreSQL làm hệ quản trị cơ sở dữ liệu. Redis được sử dụng để hỗ trợ quản lý phiên và dữ liệu tạm, trong khi Meilisearch hỗ trợ tăng cường khả năng tìm kiếm.

Kết quả triển khai cho thấy SpecHub đã xây dựng được nền tảng cơ bản cho việc quản lý và khai thác dữ liệu thiết bị theo hướng có cấu trúc, có khả năng mở rộng và hỗ trợ nhiều nhóm người dùng. Trong tương lai, hệ thống có thể tiếp tục mở rộng dữ liệu thực tế, hoàn thiện kiểm thử, tối ưu hiệu năng và nâng cao chất lượng các chức năng trí tuệ nhân tạo.

LỜI CAM ĐOAN

Nhóm thực hiện xin cam đoan báo cáo đồ án tốt nghiệp này là kết quả của quá trình nghiên cứu, phân tích, thiết kế, xây dựng và đánh giá dự án Spechub. Nội dung báo cáo được trình bày trung thực, thống nhất với mã nguồn, schema dữ liệu, cấu hình hệ thống và kết quả test tại thời điểm đánh giá.

Các kiến thức, tài liệu và công nghệ được tham khảo đều được ghi rõ trong phần Tài liệu tham khảo. Những số liệu, hình ảnh giao diện và sơ đồ do nhóm thực hiện xây dựng được sử dụng đúng mục đích học thuật. Nhóm thực hiện chịu trách nhiệm về tính chính xác và trung thực của nội dung báo cáo.

Hà Nội, tháng 8 năm 2026

Nhóm phát triển Spechub

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn Trường Công nghệ Thông tin Phenikaa - Đại học Phenikaa đã tạo điều kiện cho tôi hoàn thành tốt Đồ án khóa luận tốt nghiệp. Đồ án khóa luận tốt nghiệp này đã giúp cho tôi có thêm rất nhiều kinh nghiệm quý báu để tôi có thể hoàn thiện hơn trong công việc và môi trường làm việc sau này.

Tôi xin gửi lời cảm ơn đến quý thầy cô và Trường Công nghệ Thông tin Phenikaa - Đại học Phenikaa đã tạo điều kiện tuyệt vời cho tôi hoàn thành đồ án tốt nghiệp. Cảm ơn các thầy cô vì đã chia sẻ kiến thức và kinh nghiệm quý báu, giúp tôi vượt qua những thách thức và hoàn thiện công việc của mình. Đặc biệt, tôi muốn bày tỏ lòng biết ơn sâu sắc đến cô Đặng Thị Thúy An, người đã dành thời gian và tâm huyết để hướng dẫn và góp ý cho đồ án của tôi.

Mặc dù đã nỗ lực hết mình, nhưng tôi nhận thức được rằng có thể vẫn còn những thiếu sót trong suất chiếu/vé cũng như trong báo cáo của tôi. Vì vậy tôi mong rằng trong quá trình bảo vệ, tôi có thể nhận được sự đồng cảm và chia sẻ từ quý thầy cô để hoàn thiện hơn kết quả của mình.

Tôi xin chân thành cảm ơn!

MỤC LỤC

TÓM TẮT.....	i
LỜI CAM ĐOAN.....	ii
LỜI CẢM ƠN.....	iii
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU.....	viii
DANH MỤC TỪ VIẾT TẮT.....	x
MỞ ĐẦU.....	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu.....	1
3. Đối tượng và phạm vi.....	2
4. Phương pháp thực hiện.....	2
5. Đóng góp của đồ án.....	2
6. Cấu trúc báo cáo.....	2
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	3
1.1. Bối cảnh và bài toán.....	3
1.2. Khoảng trống của các cách tiếp cận hiện có.....	4
1.3. Tầm nhìn và phạm vi sản phẩm.....	4
1.4. Các bên liên quan.....	5
1.5. Yêu cầu chức năng.....	6
1.6. Yêu cầu phi chức năng.....	7
1.7. Rủi ro và ràng buộc.....	8
1.8. Kết luận chương.....	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	9
2.1. Nguyên tắc kiến trúc.....	9
2.2. Monorepo và tổ chức mã nguồn.....	10
2.3. Presentation layer với Next.js và React.....	11
2.4. API với NestJS và Fastify.....	11
2.5. Prisma, PostgreSQL và tìm kiếm dữ liệu.....	12

MỤC LỤC (TIẾP)

2.6. Redis, background job và resilience.....	12
2.7. Retrieval-Augmented Generation (RAG) và AI có citation.....	13
2.8. Authentication, authorization và security.....	13
2.9. PWA, observability và triển khai.....	14
2.10. Kết luận chương.....	16
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	18
3.1. Tác nhân và mô hình use case.....	18
3.2. Đặc tả use case.....	21
3.3. Thiết kế sequence diagram.....	29
3.4. Thiết kế background workflow.....	32
3.5. Thiết kế dữ liệu.....	34
3.6. Thiết kế API và data contract.....	39
3.7. Thiết kế an toàn và kiểm soát chất lượng.....	40
3.8. Kết luận chương.....	41
CHƯƠNG 4. PHÁT TRIỂN VÀ KẾT QUẢ.....	42
4.1. Môi trường và quy trình phát triển.....	42
4.2. Hiện thực các module chính.....	43
4.3. Kết quả giao diện.....	43
4.4. Kết quả API và dữ liệu.....	53
4.5. Test và đánh giá.....	54
4.6. Đánh giá mức độ đáp ứng.....	55
4.7. Hạn chế.....	56
4.8. Kết luận chương.....	57
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	58
TÀI LIỆU THAM KHẢO.....	59
PHỤ LỤC A. DANH MỤC API.....	60
PHỤ LỤC B. DANH MỤC MÔ HÌNH DỮ LIỆU.....	62
PHỤ LỤC C. HƯỚNG DẪN VẬN HÀNH VÀ KIỂM THỬ.....	67
PHỤ LỤC D. HƯỚNG DẪN SỬ DỤNG TÓM TẮT.....	69

DANH MỤC HÌNH ẢNH

Hình 1.1. Sơ đồ context hệ thống Spechub.....	3
Hình 2.1. Cấu trúc monorepo Spechub.....	10
Hình 2.2. Kiến trúc logic nhiều lớp.....	15
Hình 2.3. Mô hình triển khai tham chiếu.....	16
Hình 3.1. Use case phía người dùng.....	19
Hình 3.2. Use case biên tập và quản trị.....	20
Hình 3.3. Sequence diagram đăng nhập và refresh session.....	30
Hình 3.4. Sequence diagram tìm kiếm và so sánh.....	31
Hình 3.5. Sequence diagram trợ lý AI theo RAG.....	32
Hình 3.6. Workflow theo dõi giá và phát cảnh báo.....	33
Hình 3.7. Ingestion workflow.....	34
Hình 3.8. ERD nhóm catalog.....	36
Hình 3.9. ERD nhóm hardware và scoring.....	37

DANH MỤC HÌNH ẢNH (TIẾP)

Hình 3.10. ERD nhóm content, crawler và AI/search.....	38
Hình 3.11. ERD nhóm user engagement, subscription và B2B.....	39
Hình 4.1. Trang chủ Spechub.....	45
Hình 4.2. Trang danh mục thiết bị.....	46
Hình 4.3. Trang chi tiết thiết bị.....	47
Hình 4.4. Kết quả tìm kiếm thiết bị và phần cứng.....	48
Hình 4.5. Giao diện chọn thiết bị để so sánh.....	49
Hình 4.6. Quy trình khuyến nghị theo nhu cầu.....	50
Hình 4.7. Giao diện trợ lý AI.....	51
Hình 4.8. Giao diện Wiki công nghệ.....	52
Hình 4.9. Giao diện đăng nhập.....	53

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh cách tiếp cận dữ liệu thiết bị.....	4
Bảng 1.2. Các bên liên quan.....	5
Bảng 1.3. Yêu cầu chức năng chính.....	6
Bảng 1.4. Yêu cầu phi chức năng.....	7
Bảng 1.5. Rủi ro và biện pháp giảm thiểu.....	8
Bảng 2.1. Thành phần công nghệ và phiên bản.....	9
Bảng 2.2. Biện pháp security theo lớp.....	13
Bảng 2.3. Lựa chọn kiến trúc và đánh đổi.....	16
Bảng 3.1. Tác nhân hệ thống.....	18
Bảng 3.2. Danh mục use case.....	20
Bảng 3.3. Đặc tả UC-01 – Tìm kiếm và lọc thiết bị.....	22
Bảng 3.4. Đặc tả UC-02 – So sánh thiết bị.....	23
Bảng 3.5. Đặc tả UC-03 – Nhận khuyến nghị và hỏi AI.....	24
Bảng 3.6. Đặc tả UC-04 – Đăng nhập và quản lý phiên.....	25
Bảng 3.7. Đặc tả UC-05 – Wishlist và cảnh báo giá.....	26
Bảng 3.8. Đặc tả UC-06 – Đóng góp và duyệt Wiki.....	27

DANH MỤC BẢNG BIỂU (TIẾP)

Bảng 3.9. Đặc tả UC-07 – Thu thập và duyệt catalog.....	28
Bảng 3.10. Đặc tả UC-08 – Khai thác API B2B.....	29
Bảng 3.11. Phân nhóm 139 data model.....	34
Bảng 3.12. Quy ước hợp đồng API.....	39
Bảng 4.1. Môi trường phần mềm.....	42
Bảng 4.2. Module hiện thực chính.....	43
Bảng 4.3. Bản đồ route giao diện.....	44
Bảng 4.4. Thống kê endpoint API.....	53
Bảng 4.5. Bảng chứng test và kiểm chứng.....	55
Bảng 4.6. Đánh giá mức độ đáp ứng yêu cầu.....	55
Bảng 4.7. Hạn chế và tác động.....	56
Bảng A.1. Danh mục controller và endpoint.....	60
Bảng B.1. Danh sách Prisma model.....	62
Bảng C.1. Ma trận lệnh vận hành và mục đích.....	67
Bảng D.1. Tác vụ người dùng thường gặp.....	69

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
B2B	Business-to-Business	Dịch vụ giữa các tổ chức
CI	Continuous Integration	Tích hợp liên tục
CRUD	Create, Read, Update, Delete	Các thao tác dữ liệu cơ bản
CSR	Client-Side Rendering	Kết xuất phía trình duyệt
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
ERD	Entity-Relationship Diagram	Sơ đồ thực thể-quan hệ
FK	Foreign Key	Foreign key
HTTP(S)	Hypertext Transfer Protocol (Secure)	Giao thức truyền siêu văn bản (security)
JWT	JSON Web Token	Mã thông báo web JSON
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NFR	Non-functional Requirement	Yêu cầu phi chức năng
ORM	Object-Relational Mapping	Ánh xạ đối tượng-quan hệ
PK	Primary Key	Khóa chính
PWA	Progressive Web Application	Ứng dụng web tiến bộ
RAG	Retrieval-Augmented Generation	Sinh nội dung có Retrieval-Augmented Generation (RAG)
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
REST	Representational State Transfer	Phong cách kiến trúc dịch vụ web
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SLO	Service Level Objective	Mục tiêu mức dịch vụ
SPA	Single-Page Application	Ứng dụng đơn trang
SSR	Server-Side Rendering	Kết xuất server-side

Từ viết tắt	Tiếng Anh	Ý nghĩa
UI/UX	User Interface / User Experience	Giao diện / trải nghiệm người dùng
UUID	Universally Unique Identifier	Định danh duy nhất phổ quát

MỞ ĐẦU

Trong những năm gần đây, sự phát triển nhanh chóng của công nghệ đã tạo ra ngày càng nhiều thiết bị thông minh với đa dạng dòng sản phẩm, phiên bản, cấu hình phần cứng và tính năng khác nhau. Điện thoại thông minh, máy tính bảng, thiết bị đeo và nhiều nhóm sản phẩm công nghệ khác liên tục được cập nhật, khiến nhu cầu tra cứu, so sánh và tìm hiểu thông tin của người dùng ngày càng tăng. Tuy nhiên, dữ liệu về các thiết bị này hiện được phân tán trên nhiều nguồn như website của nhà sản xuất, nhà bán lẻ, các nền tảng đánh giá hiệu năng và các bài phân tích chuyên môn. Cách đặt tên, đơn vị đo và mức độ chi tiết giữa các nguồn cũng không hoàn toàn thống nhất, gây khó khăn cho quá trình tổng hợp và đối chiếu thông tin. Đây cũng là vấn đề được xác định trong đặc tả của SpecHub khi cùng một thiết bị có thể tồn tại nhiều tên thương mại, biến thể và cấu hình khác nhau.

Trong thực tế, người dùng không chỉ có nhu cầu xem một bảng thông số đơn lẻ mà còn cần hiểu mối liên hệ giữa thiết bị và các thành phần cấu thành như chipset, CPU, GPU, màn hình, camera, pin, bộ nhớ và hệ điều hành. Bên cạnh đó, việc so sánh nhiều thiết bị, đánh giá sự khác biệt giữa các phiên bản, theo dõi giá hoặc tìm kiếm sản phẩm phù hợp với nhu cầu cũng đòi hỏi dữ liệu phải được tổ chức theo một cấu trúc thống nhất. SpecHub được định hướng như một nền tảng tri thức thiết bị có cấu trúc, kết hợp dữ liệu thiết bị, linh kiện, nguồn tham chiếu, nội dung Wiki, công cụ so sánh, hệ thống chấm điểm, theo dõi giá và các chức năng hỗ trợ khác trên cùng một hệ thống.

Sự phát triển của trí tuệ nhân tạo cũng mở ra khả năng khai thác dữ liệu thiết bị theo cách trực quan hơn thông qua các chức năng hỏi đáp, nghiên cứu và khuyến nghị. Tuy nhiên, để bảo đảm tính hữu ích của kết quả, dữ liệu đầu vào cần có cấu trúc rõ ràng và có nguồn tham chiếu. Vì vậy, SpecHub không chỉ tập trung vào việc hiển thị thông tin mà còn chú trọng đến quá trình chuẩn hóa, quản lý nguồn, kiểm duyệt và tổ chức dữ liệu trước khi cung cấp cho người dùng hoặc các chức năng AI. Hệ thống hiện hỗ trợ các hành trình như tìm kiếm, so sánh, khuyến nghị, trợ lý AI, Wiki, danh sách yêu thích và cảnh báo giá.

Xuất phát từ những nhu cầu trên, đề án **“Xây dựng nền tảng web thư viện danh cho thiết bị thông minh”** được thực hiện với sản phẩm là nền tảng **SpecHub**. Đề án hướng đến việc xây dựng một thư viện thiết bị trên nền tảng web, trong đó thông tin được quản lý theo cấu trúc thống nhất, hỗ trợ người dùng tra cứu, tìm kiếm, so sánh và nghiên cứu thiết bị một cách thuận tiện hơn. Đồng thời, hệ thống tạo nền tảng để tích hợp các chức năng mở rộng như Wiki, chấm điểm, theo dõi giá, khuyến nghị và trợ lý AI, qua đó nâng cao khả năng khai thác dữ liệu và hỗ trợ người dùng trong quá trình tìm hiểu, đánh giá và lựa chọn thiết bị.

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, thị trường thiết bị công nghệ phát triển nhanh với số lượng lớn sản phẩm, phiên bản, cấu hình bộ nhớ và thành phần phần cứng khác nhau. Cùng một thiết bị có thể được cung cấp dưới nhiều tên thương mại hoặc biến thể theo từng khu vực, trong khi thông tin kỹ thuật lại được phân tán trên website của nhà sản xuất, nhà bán lẻ, các nền tảng đánh giá và bài viết chuyên môn. Sự khác biệt về cách trình bày, đơn vị đo và mức độ chi tiết giữa các nguồn khiến người dùng gặp nhiều khó khăn trong quá trình tra cứu, đối chiếu và so sánh thông tin.

Bên cạnh đó, nhiều nền tảng hiện nay chủ yếu tập trung vào hoạt động bán hàng hoặc trình bày thông số riêng lẻ, chưa thể hiện đầy đủ mối quan hệ giữa thiết bị với các thành phần như chipset, CPU, GPU, màn hình, camera và các nguồn tham chiếu. Khi ứng dụng trí tuệ nhân tạo vào quá trình tra cứu, việc thiếu một nguồn dữ liệu được chuẩn hóa và có khả năng kiểm chứng cũng có thể làm giảm độ tin cậy của kết quả trả về.

Xuất phát từ những vấn đề trên, đề tài “**Xây dựng nền tảng web thư viện dành cho thiết bị thông minh (SpecHub)**” được lựa chọn nhằm xây dựng một hệ thống dữ liệu thiết bị có cấu trúc, hỗ trợ tra cứu, so sánh, đánh giá và khai thác thông tin từ nhiều nguồn trên một nền tảng thống nhất. Hệ thống đồng thời chú trọng đến khả năng truy nguyên nguồn dữ liệu và cung cấp trợ lý AI dựa trên các dữ liệu đã được kiểm duyệt. Đây cũng là định hướng cốt lõi được xác định trong đặc tả của SpecHub.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu và xây dựng **SpecHub**, một nền tảng web phục vụ tra cứu, so sánh và nghiên cứu thông tin thiết bị công nghệ trên cơ sở dữ liệu có cấu trúc, thống nhất và có khả năng truy nguyên nguồn.

Các mục tiêu cụ thể của đề tài gồm:

- Xây dựng và chuẩn hóa danh mục thiết bị, biến thể sản phẩm, linh kiện phần cứng và các thuộc tính kỹ thuật liên quan.
- Phát triển các chức năng tìm kiếm, lọc, xem thông tin chi tiết, so sánh thiết bị và khuyến nghị sản phẩm theo nhu cầu người dùng.
- Xây dựng trợ lý trí tuệ nhân tạo dựa trên dữ liệu của hệ thống, hỗ trợ truy xuất thông tin và cung cấp câu trả lời có nguồn tham chiếu.
- Xây dựng hệ thống Wiki hỗ trợ quản lý nội dung, lịch sử phiên bản, kiểm duyệt, bình luận và liên kết với nguồn dữ liệu.
- Hỗ trợ các chức năng cá nhân hóa như danh sách yêu thích, theo dõi lịch sử giá, cảnh báo giá và thông báo.
- Xây dựng công cụ hỗ trợ quản trị và biên tập dữ liệu, đồng thời cung cấp khả năng khai thác dữ liệu thông qua API dành cho các đối tác.
- Đảm bảo hệ thống có khả năng kiểm thử, bảo trì và mở rộng khi số lượng dữ liệu, người dùng và chức năng tăng lên.

1.3. Đối tượng và phạm vi

1.3.1. Đối tượng nghiên cứu

- Quy trình quản lý và chuẩn hóa dữ liệu thiết bị, biến thể, linh kiện và thông số kỹ thuật.
- Các chức năng tra cứu, tìm kiếm, so sánh và đánh giá thiết bị trên nền tảng web.
- Ứng dụng trí tuệ nhân tạo RAG trong khai thác dữ liệu và trả lời có nguồn tham chiếu.

1.3.2. Phạm vi nghiên cứu

Về chức năng:

- Người dùng: Tra cứu, tìm kiếm, lọc, so sánh thiết bị, sử dụng Wiki, AI, danh sách yêu thích và cảnh báo giá.
- Quản trị viên: Quản lý danh mục thiết bị, linh kiện, nguồn dữ liệu, nội dung và các chức năng quản trị hệ thống.

Về công nghệ:

- Frontend: Sử dụng Next.js và React để xây dựng giao diện web.
- Backend: Sử dụng NestJS và Fastify để xây dựng API và xử lý nghiệp vụ.
- Cơ sở dữ liệu: Sử dụng PostgreSQL kết hợp Prisma; Redis hỗ trợ phiên và dữ liệu tạm.
- AI và tìm kiếm: RAG, Ollama với qwen2.5:3b, tìm kiếm véc-tơ và Meilisearch. Đặc tả hệ thống có hỗ trợ phương án sinh câu trả lời AI cục bộ qua Ollama.

1.4. Phương pháp thực hiện

Phương pháp khảo sát và phân tích yêu cầu:

- Khảo sát bài toán, xác định đối tượng sử dụng, phạm vi và các yêu cầu của hệ thống.
- Phân tích cấu trúc dữ liệu, chức năng và các quy trình nghiệp vụ của SpecHub.

Phương pháp phân tích và thiết kế hệ thống:

- Sử dụng UML để xây dựng Use Case, biểu đồ tuần tự và quy trình nghiệp vụ.
- Thiết kế mô hình cơ sở dữ liệu và các mối quan hệ giữa các thực thể.

Phương pháp phát triển phần mềm:

- Xây dựng giao diện Web, API, tác vụ nền và các thành phần dùng chung theo kiến trúc đã lựa chọn.
- Tích hợp tìm kiếm và trợ lý AI theo phương pháp RAG.

Phương pháp kiểm thử và đánh giá:

- Thực hiện kiểm thử API, kiểm tra cơ sở dữ liệu và khả năng vận hành của hệ thống.

- Đối chiếu kết quả với yêu cầu, xác định hạn chế và đề xuất hướng phát triển.

1.5. Định hướng giải pháp

Để giải quyết các vấn đề đặt ra, đề án định hướng xây dựng nền tảng web thư viện dành cho thiết bị thông minh theo hướng tập trung hóa dữ liệu, hỗ trợ người dùng tra cứu, tìm kiếm, so sánh và tiếp cận thông tin kỹ thuật của thiết bị một cách thuận tiện.

Giải pháp được định hướng như sau:

- Xây dựng hệ thống dữ liệu có cấu trúc, quản lý thông tin thiết bị theo từng nhóm như dòng sản phẩm, phiên bản, cấu hình và các thành phần phần cứng.
- Phát triển giao diện web trực quan, hỗ trợ người dùng tra cứu thông số, xem chi tiết thiết bị và so sánh giữa nhiều thiết bị.
- Xây dựng hệ thống tìm kiếm nhằm hỗ trợ truy xuất nhanh thiết bị và thông tin kỹ thuật theo nhu cầu của người dùng.
- Tích hợp chức năng đánh giá và chấm điểm dựa trên thông số kỹ thuật, kết quả benchmark và các tiêu chí được xác định cho từng nhóm thành phần.
- Ứng dụng AI để hỗ trợ tìm kiếm, tổng hợp và giải đáp các câu hỏi liên quan đến dữ liệu thiết bị có trong hệ thống.
- Xây dựng khu vực quản trị phục vụ việc quản lý, cập nhật và kiểm soát dữ liệu trước khi cung cấp đến người dùng.
- Thiết kế hệ thống theo hướng module hóa, tạo điều kiện thuận lợi cho việc bảo trì, mở rộng thêm loại thiết bị, thông số và chức năng trong tương lai.

1.6. Khảo sát hiện trạng

Qua quá trình khảo sát, có thể nhận thấy thông tin về thiết bị thông minh hiện nay được phân bố trên nhiều nguồn khác nhau như website của nhà sản xuất, nhà bán lẻ, cơ sở dữ liệu đánh giá hiệu năng và các bài phân tích kỹ thuật. Mỗi nguồn có cách tổ chức và mức độ chi tiết riêng, dẫn đến khó khăn trong việc tổng hợp, đối chiếu và so sánh dữ liệu giữa các thiết bị.

1.6.1. Đánh giá các nền tảng tra cứu thiết bị hiện nay

Các nguồn thông tin thiết bị hiện nay có thể chia thành một số nhóm chính:

- **Website của nhà sản xuất:** cung cấp thông tin chính thức về sản phẩm nhưng thường tập trung vào từng thương hiệu và chưa thuận tiện khi cần so sánh thiết bị giữa nhiều hãng.
- **Website thương mại điện tử:** hỗ trợ tìm kiếm sản phẩm, giá bán và mua hàng nhưng thông số kỹ thuật chủ yếu phục vụ mục đích thương mại.
- **Nền tảng đánh giá và thông số:** cung cấp nhiều dữ liệu kỹ thuật và kết quả benchmark, tuy nhiên dữ liệu giữa các nguồn có thể khác nhau về cách đặt tên, đơn vị và mức độ chi tiết.
- **Bài viết và nguồn đánh giá chuyên môn:** cung cấp nhiều thông tin thực tế nhưng dữ liệu thường tồn tại dưới dạng nội dung bài viết, khó khai thác trực tiếp cho việc lọc và so sánh.

Từ thực trạng trên, cần có một nền tảng tổ chức thông tin thiết bị theo cấu trúc thống nhất, liên kết thiết bị với các thành phần phần cứng, nguồn dữ liệu và các thông tin liên quan.

1.6.2. Phân tích các vai trò tham gia hệ thống

Hệ thống SpecHub xác định các nhóm người dùng chính như sau:

- **Khách truy cập:** tra cứu thông tin, tìm kiếm, so sánh thiết bị và nhận các gợi ý phù hợp.
- **Người dùng:** sử dụng các chức năng cá nhân hóa như danh sách yêu thích, cảnh báo giá, thông báo và quản lý hồ sơ.
- **Biên tập viên và kiểm duyệt viên:** quản lý nội dung, cập nhật dữ liệu thiết bị, nguồn tham chiếu và kiểm duyệt các nội dung trước khi xuất bản.
- **Quản trị viên:** quản lý người dùng, phân quyền, danh mục thiết bị, dữ liệu hệ thống và các chức năng quản trị.
- **Đối tác:** khai thác một phần dữ liệu của hệ thống thông qua API theo quyền và hạn mức được cấp.

1.6.3. Khảo sát quy trình nghiệp vụ chính

Các quy trình nghiệp vụ chính của hệ thống gồm:

- **Tra cứu thiết bị:** người dùng tìm kiếm hoặc lọc danh sách thiết bị, xem thông tin chi tiết và các thành phần phần cứng liên quan.
- **So sánh và khuyến nghị:** người dùng lựa chọn nhiều thiết bị để so sánh hoặc cung cấp nhu cầu và ngân sách để hệ thống đưa ra gợi ý phù hợp.
- **Biên tập và xuất bản dữ liệu:** dữ liệu được tạo hoặc thu thập sẽ được đưa vào quá trình kiểm tra, biên tập và kiểm duyệt trước khi xuất bản. Dữ liệu thu thập từ bên ngoài không được đưa trực tiếp vào danh mục công khai.
- **Hỏi đáp với trợ lý AI:** người dùng gửi câu hỏi, hệ thống truy xuất dữ liệu liên quan trong kho tri thức và sử dụng dữ liệu đó làm cơ sở để tạo câu trả lời có nguồn tham chiếu.
- **Quản lý nội dung Wiki:** bài viết được tạo và chỉnh sửa theo phiên bản, sau đó được kiểm duyệt trước khi xuất bản cho người dùng.

1.6.4. Tổng hợp các chức năng chính

Từ quá trình khảo sát và phân tích nghiệp vụ, hệ thống được xác định gồm các nhóm chức năng chính:

- Đăng ký, đăng nhập và quản lý tài khoản người dùng.
- Quản lý danh mục thiết bị, biến thể và các thành phần phần cứng.

- Tìm kiếm, lọc và xem thông tin chi tiết thiết bị.
- So sánh và khuyến nghị thiết bị theo nhu cầu.
- Tra cứu thông tin bằng trợ lý AI.
- Quản lý và khai thác nội dung Wiki.
- Quản lý danh sách yêu thích, lịch sử giá và cảnh báo giá.
- Quản lý nguồn dữ liệu, nội dung và quy trình biên tập.
- Quản trị người dùng, phân quyền và dữ liệu hệ thống.
- Cung cấp API phục vụ khai thác dữ liệu cho đối tác.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

1.1. Công nghệ sử dụng phát triển Web

Đề tài “**Xây dựng nền tảng Web thư viện dành cho thiết bị thông minh**” hướng đến xây dựng một hệ thống có khả năng quản lý dữ liệu thiết bị với cấu trúc phức tạp, hỗ trợ tra cứu, tìm kiếm, so sánh, đánh giá, quản lý nội dung và khai thác dữ liệu bằng trí tuệ nhân tạo. Do đó, hệ thống cần một nền tảng công nghệ có khả năng tổ chức mã nguồn rõ ràng, xử lý dữ liệu có quan hệ chặt chẽ, hỗ trợ giao tiếp giữa các thành phần và thuận lợi cho việc mở rộng trong quá trình phát triển.

SpecHub được xây dựng chủ yếu trên hệ sinh thái **TypeScript** và **Node.js**. Ứng dụng Web sử dụng **React** và **Next.js**; Backend sử dụng **NestJS** kết hợp **Fastify**; dữ liệu được lưu trữ trong **PostgreSQL** và truy cập thông qua **Prisma ORM**. Bên cạnh đó, hệ thống sử dụng **Redis** cho dữ liệu phiên và dữ liệu tạm, **Meilisearch** hỗ trợ tìm kiếm danh mục, **pgvector** phục vụ dữ liệu véc-tơ và **Ollama** làm nền tảng thực thi mô hình ngôn ngữ cho chức năng AI. Các thành phần được tổ chức trong cùng một monorepo nhằm chia sẻ kiểu dữ liệu, cấu hình và các thư viện dùng chung.

2.1.1. Ngôn ngữ lập trình *TypeScript*

TypeScript là ngôn ngữ lập trình được xây dựng trên JavaScript và bổ sung cơ chế kiểm tra kiểu tĩnh. Trình biên dịch TypeScript có khả năng kiểm tra tính phù hợp của kiểu dữ liệu trước khi chương trình được thực thi, sau đó chuyển mã nguồn về JavaScript để chạy trong trình duyệt hoặc môi trường Node.js.

Trong SpecHub, TypeScript là ngôn ngữ chủ đạo được sử dụng cho ứng dụng Web, Backend và phần lớn các package dùng chung. Phiên bản được khai báo trong dự án là **TypeScript 5.7.2**.

a) Hệ thống kiểu dữ liệu

Một đặc điểm quan trọng của TypeScript là khả năng khai báo kiểu cho biến, tham số, giá trị trả về và các cấu trúc dữ liệu. Cơ chế này cho phép phát hiện sớm những trường

hợp sử dụng sai kiểu hoặc truy cập thuộc tính không tồn tại trước khi chương trình được đưa vào thực thi.

Đặc điểm trên phù hợp với SpecHub vì dữ liệu của hệ thống không chỉ bao gồm các thông tin sản phẩm đơn giản mà còn có nhiều nhóm thực thể như mẫu thiết bị, biến thể, chipset, CPU, GPU, NPU, modem, camera, màn hình, pin, bộ nhớ, hệ điều hành, benchmark và nguồn tham chiếu. Việc định nghĩa cấu trúc dữ liệu rõ ràng giúp hạn chế sai lệch khi cùng một đối tượng được sử dụng xuyên suốt giữa giao diện, API và tầng dữ liệu.

b) Interface và kiểu dữ liệu dùng chung

TypeScript hỗ trợ `interface`, `type`, `generic` và các cơ chế xây dựng kiểu dữ liệu có cấu trúc. Điều này cho phép mô tả rõ hợp đồng dữ liệu giữa các thành phần của ứng dụng.

Trong kiến trúc SpecHub, monorepo có package `types` dùng cho các kiểu dữ liệu chung và `api-client` phục vụ việc giao tiếp với API. Nhờ đó, một số cấu trúc dữ liệu có thể được thống nhất giữa các ứng dụng thay vì được khai báo lại ở nhiều nơi.

c) Vai trò của TypeScript trong hệ thống

Việc sử dụng TypeScript trên cả Frontend và Backend mang lại các lợi ích chính:

- Giảm lỗi do không thống nhất kiểu dữ liệu giữa Web và API.
- Hỗ trợ kiểm tra lỗi ngay trong quá trình phát triển.
- Tăng khả năng gợi ý và điều hướng mã nguồn trong môi trường lập trình.
- Thuận lợi khi tái cấu trúc hoặc mở rộng mô hình dữ liệu.
- Cho phép chia sẻ các kiểu dữ liệu giữa những package trong monorepo.
- Tạo sự thống nhất về ngôn ngữ lập trình giữa phần giao diện và phần xử lý nghiệp vụ.

Đối với một nền tảng có nhiều miền dữ liệu như SpecHub, việc sử dụng TypeScript giúp duy trì tính nhất quán của mã nguồn khi số lượng module và quan hệ giữa các đối tượng tăng lên.

2.1.2. Thư viện React và Framework Next.js

Phần giao diện của SpecHub được xây dựng trên **React 19.x** và **Next.js 15.1.3**. React đảm nhiệm việc tổ chức giao diện theo các thành phần, trong khi Next.js cung cấp cấu trúc

ứng dụng Web, hệ thống định tuyến và cơ chế xử lý kết hợp giữa phía máy chủ và phía trình duyệt.

a) Thư viện React

React là thư viện dùng để xây dựng giao diện người dùng dựa trên các **component**. Mỗi component có thể quản lý một phần giao diện và có trạng thái riêng; nhiều component có thể kết hợp với nhau để hình thành một trang hoàn chỉnh. React cũng cung cấp cơ chế quản lý *state*, cho phép giao diện được cập nhật khi dữ liệu hoặc thao tác người dùng thay đổi.

Trong SpecHub, cách tổ chức theo component phù hợp với đặc điểm của trang thông tin thiết bị. Một trang chi tiết có thể bao gồm nhiều khối độc lập như:

- Thông tin tổng quan.
- Hình ảnh và video.
- Cấu hình chipset, CPU và GPU.
- Màn hình.
- Camera.
- Pin và sạc.
- Bộ nhớ và lưu trữ.
- Hệ điều hành.
- Benchmark.
- Điểm đánh giá.
- Nguồn tham chiếu.

Các thành phần này có thể được xây dựng độc lập và tái sử dụng trên nhiều mẫu thiết bị khác nhau.

Mã nguồn SpecHub đã tổ chức các thành phần giao diện chuyên biệt như `device-spec-modules` để trình bày thông số theo nhóm, `device-scorecard` để hiển thị điểm, `device-media-gallery` cho dữ liệu đa phương tiện, `device-recommender` cho chức năng khuyến nghị và các thành phần dành cho Wiki, AI và Catalog Studio.

b) Framework Next.js

Next.js là framework phát triển ứng dụng Web dựa trên React. SpecHub sử dụng **App Router**, là cơ chế định tuyến dựa trên cấu trúc hệ thống tệp và hỗ trợ Server Components, Client Components, layout và các tính năng kết xuất của React.

Trong App Router, các trang và layout mặc định có thể được xử lý dưới dạng Server Components. Khi giao diện cần trạng thái, sự kiện người dùng hoặc API của trình duyệt, Client Components được sử dụng cho phần tương tác tương ứng.

SpecHub áp dụng mô hình này theo hướng:

- Thành phần phía máy chủ tải dữ liệu ban đầu khi phù hợp.
- Thành phần phía máy khách xử lý các thao tác tương tác.
- Trạng thái đăng nhập được quản lý thông qua `AuthProvider`.
- Dữ liệu bất đồng bộ phía Client được quản lý thông qua `QueryProvider`.
- Các trang lỗi, đang tải, không tìm thấy và ngoại tuyến được tổ chức riêng.

Hệ thống Web bao gồm các tuyến phục vụ nhiều nhóm chức năng như danh mục thiết bị, chi tiết thiết bị, phân cứng, tìm kiếm, so sánh, khuyến nghị, AI, Wiki, danh sách yêu thích, cảnh báo, thông báo và quản trị.

c) Server Components và Client Components

Server Components phù hợp với những phần giao diện chủ yếu phục vụ việc tải và hiển thị dữ liệu. Việc xử lý tại phía máy chủ giúp giảm lượng logic cần thực thi trực tiếp tại trình duyệt và cho phép lấy dữ liệu trước khi giao diện được gửi đến người dùng. Ngược lại, Client Components phù hợp với những chức năng cần `state`, xử lý sự kiện hoặc các API chỉ tồn tại trong trình duyệt.

Đối với SpecHub, sự kết hợp này phù hợp với hai nhóm nhu cầu khác nhau. Các trang thông tin thiết bị chủ yếu tập trung vào trình bày dữ liệu, trong khi bộ lọc, bảng so sánh, biểu mẫu quản trị, danh sách yêu thích hay các chức năng tương tác cần được xử lý nhiều hơn ở phía Client.

d) TanStack Query

SpecHub sử dụng **TanStack Query 5.62.10** để quản lý trạng thái dữ liệu nhận từ Backend ở phía Client.

TanStack Query tập trung vào các nhiệm vụ lấy dữ liệu, lưu cache, đồng bộ và cập nhật dữ liệu phía máy chủ trong ứng dụng Web. Dữ liệu truy vấn được tổ chức thông qua cache

và các `query key`, giúp các thành phần có thể tái sử dụng kết quả đã tải hoặc yêu cầu cập nhật lại dữ liệu khi cần.

Trong SpecHub, TanStack Query hỗ trợ những trường hợp như:

- Tải danh sách thiết bị.
- Thực hiện tìm kiếm và lọc.
- Lấy dữ liệu chi tiết.
- Quản lý trạng thái đang tải hoặc lỗi.
- Làm mới dữ liệu sau thao tác tạo hoặc cập nhật.
- Hạn chế việc tự xây dựng lại cơ chế quản lý yêu cầu API ở từng component.

e) Tailwind CSS

Tailwind CSS được sử dụng để hỗ trợ xây dựng giao diện của SpecHub. Cấu hình Tailwind nằm trong package cấu hình dùng chung của monorepo.

Tailwind cung cấp hệ thống các lớp tiện ích để xác định bố cục, kích thước, khoảng cách và trạng thái hiển thị trực tiếp trên thành phần giao diện. Các utility cũng có thể được kết hợp với breakpoint để thay đổi cách hiển thị theo kích thước màn hình.

Đối với SpecHub, cách tiếp cận này hỗ trợ xây dựng giao diện nhất quán giữa các khu vực như trang danh mục, trang chi tiết, bảng so sánh, Wiki và giao diện quản trị, đồng thời thuận lợi khi thiết kế giao diện thích ứng trên nhiều kích thước màn hình.

2.1.3. Framework NestJS và Fastify

Backend của SpecHub được phát triển bằng **NestJS 11.0.6** và sử dụng **Fastify 5.2.0** làm HTTP adapter.

a) Framework NestJS

NestJS là framework dành cho Node.js, hỗ trợ TypeScript và tổ chức ứng dụng theo các thành phần như Module, Controller và Provider. Controller có nhiệm vụ tiếp nhận yêu cầu từ Client, trong khi các Service hoặc Provider đảm nhiệm phần xử lý nghiệp vụ phức tạp hơn. NestJS cũng tích hợp cơ chế Dependency Injection để quản lý sự phụ thuộc giữa các thành phần.

Trong một module của NestJS, các thành phần chính có thể bao gồm:

- **Controller:** tiếp nhận HTTP Request và trả về Response.
- **Service:** xử lý logic nghiệp vụ.
- **Provider:** cung cấp các dịch vụ hoặc tài nguyên cần được tiêm vào thành phần khác.
- **DTO:** mô tả cấu trúc dữ liệu được truyền vào API.
- **Pipe:** kiểm tra hoặc chuyển đổi dữ liệu.
- **Guard:** kiểm tra điều kiện truy cập.
- **Exception Filter:** chuẩn hóa quá trình xử lý lỗi.

Kiến trúc này phù hợp với SpecHub vì Backend gồm nhiều miền nghiệp vụ tương đối độc lập như xác thực, thiết bị, biến thể, linh kiện, Wiki, nguồn dữ liệu, AI, cảnh báo, thông báo và quản trị.

b) Controller và Service

Controller trong NestJS chịu trách nhiệm định tuyến các yêu cầu đến đúng thành phần xử lý. Framework cung cấp các decorator tương ứng với các phương thức HTTP như `@Get()`, `@Post()`, `@Put()`, `@Patch()` và `@Delete()`.

Logic xử lý chính không được đặt toàn bộ tại Controller mà được chuyển đến Service hoặc Provider. Điều này giúp phân tách phần giao tiếp HTTP khỏi nghiệp vụ và làm cho mã nguồn dễ kiểm thử, bảo trì và mở rộng hơn.

Trong SpecHub, Controller chuyển yêu cầu tới lớp Service; Service tiếp tục kiểm tra quyền sở hữu, trạng thái nghiệp vụ và truy cập Prisma, Redis hoặc các dịch vụ ngoài tùy theo từng chức năng.

c) Dependency Injection

Dependency Injection là cơ chế trong đó việc khởi tạo và cung cấp các phụ thuộc được giao cho container của framework thay vì mỗi đối tượng tự tạo phụ thuộc cho mình. NestJS xây dựng cơ chế DI trực tiếp trong lõi của framework.

Trong SpecHub, cơ chế này giúp các Service có thể sử dụng các thành phần như lớp truy cập cơ sở dữ liệu, dịch vụ xác thực, cấu hình hoặc các dịch vụ tích hợp mà không cần tự quản lý toàn bộ vòng đời của những đối tượng đó.

d) Fastify

Fastify là framework HTTP dành cho Node.js. Framework cung cấp cơ chế routing, request lifecycle, validation và serialization cho quá trình xử lý yêu cầu Web.

Trong SpecHub, Fastify đóng vai trò tiếp nhận yêu cầu HTTP trước khi yêu cầu được xử lý bởi các lớp của NestJS. Luồng xử lý chính của API bao gồm:

- Fastify tiếp nhận yêu cầu và gán mã định danh.
- Helmet và CORS áp dụng các cấu hình nền.
- Cơ chế giới hạn tần suất kiểm tra Request.
- JWT Guard kiểm tra xác thực.
- Roles Guard kiểm tra vai trò.
- ValidationPipe kiểm tra dữ liệu đầu vào.
- Controller chuyển yêu cầu đến Service.
- Service xử lý nghiệp vụ và truy cập các thành phần dữ liệu.
- Bộ lọc lỗi chuẩn hóa phản hồi.

Sự kết hợp giữa NestJS và Fastify giúp Backend vừa duy trì cách tổ chức module rõ ràng, vừa có một lớp HTTP chuyên trách cho quá trình tiếp nhận và xử lý Request.

2.1.4. Hệ quản trị cơ sở dữ liệu *PostgreSQL* và *Prisma ORM*

Đối với SpecHub, dữ liệu là thành phần trung tâm của hệ thống. Nền tảng không chỉ lưu thông tin cơ bản của sản phẩm mà còn phải biểu diễn quan hệ giữa mẫu thiết bị, biến thể, linh kiện, thông số, benchmark, nội dung Wiki, nguồn tham chiếu và các dữ liệu liên quan. Vì vậy, **PostgreSQL** được sử dụng làm hệ quản trị cơ sở dữ liệu chính, kết hợp với **Prisma ORM** ở tầng ứng dụng.

a) PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ hỗ trợ SQL, bảng dữ liệu, các phép JOIN, khóa ngoại, transaction và nhiều tính năng mở rộng.

Mô hình quan hệ phù hợp với SpecHub do dữ liệu có nhiều mối liên kết. Ví dụ:

- Một hãng có nhiều dòng sản phẩm.
- Một dòng sản phẩm có nhiều mẫu thiết bị.
- Một mẫu thiết bị có thể có nhiều biến thể.
- Một biến thể liên kết với nhiều thành phần phụ tùng.

- Một linh kiện có thể được sử dụng bởi nhiều thiết bị.
- Một thiết bị có thể liên kết với nhiều nguồn và dữ liệu benchmark.

Việc tách dữ liệu thành các thực thể và bảng liên kết giúp giảm lặp dữ liệu, đồng thời cho phép truy vấn theo nhiều chiều.

b) Khóa chính, khóa ngoại và tính toàn vẹn tham chiếu

PostgreSQL hỗ trợ các ràng buộc như Primary Key, Unique và Foreign Key. Foreign Key yêu cầu giá trị ở bảng tham chiếu phải liên kết đến bản ghi hợp lệ của bảng được tham chiếu, qua đó duy trì tính toàn vẹn giữa các bảng có quan hệ.

Đối với SpecHub, cơ chế này giúp bảo đảm một cấu hình thiết bị không tham chiếu đến linh kiện không tồn tại, benchmark được liên kết đúng đối tượng và các bảng nối duy trì mối quan hệ hợp lệ giữa những thực thể trong catalog.

c) Transaction

Transaction cho phép gom nhiều thao tác cơ sở dữ liệu thành một đơn vị xử lý. Các thay đổi có thể được xác nhận bằng `COMMIT` hoặc hủy bằng `ROLLBACK` khi một phần của quá trình không thể hoàn thành.

Đặc điểm này quan trọng với các quy trình cần cập nhật nhiều bảng cùng lúc. Trong SpecHub, quá trình xuất bản dữ liệu từ Catalog Studio sử dụng giao dịch nhằm tránh tạo trạng thái dữ liệu chỉ hoàn thành một phần khi có lỗi xảy ra.

d) Index

Index là cơ chế hỗ trợ cơ sở dữ liệu xác định và truy xuất các bản ghi phù hợp nhanh hơn so với việc quét toàn bộ bảng, nhưng đồng thời làm tăng chi phí lưu trữ và cập nhật, do đó cần được áp dụng phù hợp với đặc điểm truy vấn.

Đối với SpecHub, các trường thường xuyên được sử dụng khi tìm kiếm, lọc hoặc liên kết dữ liệu có thể được lập chỉ mục nhằm hỗ trợ những truy vấn trên danh mục thiết bị có số lượng bản ghi lớn.

e) Prisma ORM

Prisma là ORM dành cho Node.js và TypeScript, cung cấp Prisma Client có kiểu dữ liệu, Prisma Migrate và hệ thống Prisma Schema để mô tả mô hình dữ liệu.

Trong SpecHub, Prisma đóng vai trò trung gian giữa NestJS và PostgreSQL. Các model và quan hệ được khai báo trong `schema.prisma`, sau đó Prisma Client được sử dụng tại Backend để thực hiện truy vấn.

Prisma hỗ trợ:

- Khai báo model dữ liệu.
- Xác định quan hệ giữa các model.
- Sinh Client có kiểu cho TypeScript.
- Thực hiện các thao tác tạo, đọc, cập nhật và xóa.
- Quản lý thay đổi schema thông qua migration.
- Đồng bộ cấu trúc dữ liệu giữa tầng ứng dụng và cơ sở dữ liệu.

Dự án sử dụng **Prisma 6.1.0**. Lược đồ hiện tại có **139 model**, bao phủ các nhóm danh mục thiết bị, phần cứng, benchmark, Wiki, AI, người dùng, cảnh báo và các chức năng liên quan.

f) pgvector

SpecHub sử dụng phần mở rộng **pgvector** trên PostgreSQL để hỗ trợ lưu trữ và truy vấn dữ liệu véc-tơ. **pgvector** bổ sung kiểu vector cùng các phép đo khoảng cách như L2, inner product và cosine distance; đồng thời hỗ trợ tìm kiếm lân cận chính xác hoặc gần đúng.

Trong dự án, PostgreSQL được cấu hình cùng các extension `pgvector`, `pg_trgm` và `unaccent`.

Dữ liệu véc-tơ tạo nền tảng cho các chức năng tìm kiếm ngữ nghĩa và truy xuất thông tin phục vụ hệ thống AI, trong khi dữ liệu thiết bị có cấu trúc vẫn được quản lý bằng mô hình quan hệ.

2.1.5. Redis

Redis là hệ thống lưu trữ dữ liệu hỗ trợ nhiều cấu trúc như String, Hash, List, Set, Sorted Set và các dạng dữ liệu khác. Redis thường phù hợp với các dữ liệu cần truy cập nhanh hoặc có vòng đời ngắn.

Trong SpecHub, Redis được sử dụng chủ yếu cho **phiên đăng nhập có khả năng thu hồi và dữ liệu tạm**. Redis là thành phần cần thiết đối với các luồng đã xác thực của hệ thống.

a) Quản lý dữ liệu phiên

Thông tin phiên đăng nhập có đặc điểm khác dữ liệu người dùng. Hồ sơ người dùng cần được lưu lâu dài trong PostgreSQL, trong khi phiên chỉ có hiệu lực trong một khoảng thời gian và phải có khả năng bị thu hồi.

SpecHub sử dụng access token có thời hạn ngắn; refresh token được liên kết với định danh phiên trong Redis. Khi cần làm mới thông tin xác thực, Backend kiểm tra trạng thái phiên trước khi tiếp tục xử lý.

b) Vai trò của Redis trong hệ thống

Việc tách dữ liệu phiên và dữ liệu tạm khỏi PostgreSQL giúp phân định rõ hai loại dữ liệu:

- PostgreSQL lưu thông tin cần duy trì lâu dài.
- Redis quản lý trạng thái tạm thời của ứng dụng.
- Phiên có thể được thu hồi khi người dùng đăng xuất hoặc bị vô hiệu hóa.
- Các luồng xác thực không cần lưu toàn bộ trạng thái ngắn hạn vào các bảng nghiệp vụ chính.

Redis vì vậy được sử dụng như một thành phần hỗ trợ cho PostgreSQL chứ không thay thế cơ sở dữ liệu chính.

2.1.6. Công cụ tìm kiếm Meilisearch

SpecHub là nền tảng thư viện thiết bị nên chức năng tìm kiếm có vai trò quan trọng. Người dùng có thể bắt đầu từ tên thiết bị, tên thương mại, biến thể hoặc một thành phần phần cứng thay vì duyệt toàn bộ cấu trúc danh mục.

Hệ thống sử dụng **Meilisearch** như một thành phần hỗ trợ tăng cường tìm kiếm. Theo cấu hình dự án, Meilisearch không phải thành phần bắt buộc để hệ thống cơ bản hoạt động, nhưng có thể được kích hoạt để cải thiện khả năng tìm kiếm danh mục.

a) Tìm kiếm toàn văn

Meilisearch lập chỉ mục dữ liệu phục vụ tìm kiếm thay vì phụ thuộc hoàn toàn vào các truy vấn trực tiếp trên bảng quan hệ. Điều này phù hợp với trường hợp cần tìm tên sản phẩm, tên thay thế hoặc các từ khóa liên quan trong danh mục.

Yêu cầu tìm kiếm của SpecHub bao gồm tìm theo tên, slug, alias và dữ liệu liên quan, đồng thời hỗ trợ lọc và phân trang kết quả.

b) Khả năng xử lý lỗi nhập liệu

Meilisearch hỗ trợ **typo tolerance**, cho phép kết quả vẫn có thể được tìm thấy khi từ khóa của người dùng có một số lỗi chính tả. Cơ chế này có thể được cấu hình theo index, từ khóa hoặc thuộc tính.

Tính năng này có ý nghĩa đối với thư viện thiết bị vì tên thương mại, mã thiết bị hoặc linh kiện thường bao gồm nhiều ký tự và số thập hệ, làm tăng khả năng người dùng nhập thiếu hoặc sai một phần từ khóa.

c) Kết hợp với PostgreSQL

PostgreSQL vẫn là nguồn dữ liệu chính của SpecHub. Meilisearch đóng vai trò lập chỉ mục và phục vụ tìm kiếm, trong khi dữ liệu gốc và các quan hệ giữa thực thể tiếp tục được quản lý trong PostgreSQL.

Cách phân chia này giúp hệ thống sử dụng đúng thế mạnh của từng thành phần: cơ sở dữ liệu quan hệ bảo đảm tính nhất quán của dữ liệu, còn công cụ tìm kiếm phục vụ trải nghiệm tra cứu nhanh và linh hoạt hơn.

2.1.7. Nền tảng Ollama và mô hình Qwen2.5:3B

Bên cạnh các chức năng tra cứu theo giao diện thông thường, SpecHub tích hợp trợ lý AI để người dùng có thể khai thác dữ liệu thiết bị thông qua ngôn ngữ tự nhiên. Trong đó, **Ollama** được sử dụng để vận hành mô hình **Qwen2.5:3B**.

a) Nền tảng Ollama

Ollama cung cấp API để ứng dụng gửi nội dung đến mô hình ngôn ngữ và nhận kết quả sinh văn bản. API Chat nhận tên mô hình cùng lịch sử hội thoại và có thể trả kết quả theo dạng luồng. Ollama cũng cung cấp endpoint riêng cho việc tạo embedding.

Trong kiến trúc SpecHub, `ollama` là một trong các giá trị cấu hình của `AI_PROVIDER`. Khi lựa chọn Ollama, mô hình được gọi thông qua giao diện HTTP; luồng sinh câu trả lời được thực hiện trong Backend API.

Việc sử dụng Ollama giúp phân xử lý mô hình được tách khỏi logic nghiệp vụ chính. Backend chuẩn bị nội dung và ngữ cảnh, gửi yêu cầu tới Ollama, sau đó xử lý kết quả trước khi trả về giao diện.

b) Mô hình Qwen2.5:3B

Qwen2.5 là họ mô hình ngôn ngữ do Qwen phát triển với nhiều kích thước, trong đó có biến thể **3B**. Qwen2.5 tập trung cải thiện khả năng làm theo chỉ dẫn, xử lý dữ liệu có cấu trúc và hỗ trợ nhiều ngôn ngữ, bao gồm tiếng Việt.

Ollama cung cấp trực tiếp biến thể `qwen2.5:3b` trong thư viện mô hình.

Trong SpecHub, mô hình không được sử dụng như nguồn dữ liệu chính về thông số thiết bị. Dữ liệu làm căn cứ vẫn được lấy từ kho dữ liệu đã được hệ thống quản lý và kiểm duyệt; mô hình đảm nhiệm việc tổng hợp và diễn đạt thông tin thành câu trả lời tự nhiên.

c) Vai trò trong chức năng AI

Luồng xử lý tổng quát có thể biểu diễn như sau:

Người dùng → Giao diện Web → Backend API → Truy xuất dữ liệu liên quan → Xây dựng ngữ cảnh → Ollama/Qwen2.5:3B → Kiểm tra kết quả → Trả câu trả lời

Kho dữ liệu có thể được sử dụng cho AI bao gồm các mẫu và biến thể thiết bị đã xuất bản, dữ liệu phân cứng, benchmark, điểm đánh giá, Wiki, dữ liệu thô đã được duyệt và nguồn tham chiếu công khai. Các dữ liệu riêng tư như mật khẩu, phiên, khóa API, bản ghi thanh toán hoặc bản nháp chưa duyệt không được đưa vào phạm vi trả lời công khai.

Cơ chế truy xuất và RAG sẽ được trình bày riêng ở phần cơ sở lý thuyết về trí tuệ nhân tạo nhằm làm rõ quá trình tạo embedding, tìm kiếm ngữ nghĩa, xây dựng ngữ cảnh và sinh câu trả lời.

2.1.8. Phương pháp tiếp cận RESTful API

SpecHub sử dụng API dựa trên HTTP để thực hiện giao tiếp giữa giao diện Web và Backend. Việc tách giao diện ra khỏi tầng xử lý giúp Frontend tập trung vào trình bày và tương tác, trong khi Backend chịu trách nhiệm xử lý nghiệp vụ, xác thực, phân quyền và truy cập dữ liệu.

NestJS cung cấp các decorator tương ứng với các phương thức HTTP tiêu chuẩn như GET, POST, PUT, PATCH và DELETE để định nghĩa endpoint.

Trong SpecHub, API sử dụng tiền tố toàn cục `/api` và phiên bản mặc định `/v1`. Các yêu cầu được kiểm tra dữ liệu đầu vào, xử lý lỗi tập trung và gắn mã định danh Request.

a) Phương thức GET

GET được sử dụng chủ yếu cho các thao tác lấy dữ liệu mà không tạo thay đổi trực tiếp đến tài nguyên.

Ví dụ trong SpecHub:

GET /api/v1/device-models

GET /api/v1/device-models/:slug

GET /api/v1/device-variants

GET /api/v1/device-variants/compare

GET /api/v1/hardware/:kind/:slug

GET /api/v1/search

Các endpoint này phục vụ việc đọc danh mục, tra cứu thông tin thiết bị, linh kiện, so sánh và tìm kiếm.

b) Phương thức POST

POST được sử dụng để tạo tài nguyên mới hoặc khởi tạo một thao tác nghiệp vụ.

Các tình huống tiêu biểu gồm:

- Đăng ký và đăng nhập.
- Tạo bản nháp dữ liệu.
- Tạo danh sách hoặc cảnh báo.
- Gửi câu hỏi đến AI.
- Thực hiện một số hành động xử lý đặc thù.

Nhóm API AI của SpecHub sử dụng POST cho các chức năng hỏi đáp, hội thoại, khuyến nghị và nghiên cứu dữ liệu phần cứng.

c) Phương thức PATCH và PUT

PATCH phù hợp với trường hợp cập nhật một phần tài nguyên, chẳng hạn thay đổi một số thuộc tính hoặc trạng thái mà không thay thế toàn bộ bản ghi.

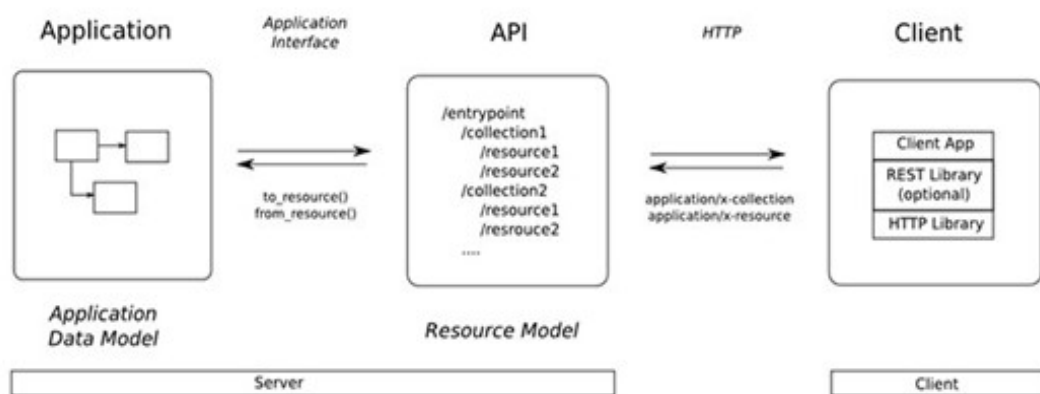
PUT được sử dụng trong những trường hợp API yêu cầu cập nhật tài nguyên theo một định danh cụ thể. Trong SpecHub, phương thức PUT được sử dụng trong luồng tải nội dung media của Catalog Studio.

d) Phương thức DELETE

DELETE được sử dụng khi người dùng hoặc quản trị viên yêu cầu loại bỏ một tài nguyên mà họ có quyền thao tác. Trước khi thực hiện, Backend vẫn phải trải qua quá trình xác thực, phân quyền và kiểm tra điều kiện nghiệp vụ.

e) Giao tiếp giữa các thành phần

Luồng giao tiếp chính của hệ thống có thể mô tả:



Hình 2. 1: Phương thức RESful API

Kiến trúc hiện tại của SpecHub xác định Web giao tiếp với API, sau đó API đóng vai trò trung tâm kết nối cơ sở dữ liệu, Redis, công cụ tìm kiếm, AI và các dịch vụ tích hợp khác.

Cách tổ chức API này cũng tạo điều kiện để dữ liệu SpecHub được cung cấp cho những hệ thống khác thông qua API B2B mà không phụ thuộc vào giao diện Web.

2.1.9. Cơ chế xác thực JWT và phân quyền người dùng

SpecHub có nhiều nhóm người dùng với quyền hạn khác nhau như người dùng thông thường, biên tập viên, kiểm duyệt viên, quản trị viên và đối tác sử dụng API. Do đó, Backend cần xác định được danh tính của người gửi yêu cầu và kiểm tra quyền trước khi cho phép truy cập các tài nguyên được bảo vệ.

a) JSON Web Token

JSON Web Token (JWT) là một chuẩn biểu diễn các claim theo dạng nhỏ gọn và an toàn cho việc truyền qua URL. Các claim được biểu diễn trong đối tượng JSON và có thể được bảo vệ về tính toàn vẹn bằng chữ ký hoặc mã xác thực thông điệp.

Một JWT thường được biểu diễn bởi ba phần:

- **Header:** chứa thông tin mô tả loại token và thuật toán.
- **Payload:** chứa các claim.
- **Signature:** dùng để kiểm tra tính toàn vẹn và nguồn tạo token.

Trong SpecHub, API mặc định sử dụng **Bearer JWT** đối với các endpoint cần xác thực; các endpoint công khai phải được khai báo riêng. API dành cho đối tác B2B sử dụng cơ chế `X-API-Key`.

b) Kết hợp JWT với phiên Redis

SpecHub không chỉ dựa vào token mà còn duy trì trạng thái phiên thông qua Redis.

Access token có thời hạn ngắn, trong khi refresh token được liên kết với định danh phiên lưu trong Redis. Nhờ đó, hệ thống có thể kiểm tra hoặc thu hồi phiên khi cần thiết.

Quy trình cơ bản gồm:

- Người dùng gửi thông tin đăng nhập.
- Backend xác minh tài khoản và mật khẩu.
- Redis lưu phiên có thời hạn.
- Backend trả access token và thông tin làm mới.
- Client sử dụng access token cho các Request tiếp theo.
- Khi access token hết hạn, hệ thống kiểm tra phiên trước khi cấp thông tin xác thực mới.
- Khi đăng xuất hoặc phiên bị thu hồi, việc làm mới không còn được chấp nhận.

c) Phân quyền theo vai trò

Sau khi xác thực danh tính, Backend tiếp tục kiểm tra quyền thực hiện chức năng.

Chuỗi Guard toàn cục của SpecHub được tổ chức theo thứ tự:

- **ThrottlerGuard:** kiểm soát tần suất Request.
- **JwtAuthGuard:** kiểm tra trạng thái xác thực.
- **RolesGuard:** kiểm tra vai trò người dùng.

Ngoài việc kiểm tra vai trò ở tầng Guard, các điều kiện về quyền sở hữu tài nguyên hoặc trạng thái nghiệp vụ vẫn được kiểm tra tại Service. Cách tổ chức này hạn chế trường hợp một người dùng đã đăng nhập nhưng cố gắng thao tác trên tài nguyên không thuộc phạm vi quyền của mình.

Nhờ sự kết hợp giữa JWT, Redis và cơ chế phân quyền theo vai trò, Backend có thể kiểm soát quá trình truy cập từ khi Request được gửi đến cho tới trước khi logic nghiệp vụ và dữ liệu được thực thi.

1.2. Trình biên dịch (IDE & Text Editor)

Quá trình phát triển SpecHub bao gồm nhiều thành phần như giao diện Web, Backend API, cơ sở dữ liệu, tác vụ nền và dịch vụ AI. Bên cạnh việc chỉnh sửa mã nguồn TypeScript, dự án còn thường xuyên thực hiện các thao tác với Prisma, PostgreSQL, Redis, pnpm và Turborepo thông qua giao diện dòng lệnh. Vì vậy, một môi trường phát triển có khả năng hỗ trợ nhiều loại mã nguồn, tích hợp Terminal và quản lý dự án theo cấu trúc monorepo là cần thiết.

Trong đồ án, **Visual Studio Code (VS Code)** được sử dụng làm công cụ soạn thảo và phát triển mã nguồn chính. Công cụ hỗ trợ tốt hệ sinh thái JavaScript/TypeScript, tích hợp Terminal, Git, khả năng gỡ lỗi và hệ thống tiện ích mở rộng, phù hợp với cấu trúc công nghệ của SpecHub.

2.2.1. Giới thiệu chung về Visual Studio Code

Visual Studio Code là trình soạn thảo mã nguồn do Microsoft phát triển, hỗ trợ các hệ điều hành phổ biến như Windows, Linux và macOS. VS Code được thiết kế theo hướng gọn nhẹ nhưng vẫn cung cấp nhiều chức năng thường có trong các môi trường phát triển tích hợp như gợi ý mã nguồn, gỡ lỗi, Terminal, quản lý phiên bản và hệ thống tiện ích mở rộng.

Báo cáo mẫu cũng sử dụng Visual Studio Code làm môi trường phát triển chính và tập trung phân tích các khả năng như IntelliSense, Debugging, Version Control, Terminal và Extensions.

Một số đặc điểm chính của Visual Studio Code gồm:

- **IntelliSense:** hỗ trợ tự động hoàn thành mã nguồn, hiển thị thông tin kiểu dữ liệu, tham số của hàm và các thành phần liên quan.
- **Debugging:** hỗ trợ đặt điểm dừng, theo dõi giá trị biến và kiểm tra luồng thực thi của chương trình.
- **Integrated Terminal:** cung cấp cửa sổ dòng lệnh trực tiếp trong môi trường soạn thảo.
- **Source Control:** tích hợp Git, hỗ trợ theo dõi thay đổi mã nguồn và quản lý các nhánh phát triển.
- **Extensions:** cho phép bổ sung khả năng hỗ trợ framework, ngôn ngữ, cơ sở dữ liệu và các công cụ phát triển khác.
- **Workspace:** hỗ trợ quản lý dự án có nhiều thư mục và package, phù hợp với những hệ thống tổ chức theo monorepo.

Nhờ khả năng hỗ trợ nhiều loại tệp và công nghệ trong cùng một môi trường, VS Code phù hợp với SpecHub khi mã nguồn của hệ thống được chia thành nhiều ứng dụng và package. Cấu trúc dự án hiện gồm các ứng dụng Web, API, Worker, AI Service cùng các package dùng chung về cơ sở dữ liệu, xác thực, AI, chấm điểm, kiểu dữ liệu và giao diện.

2.2.2. Vai trò và tiện ích của VS Code trong phát triển đồ án

Trong quá trình xây dựng SpecHub, Visual Studio Code được sử dụng xuyên suốt từ việc phát triển giao diện, xây dựng API, thiết kế mô hình dữ liệu đến chạy và kiểm tra các dịch vụ của hệ thống.

a) Hỗ trợ phát triển TypeScript và mã nguồn Web

TypeScript là ngôn ngữ chính của SpecHub, được sử dụng trong ứng dụng Next.js, NestJS và phần lớn các package dùng chung.

VS Code có khả năng hỗ trợ trực tiếp JavaScript và TypeScript, nhờ đó quá trình phát triển được thuận lợi hơn thông qua các chức năng:

- Tự động hoàn thành tên biến, hàm, lớp và thuộc tính.
- Hiển thị thông tin kiểu dữ liệu khi lập trình.
- Phát hiện lỗi kiểu dữ liệu ngay trong quá trình soạn thảo.
- Hỗ trợ tự động import các module.
- Điều hướng nhanh đến định nghĩa của hàm, lớp hoặc interface.
- Tìm kiếm vị trí một thành phần đang được sử dụng trong toàn bộ dự án.

Những chức năng trên có ý nghĩa đối với SpecHub do cùng một cấu trúc dữ liệu có thể được sử dụng qua nhiều lớp khác nhau, từ model và DTO ở Backend đến kiểu dữ liệu và component tại Frontend.

VS Code đồng thời hỗ trợ các loại mã nguồn khác xuất hiện trong dự án như TSX, JSON, CSS, Markdown, Prisma Schema và các tệp cấu hình.

b) Quản lý dự án theo cấu trúc Monorepo

SpecHub được tổ chức theo mô hình monorepo, trong đó nhiều ứng dụng và thư viện được quản lý trong cùng một kho mã nguồn. Các thành phần chính gồm:

- apps/web: ứng dụng Next.js.
- apps/api: API NestJS/Fastify.
- apps/worker: các tiến trình nền.
- apps/ai-service: thành phần liên quan đến AI.
- packages/database: Prisma, migration và dữ liệu.
- packages/types: kiểu dữ liệu dùng chung.
- packages/ui: các thành phần giao diện dùng chung.
- Các package hỗ trợ AI, xác thực, cảnh báo và chấm điểm.

Workspace của VS Code cho phép mở toàn bộ cấu trúc trên trong cùng một môi trường. Chức năng tìm kiếm toàn dự án giúp xác định nhanh những vị trí đang sử dụng một model, API, kiểu dữ liệu hoặc component khi cần thay đổi.

Điều này đặc biệt hữu ích khi một thay đổi có thể ảnh hưởng đồng thời đến nhiều phần của hệ thống. Ví dụ, thay đổi cấu trúc dữ liệu thiết bị có thể liên quan đến Prisma Schema, Backend Service, DTO, API Client và giao diện hiển thị.

c) Tích hợp Terminal và công cụ dòng lệnh

Một phần lớn quá trình phát triển SpecHub được thực hiện thông qua các lệnh Terminal. VS Code cung cấp Integrated Terminal, cho phép thực hiện những thao tác này ngay trong workspace mà không cần chuyển sang một ứng dụng dòng lệnh riêng.

Dự án sử dụng **pnpm** để quản lý package và **Turborepo** để quản lý các tác vụ trong monorepo. Ngoài ra, Prisma CLI được sử dụng cho quá trình sinh Client, migration và quản lý cơ sở dữ liệu.

Một số thao tác phát triển của SpecHub gồm:

```
pnpm install
pnpm db:generate
pnpm db:migrate
pnpm db:seed
pnpm dev:web
pnpm dev:api
pnpm dev:worker
```

Các lệnh trên tương ứng với quá trình cài đặt thư viện, tạo Prisma Client, cập nhật cơ sở dữ liệu và khởi chạy từng thành phần của hệ thống.

Terminal tích hợp giúp lập trình viên có thể đồng thời quan sát mã nguồn và trạng thái chạy của Web, API hoặc Worker, từ đó thuận tiện hơn trong quá trình kiểm tra và xử lý lỗi.

d) Hỗ trợ phát triển giao diện

Đối với ứng dụng Web, SpecHub sử dụng React, Next.js và Tailwind CSS. VS Code hỗ trợ việc chỉnh sửa TSX, CSS và các lớp tiện ích Tailwind trong cùng một môi trường.

Trong quá trình xây dựng giao diện, công cụ hỗ trợ:

- Kiểm tra cấu trúc component React.
- Phát hiện lỗi TypeScript trong component.
- Tự động hoàn thành thuộc tính và kiểu dữ liệu.
- Tìm kiếm và tái sử dụng component.
- Hỗ trợ các lớp tiện ích Tailwind.
- Định dạng mã nguồn thống nhất.

Khả năng này phù hợp với SpecHub vì giao diện có nhiều nhóm component chuyên biệt như thông số thiết bị, bảng điểm, thư viện media, công cụ khuyến nghị, Wiki và giao diện Catalog Studio.

e) Hỗ trợ phát triển Backend và API

Phần Backend sử dụng NestJS và được tổ chức thành nhiều Controller, Service, DTO, Guard và Module. VS Code hỗ trợ điều hướng giữa các lớp này thông qua chức năng tìm định nghĩa, tìm tham chiếu và tìm kiếm trên toàn workspace.

Khi phát triển một API mới, lập trình viên có thể làm việc lần lượt với:

- Controller để định nghĩa endpoint.
- DTO để mô tả dữ liệu đầu vào.
- Service để xử lý nghiệp vụ.
- Prisma Client để truy cập dữ liệu.
- Guard để kiểm tra xác thực và quyền.
- Các tệp kiểm thử tương ứng.

Điều này phù hợp với luồng xử lý thực tế của SpecHub, trong đó Request sau khi được kiểm tra xác thực, phân quyền và DTO sẽ được Controller chuyển tới Service để tiếp tục xử lý nghiệp vụ và truy cập Prisma, Redis hoặc các dịch vụ liên quan.

f) Hỗ trợ Prisma và cơ sở dữ liệu

Cơ sở dữ liệu của SpecHub được mô hình hóa bằng Prisma Schema. Trong quá trình phát triển, VS Code được sử dụng để chỉnh sửa các model, quan hệ và cấu hình trong `schema.prisma`.

Kết hợp với Terminal, lập trình viên có thể thực hiện các thao tác như:

```
pnpm db:generate
pnpm db:migrate
```

Qua đó, quá trình thay đổi mô hình dữ liệu, tạo Prisma Client và áp dụng migration có thể được thực hiện trong cùng môi trường phát triển.

Đặc điểm này có ý nghĩa đối với SpecHub do lược đồ dữ liệu có phạm vi lớn, bao gồm các nhóm thiết bị, biến thể, phần cứng, benchmark, Wiki, AI, nguồn dữ liệu và các chức năng người dùng.

g) Quản lý mã nguồn với Git

VS Code tích hợp chức năng Source Control dành cho Git, cho phép theo dõi các thay đổi của mã nguồn trực tiếp trong trình soạn thảo.

Trong quá trình phát triển đồ án, Git hỗ trợ:

- Theo dõi các tệp đã thay đổi.
- So sánh mã nguồn trước và sau chỉnh sửa.

- Tạo commit cho từng nhóm thay đổi.
- Quản lý các nhánh phát triển.
- Khôi phục hoặc kiểm tra lại những thay đổi trước đó.
- Giảm nguy cơ mất mã nguồn trong quá trình chỉnh sửa hệ thống.

Đối với SpecHub, việc quản lý phiên bản có ý nghĩa khi một thay đổi có thể liên quan đồng thời đến nhiều ứng dụng và package trong monorepo.

h) Hệ sinh thái tiện ích mở rộng

Một trong những ưu điểm của Visual Studio Code là khả năng mở rộng thông qua Extensions. Đối với hệ công nghệ của SpecHub, các nhóm tiện ích hỗ trợ chủ yếu phục vụ:

- TypeScript và JavaScript.
- React và Next.js.
- Prisma Schema.
- Tailwind CSS.
- ESLint và kiểm tra chất lượng mã nguồn.
- Định dạng mã nguồn.
- Git và quản lý phiên bản.
- Docker và các dịch vụ phát triển khi cần thiết.

Trong đó, những tiện ích hỗ trợ gợi ý Tailwind, định dạng và kiểm tra mã nguồn giúp giảm sai sót khi làm việc với số lượng lớn tệp trong monorepo. Báo cáo mẫu cũng lựa chọn hệ sinh thái Extensions, Terminal tích hợp, gỡ lỗi và hỗ trợ mã nguồn thông minh làm các vai trò chính của VS Code trong quá trình phát triển đồ án.

Nhờ kết hợp khả năng soạn thảo mã nguồn, Terminal, quản lý Git, gỡ lỗi và hệ thống tiện ích mở rộng, Visual Studio Code đáp ứng được nhu cầu phát triển đồng thời nhiều thành phần của SpecHub trong một môi trường thống nhất.

1.3. Thiết kế kiến trúc phần mềm

SpecHub là nền tảng Web có phạm vi nghiệp vụ tương đối lớn, bao gồm quản lý thư viện thiết bị, thành phần phần cứng, tìm kiếm, so sánh, chấm điểm, Wiki, quản lý nguồn dữ liệu, cảnh báo giá và các chức năng hỗ trợ AI. Nếu toàn bộ mã nguồn và logic nghiệp vụ được tổ chức trong một khối duy nhất mà không có sự phân chia rõ ràng, hệ thống sẽ khó bảo trì khi số lượng chức năng và mô hình dữ liệu tăng lên.

Để giải quyết vấn đề này, SpecHub được tổ chức theo **kiến trúc Monorepo kết hợp Modular Monolith**, trong đó API được chia thành các module theo miền nghiệp vụ, đồng thời Web, API, Worker và AI Service được tổ chức thành những ứng dụng có thể vận hành độc lập. Các thành phần dùng chung như kiểu dữ liệu, truy cập cơ sở dữ liệu, xác thực, chấm điểm và xử lý AI được tách thành các package riêng trong cùng một kho mã nguồn.

2.3.1. Tổng quan về kiến trúc Monorepo và Modular Monolith

a) Kiến trúc Monorepo

Monorepo là phương pháp tổ chức trong đó nhiều ứng dụng và thư viện của cùng một hệ thống được quản lý trong một kho mã nguồn thống nhất. Thay vì tạo một repository độc lập cho từng thành phần, các ứng dụng vẫn được phân chia thành những thư mục riêng nhưng có thể chia sẻ cấu hình, kiểu dữ liệu và các thư viện dùng chung.

Trong SpecHub, cấu trúc cấp cao được chia thành hai nhóm chính:

```
spechub/
├── apps/
│   ├── api/
│   ├── web/
│   ├── worker/
│   └── ai-service/
├── packages/
│   ├── database/
│   ├── api-client/
│   ├── ai-core/
│   ├── alerts-core/
│   ├── scoring-core/
│   ├── auth/
│   ├── analytics/
│   ├── config/
│   ├── types/
│   ├── ui/
│   └── utils/
├── docs/
├── scripts/
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

Cấu trúc trên phản ánh cách phân chia giữa **ứng dụng có khả năng triển khai** và các **package dùng chung**.

Nhóm apps bao gồm:

- **Web:** ứng dụng giao diện được xây dựng bằng Next.js.
- **API:** hệ thống Backend được xây dựng bằng NestJS và Fastify.
- **Worker:** tiến trình thực hiện những tác vụ nền như kiểm tra cảnh báo giá, thu thập dữ liệu và gửi thông báo.
- **AI Service:** thành phần hỗ trợ các tác vụ liên quan đến embedding cục bộ.

Nhóm `packages` chứa những thành phần có thể được sử dụng bởi nhiều ứng dụng:

- `database:` Prisma Schema, migration và các công cụ dữ liệu.
- `types:` các kiểu dữ liệu dùng chung.
- `api-client:` lớp giao tiếp API có kiểu.
- `ui:` các thành phần giao diện dùng chung.
- `auth:` các tiện ích liên quan đến xác thực.
- `ai-core:` các chức năng embedding và véc-tơ.
- `scoring-core:` thuật toán chuẩn hóa và tính điểm thiết bị.
- `alerts-core:` các quy tắc xử lý cảnh báo.
- `config:` cấu hình dùng chung.
- `utils:` các tiện ích chung.

Việc tổ chức theo Monorepo mang lại một số lợi ích đối với SpecHub:

- Cho phép Web và API chia sẻ kiểu dữ liệu chung.
- Hạn chế việc khai báo lặp lại cấu hình giữa nhiều ứng dụng.
- Thuận tiện khi một thay đổi ảnh hưởng đến nhiều thành phần.
- Cho phép các thuật toán như chấm điểm hoặc xử lý AI được tách khỏi ứng dụng chính và tái sử dụng.
- Dễ kiểm soát sự tương thích giữa các package trong cùng hệ thống.
- Tạo một quy trình kiểm thử và biên dịch thống nhất cho toàn dự án.

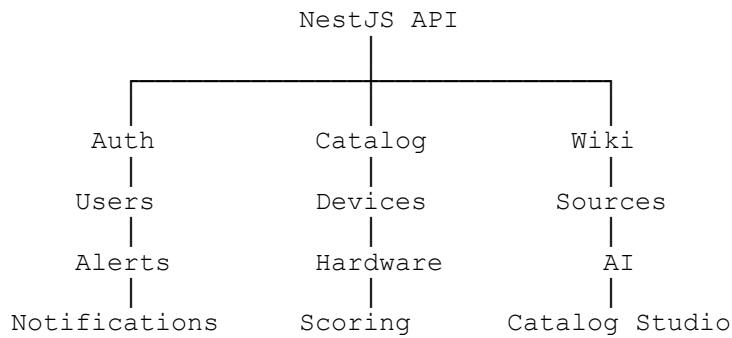
SpecHub sử dụng **pnpm workspace** để quản lý các package và **Turborepo** để điều phối các tác vụ trong Monorepo. Điều này cho phép các tác vụ như build, kiểm tra kiểu dữ liệu và kiểm thử được thực hiện theo quan hệ phụ thuộc giữa các ứng dụng và package.

b) Kiến trúc Modular Monolith

Ở tầng Backend, SpecHub áp dụng **Modular Monolith – kiến trúc nguyên khối mô-đun**.

Trong kiến trúc nguyên khối truyền thống, nhiều nghiệp vụ thường được triển khai trong cùng một ứng dụng và có nguy cơ phụ thuộc chặt chẽ vào nhau. Modular Monolith vẫn duy trì một ứng dụng Backend chính nhưng chia hệ thống thành các module theo từng miền nghiệp vụ.

Có thể biểu diễn:



Các module vẫn chạy trong cùng một Backend nhưng trách nhiệm nghiệp vụ được phân chia rõ ràng.

Trong SpecHub, API tập trung xử lý quy tắc nghiệp vụ, trong khi Web đảm nhiệm phân trình bày và Worker chịu trách nhiệm đối với các công việc định kỳ.

Cách tổ chức này phù hợp với hệ thống vì các nhóm nghiệp vụ của SpecHub có mối quan hệ dữ liệu khá chặt chẽ. Ví dụ:

- Thiết bị liên kết với linh kiện.
- Linh kiện liên kết với benchmark.
- Thiết bị và linh kiện liên kết với nguồn.
- Dữ liệu nguồn được sử dụng trong Wiki và AI.
- Biểu đồ thiết bị được sử dụng trong hệ thống chấm điểm và so sánh.

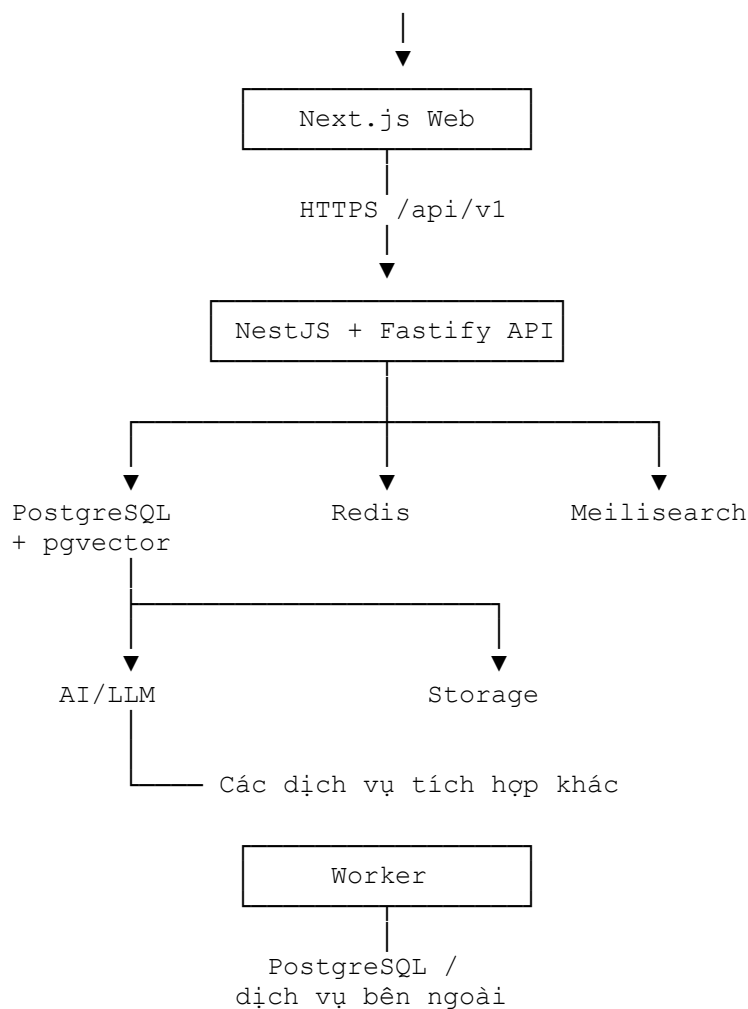
Việc tách toàn bộ các miền trên thành các dịch vụ mạng độc lập sẽ làm tăng độ phức tạp trong giao tiếp và quản lý dữ liệu. Modular Monolith cho phép các miền được phân tách về mặt mã nguồn nhưng vẫn sử dụng chung tầng dữ liệu và quá trình triển khai Backend.

Do đó, kiến trúc hiện tại của SpecHub **không phải kiến trúc Microservices thuần túy**. API chính vẫn là một Modular Monolith, trong khi một số tiến trình có đặc điểm khác biệt như Web, Worker và AI Service có thể được chạy độc lập.

c) Kiến trúc tổng thể của hệ thống

Kiến trúc tổng thể của SpecHub có thể mô tả như sau:

Người dùng



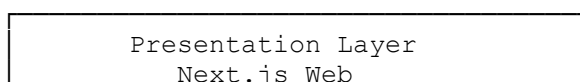
Ngoài người dùng Web, các đối tác B2B có thể truy cập một số API thông qua khóa API riêng. Worker tương tác với cơ sở dữ liệu và các dịch vụ ngoài nhưng không phụ thuộc vào giao diện Web để hoạt động.

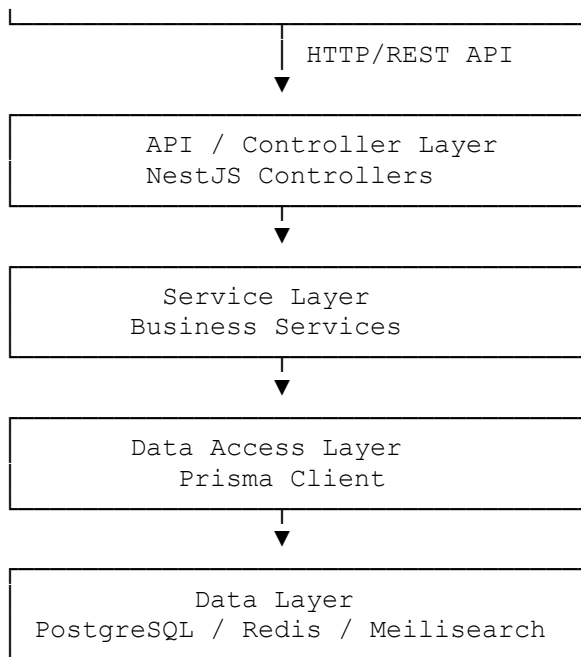
Hình 2.x. Kiến trúc tổng thể của hệ thống SpecHub

2.3.2. Kiến trúc phân tầng dịch vụ

Bên cạnh việc chia hệ thống thành các ứng dụng và module, quá trình xử lý nghiệp vụ của SpecHub được phân chia theo các lớp trách nhiệm. Mục tiêu là tránh việc giao diện, giao tiếp HTTP, xử lý nghiệp vụ và truy cập dữ liệu phụ thuộc trực tiếp vào nhau.

Có thể mô tả theo các tầng:





a) Tầng giao diện – Presentation Layer

Tầng giao diện được triển khai bằng Next.js và React. Đây là lớp trực tiếp tương tác với người dùng và chịu trách nhiệm:

- Hiển thị danh mục thiết bị.
- Hiển thị thông tin chi tiết.
- Tiếp nhận thao tác tìm kiếm và lọc.
- Xây dựng bảng so sánh.
- Hiển thị Wiki và nội dung nghiên cứu.
- Quản lý các chức năng người dùng.
- Gửi yêu cầu tới API.
- Hiển thị kết quả xử lý từ Backend.

Tầng Web không trực tiếp truy cập PostgreSQL. Các yêu cầu về dữ liệu hoặc nghiệp vụ được gửi tới Backend thông qua `/api/v1`.

Cách phân chia này giúp giao diện không phụ thuộc trực tiếp vào cấu trúc cơ sở dữ liệu. Khi mô hình lưu trữ thay đổi, giao diện chỉ cần tiếp tục tuân thủ hợp đồng API đã được xác định.

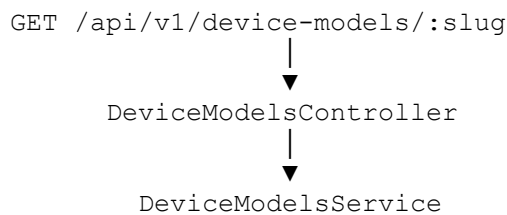
b) Tầng Controller – API Layer

Controller là lớp đầu tiên của Backend chịu trách nhiệm xử lý yêu cầu sau khi yêu cầu đã vượt qua các cơ chế bảo vệ và kiểm tra.

Controller thực hiện:

- Xác định endpoint.
- Tiếp nhận tham số URL.
- Tiếp nhận Query Parameter.
- Nhận dữ liệu Body.
- Ánh xạ Request vào DTO.
- Chuyển yêu cầu đến Service thích hợp.
- Trả kết quả về Client.

Ví dụ:



Controller không nên chứa toàn bộ logic nghiệp vụ mà chủ yếu đóng vai trò điều phối yêu cầu.

c) Tầng Service – Business Logic Layer

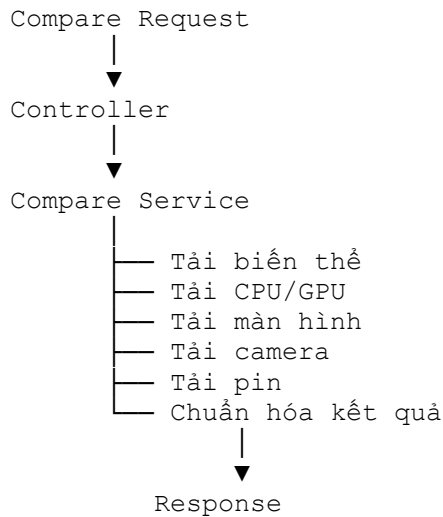
Service là tầng chịu trách nhiệm chính đối với logic nghiệp vụ của SpecHub.

Các nhiệm vụ của Service bao gồm:

- Kiểm tra trạng thái của thực thể.
- Kiểm tra quyền sở hữu.
- Xử lý quy tắc nghiệp vụ.
- Kết hợp dữ liệu từ nhiều nguồn.
- Thực hiện các phép tính.
- Gọi Prisma.
- Truy cập Redis.
- Gọi công cụ tìm kiếm.
- Giao tiếp với những dịch vụ bên ngoài khi cần thiết.

Luồng HTTP của hệ thống xác định Controller chuyển Request đến Service; sau đó Service kiểm tra trạng thái, quyền sở hữu và thực hiện truy cập Prisma, Redis hoặc dịch vụ ngoài.

Ví dụ, với chức năng so sánh thiết bị:



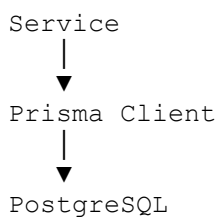
Việc đặt nghiệp vụ trong Service giúp cùng một logic có thể được sử dụng bởi nhiều endpoint mà không phải viết lại.

d) Tầng truy cập dữ liệu

Prisma Client đóng vai trò lớp giao tiếp giữa logic nghiệp vụ và PostgreSQL.

Service không cần tự xây dựng toàn bộ câu lệnh SQL cho các thao tác thông thường mà có thể truy cập các model thông qua Prisma.

Ví dụ:



Tầng dữ liệu của SpecHub bao gồm nhiều nhóm thực thể như danh mục thiết bị, phần cứng, benchmark, Wiki, nguồn, Catalog Studio, người dùng và các chức năng tương tác.

Việc cô lập quá trình truy cập dữ liệu giúp giảm sự phụ thuộc giữa logic nghiệp vụ và cách dữ liệu được lưu trữ.

e) Tầng tích hợp dịch vụ

Không phải mọi chức năng của SpecHub đều được xử lý hoàn toàn bằng PostgreSQL. Backend còn có thể giao tiếp với các dịch vụ hỗ trợ như:

- Redis cho phiên và dữ liệu tạm.
- Meilisearch cho tìm kiếm.
- Ollama hoặc nhà cung cấp AI theo cấu hình.
- Kho lưu trữ tương thích S3 cho media.
- Dịch vụ thư điện tử.
- Dịch vụ thanh toán.
- Nguồn dữ liệu bên ngoài.

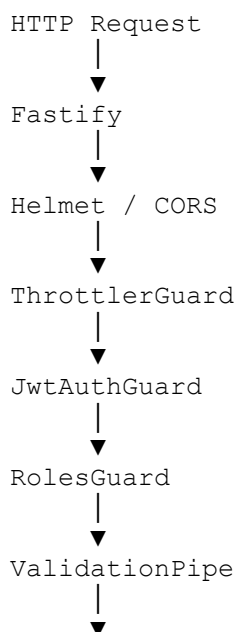
Các phụ thuộc này được đặt phía sau tầng Service thay vì để giao diện người dùng truy cập trực tiếp.

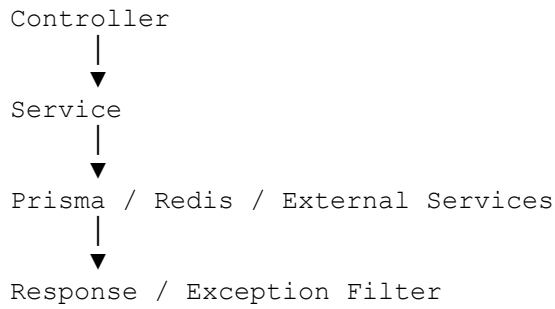
Nhờ đó, Web chỉ cần giao tiếp với API và không cần biết dữ liệu thực tế được lấy từ PostgreSQL, Redis, Meilisearch hay một dịch vụ bên ngoài.

2.3.3. Module hỗ trợ: *Middleware, Guard, Validation và Authentication*

Để bảo vệ Backend và bảo đảm Request được xử lý đúng quy tắc, SpecHub sử dụng nhiều lớp kiểm soát trước khi yêu cầu được chuyển đến nghiệp vụ.

Luồng xử lý có thể mô tả:





Đây là luồng xử lý được áp dụng tại API của SpecHub.

a) Middleware và cấu hình HTTP

Trước khi Request đi vào Controller, Fastify và NestJS áp dụng một số cấu hình chung.

Helmet được sử dụng để thiết lập các HTTP Header hỗ trợ bảo mật.

CORS kiểm soát những nguồn được phép gửi yêu cầu tới API.

Ngoài ra, mỗi Request được gán một mã định danh để phục vụ việc theo dõi và đối chiếu trong quá trình ghi nhật ký hoặc xử lý lỗi.

Luồng này giúp các yêu cầu được áp dụng cùng một tập quy tắc chung trước khi chuyển tới các module nghiệp vụ.

b) Throttler Guard

`ThrottlerGuard` đảm nhiệm việc kiểm soát tần suất gửi Request.

Mục đích của cơ chế này là hạn chế một Client gửi số lượng quá lớn yêu cầu trong thời gian ngắn, đặc biệt với các API có chi phí xử lý cao như đăng nhập, tìm kiếm hoặc AI.

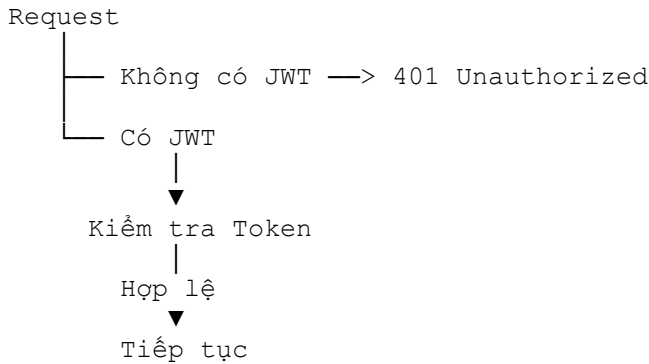
Trong chuỗi Guard toàn cục của SpecHub, `ThrottlerGuard` được xử lý trước `JwtAuthGuard` và `RolesGuard`.

c) JWT Authentication Guard

`JwtAuthGuard` chịu trách nhiệm xác định người gửi Request có được xác thực hay không.

Các endpoint cần bảo vệ sử dụng JWT để xác minh danh tính người dùng. Một số endpoint phục vụ tra cứu công khai có thể được khai báo cho phép truy cập mà không cần đăng nhập.

Luồng cơ bản:



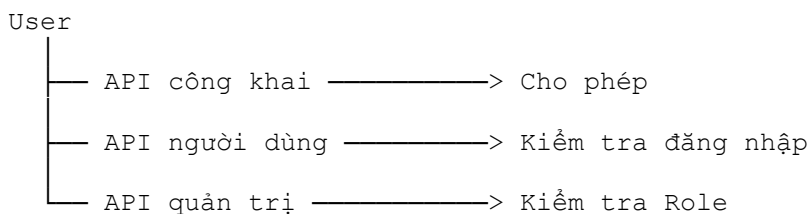
SpecHub sử dụng access token có thời hạn ngắn và liên kết quá trình làm mới với phiên được lưu trên Redis. Khi người dùng đăng xuất, phiên có thể được thu hồi tại phía máy chủ.

d) Roles Guard

Sau khi xác định người dùng đã đăng nhập, hệ thống vẫn cần xác định người dùng có đủ quyền thực hiện chức năng hay không.

RolesGuard được sử dụng để kiểm tra vai trò trên những endpoint yêu cầu quyền cụ thể.

Ví dụ:

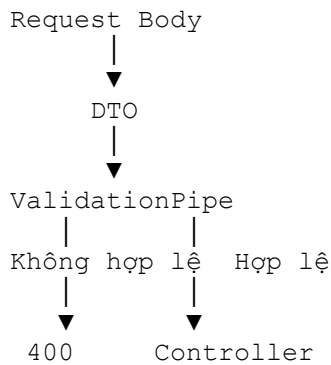


Tuy nhiên, vai trò không phải lớp kiểm tra cuối cùng. Đối với những tài nguyên thuộc về từng người dùng, Service tiếp tục kiểm tra quyền sở hữu trước khi cho phép thay đổi dữ liệu.

e) ValidationPipe và DTO

Dữ liệu do Client gửi lên không được sử dụng trực tiếp trong nghiệp vụ. SpecHub sử dụng DTO kết hợp ValidationPipe để xác định cấu trúc và kiểm tra dữ liệu đầu vào.

Quá trình có thể biểu diễn:



ValidationPipe thực hiện chuyển đổi và kiểm tra DTO, đồng thời từ chối những thuộc tính không được khai báo.

Cơ chế này giúp hạn chế:

- Thiếu trường bắt buộc.
- Kiểu dữ liệu không phù hợp.
- Giá trị không hợp lệ.
- Client gửi thêm thuộc tính không được hệ thống chấp nhận.
- Dữ liệu sai đi sâu vào tầng nghiệp vụ.

f) Global Exception Filter

Trong quá trình xử lý, lỗi có thể phát sinh ở nhiều tầng khác nhau như Guard, Validation, Service hoặc cơ sở dữ liệu.

Thay vì từng Controller tự định dạng lỗi, SpecHub sử dụng bộ lọc lỗi toàn cục để chuẩn hóa Response.

Ví dụ:

```
{
  "statusCode": 404,
  "message": "Resource not found",
  "requestId": "..."
```

}

Việc chuẩn hóa giúp Frontend có thể xử lý các lỗi theo một cấu trúc thống nhất, đồng thời mã định danh Request được giữ lại để phục vụ quá trình đối chiếu nhật ký.

2.3.4. Tiến trình nền và các thành phần triển khai độc lập

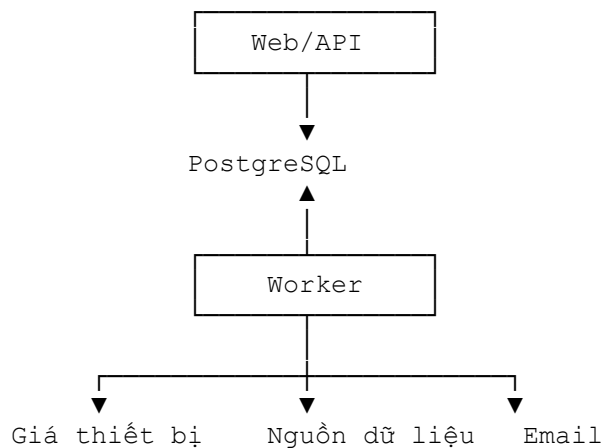
Không phải mọi nghiệp vụ đều phù hợp để thực hiện trực tiếp trong một HTTP Request. Một số công việc có tính định kỳ hoặc có thể kéo dài, nếu xử lý trực tiếp sẽ làm tăng thời gian phản hồi của API.

Vì vậy, SpecHub xây dựng ứng dụng **Worker** riêng để thực hiện các tác vụ nền.

Worker hiện đảm nhiệm ba nhóm công việc:

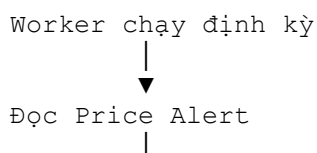
- Kiểm tra cảnh báo giá.
- Thu thập dữ liệu từ nguồn.
- Gửi thư thông báo.

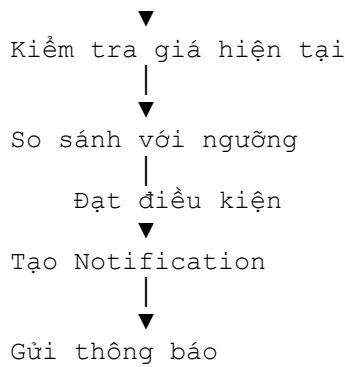
Có thể mô tả:



Worker có thể truy cập cơ sở dữ liệu và các dịch vụ ngoài mà không cần người dùng phải giữ một HTTP Request đang mở.

Ví dụ đối với cảnh báo giá:





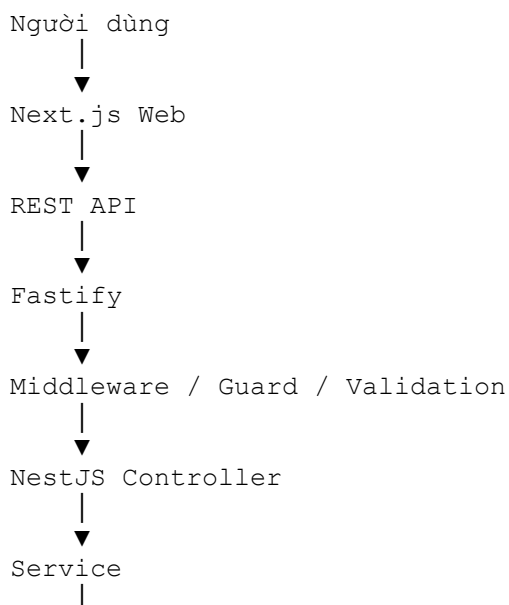
Việc tách tác vụ nền thành một tiến trình riêng mang lại các lợi ích:

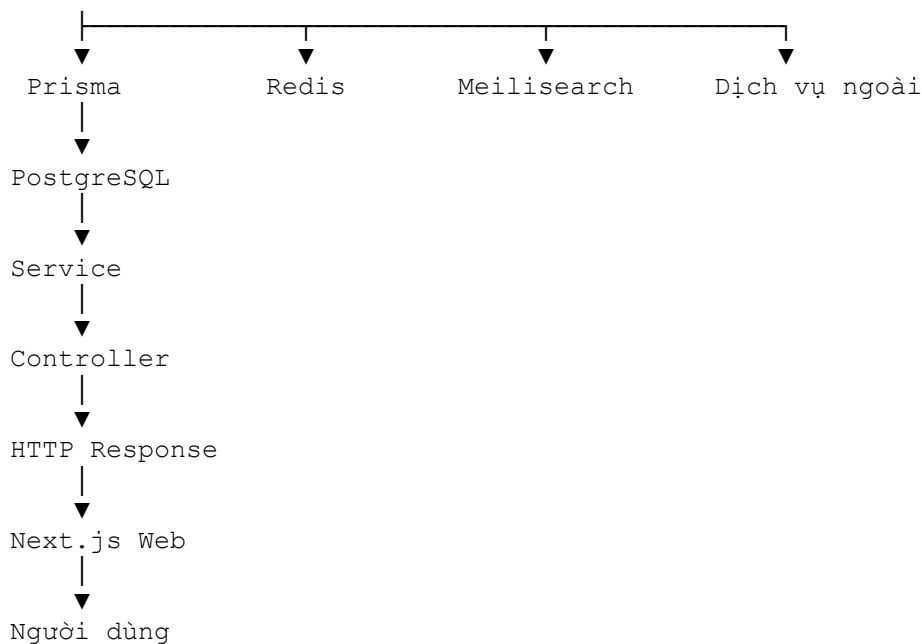
- Không làm kéo dài thời gian phản hồi của API.
- Có thể cấu hình chu kỳ chạy riêng.
- Có thể bật hoặc tắt từng nhóm công việc.
- Tách công việc định kỳ khỏi logic Request/Response.
- Cho phép triển khai và giám sát Worker độc lập với Web.

Ngoài Worker, `ai-service` cũng được tổ chức dưới dạng một ứng dụng riêng phục vụ thành phần liên quan đến embedding. Tuy nhiên, luồng sinh câu trả lời AI hiện vẫn được điều phối từ Backend API; do đó `ai-service` không được xem như một microservice nghiệp vụ độc lập của toàn hệ thống.

2.3.5. Tổng quan luồng xử lý trong kiến trúc SpecHub

Từ các thành phần trên, quá trình xử lý một yêu cầu thông thường trong SpecHub có thể tổng hợp như sau:





Kiến trúc này tạo ra sự phân chia tương đối rõ giữa **tầng giao diện, giao tiếp API, xử lý nghiệp vụ, truy cập dữ liệu và các dịch vụ hỗ trợ**. Đồng thời, Monorepo giúp các thành phần vẫn có thể chia sẻ những tài nguyên chung, còn Modular Monolith giúp các nghiệp vụ Backend được phân chia thành module mà không làm tăng độ phức tạp vận hành như một hệ thống Microservices hoàn chỉnh.

1.4. Giới thiệu tổng quan về hệ thống thư viện và chuẩn hóa dữ liệu thiết bị thông minh

Nền tảng SpecHub được xây dựng với mục tiêu chính là hình thành một **thư viện dữ liệu dành cho thiết bị thông minh**, trong đó thông tin không chỉ được lưu dưới dạng các bảng thông số độc lập mà được tổ chức thành một hệ thống dữ liệu có cấu trúc và có mối liên hệ giữa thiết bị, biến thể sản phẩm, linh kiện phần cứng, phần mềm, benchmark và nguồn tham chiếu.

Đặc điểm của dữ liệu thiết bị thông minh là một sản phẩm có thể tồn tại dưới nhiều tên thương mại, cấu hình bộ nhớ, khu vực phát hành và mã sản phẩm khác nhau. Đồng thời, một thành phần phần cứng như chipset, CPU, GPU hoặc cảm biến camera có thể được sử dụng trên nhiều thiết bị khác nhau. Nếu toàn bộ thông tin được lưu trực tiếp trên từng sản phẩm, dữ liệu sẽ dễ bị lặp, khó cập nhật và khó bảo đảm tính thống nhất.

Do đó, SpecHub áp dụng mô hình dữ liệu theo hướng **chuẩn hóa và tái sử dụng thực thể**. Mẫu thiết bị, biến thể, thành phần phần cứng và các thuộc tính kỹ thuật được quản lý thành những nhóm dữ liệu riêng, sau đó liên kết với nhau thông qua các quan hệ. Cách tổ

chức này tạo nền tảng cho các chức năng tra cứu, lọc, so sánh, chấm điểm, tìm kiếm và khai thác dữ liệu bằng AI.

2.4.1. Tổng quan về thư viện dữ liệu thiết bị thông minh

Thư viện dữ liệu thiết bị thông minh có thể được hiểu là một hệ thống tổ chức tập trung các thông tin liên quan đến thiết bị công nghệ theo một cấu trúc thống nhất.

Khác với một danh sách sản phẩm thông thường chỉ lưu tên, giá và một số thông số cơ bản, dữ liệu trong thư viện cần thể hiện được nhiều cấp thông tin như:

- Nhà sản xuất hoặc thương hiệu.
- Loại thiết bị.
- Dòng sản phẩm.
- Mẫu thiết bị.
- Biến thể cụ thể.
- Thành phần phần cứng.
- Thông số kỹ thuật.
- Phần mềm và hệ điều hành.
- Benchmark.
- Điểm đánh giá.
- Nguồn và trích dẫn.
- Lịch sử giá và thông tin thị trường.

Trong SpecHub, hệ thống được định hướng trở thành một nền tảng tri thức thiết bị có cấu trúc, kết hợp danh mục, linh kiện, nguồn tham chiếu, Wiki, công cụ so sánh, hệ thống điểm và các chức năng khai thác dữ liệu khác.

Mục tiêu của mô hình này là giúp một thông tin chỉ cần được quản lý tại vị trí phù hợp, sau đó có thể được liên kết và tái sử dụng ở nhiều thiết bị khác nhau.

Ví dụ, nếu nhiều mẫu điện thoại cùng sử dụng một chipset, hệ thống không cần lưu lại toàn bộ thông tin của chipset trên từng thiết bị. Chipset được quản lý như một thực thể độc lập và các thiết bị chỉ tạo quan hệ với thực thể đó.

Cách tiếp cận này giúp:

- Hạn chế trùng lặp dữ liệu.
- Dễ cập nhật thông tin khi một linh kiện cần chỉnh sửa.
- Cho phép truy vấn theo nhiều chiều.

- Tăng khả năng lọc và so sánh.
- Tạo mối liên hệ giữa thiết bị và linh kiện.
- Hỗ trợ mở rộng thêm các loại thiết bị trong tương lai.

2.4.2. Phân cấp danh mục thiết bị

Một vấn đề quan trọng khi xây dựng thư viện thiết bị là xác định đúng cấp độ của sản phẩm. Trong thực tế, tên thương hiệu, dòng sản phẩm, mẫu thiết bị và phiên bản cụ thể thường dễ bị trộn lẫn nếu không được mô hình hóa rõ ràng.

SpecHub tổ chức danh mục theo cấu trúc cơ bản:

Tổ chức/Thương hiệu → Loại thiết bị → Dòng sản phẩm → Mẫu thiết bị → Biến thể

Trong đó:

Tổ chức hoặc thương hiệu (Organization):

Đại diện cho nhà sản xuất hoặc tổ chức sở hữu một dòng sản phẩm, chẳng hạn Samsung, Apple, Xiaomi hoặc Qualcomm tùy theo loại thực thể được quản lý.

Loại thiết bị (Device Category):

Dùng để phân loại sản phẩm theo nhóm thiết bị, ví dụ điện thoại thông minh, máy tính bảng, máy tính xách tay hoặc các nhóm thiết bị thông minh khác.

Dòng sản phẩm (Product Family):

Đại diện cho một nhóm sản phẩm có cùng định hướng hoặc thương hiệu sản phẩm.

Mẫu thiết bị (Device Model):

Đại diện cho sản phẩm ở cấp tên thương mại chính.

Biến thể thiết bị (Device Variant):

Đại diện cho một cấu hình cụ thể của mẫu thiết bị, có thể khác nhau về bộ nhớ, dung lượng lưu trữ, khu vực phát hành hoặc mã hàng.

Đặc tả dữ liệu của SpecHub xác định quan hệ cốt lõi theo hướng một tổ chức sở hữu nhiều dòng sản phẩm, mỗi dòng sản phẩm chứa nhiều mẫu thiết bị và một mẫu thiết bị có thể có nhiều biến thể.

Có thể biểu diễn:

Việc phân chia này giải quyết tình trạng coi mọi cấu hình của cùng một thiết bị là những sản phẩm hoàn toàn độc lập.

Ví dụ, một mẫu điện thoại có thể có:

- Phiên bản 8 GB RAM + 128 GB lưu trữ.
- Phiên bản 8 GB RAM + 256 GB lưu trữ.
- Phiên bản 12 GB RAM + 512 GB lưu trữ.
- Phiên bản dành cho một khu vực thị trường khác.

Tất cả vẫn thuộc cùng một **Device Model**, nhưng được thể hiện bằng các **Device Variant** khác nhau.

2.4.3. Phân biệt mẫu thiết bị và biến thể thiết bị

Việc phân biệt giữa **Model** và **Variant** là một trong những cơ sở quan trọng của mô hình dữ liệu SpecHub.

Mẫu thiết bị mô tả sản phẩm ở mức khái quát, trong khi biến thể thể hiện sản phẩm thực tế với một cấu hình xác định. Theo quy ước của hệ thống, mẫu thiết bị là sản phẩm ở cấp tên thương mại, còn biến thể được xác định dựa trên các yếu tố như bộ nhớ, lưu trữ, khu vực hoặc mã hàng.

Có thể biểu diễn quan hệ:

Một `DeviceModel` có thể chứa nhiều `DeviceVariant`, nhưng một biến thể chỉ thuộc về một mẫu thiết bị chính.

Các thông tin mang tính chung của sản phẩm có thể được đặt ở cấp Model, trong khi thông tin phụ thuộc cấu hình cụ thể được đặt ở Variant.

Ví dụ:

Device Model:

- Tên thương mại.
- Dòng sản phẩm.
- Nhà sản xuất.
- Ngày công bố.
- Thông tin giới thiệu.
- Tên thay thế.

Device Variant:

- RAM.
- Dung lượng lưu trữ.
- Khu vực.
- Mã sản phẩm.
- Thành phần phần cứng cụ thể.
- Giá.
- Các đặc điểm riêng của phiên bản.

Cấu trúc này giúp chức năng so sánh của hệ thống hoạt động chính xác hơn vì người dùng có thể so sánh đúng các biến thể thay vì chỉ dựa vào tên chung của sản phẩm.

2.4.4. Mô hình hóa thành phần phần cứng

Thiết bị thông minh được hình thành từ nhiều thành phần phần cứng khác nhau. Vì vậy, chỉ lưu thông số trực tiếp trong bản ghi thiết bị sẽ làm giảm khả năng tái sử dụng và phân tích dữ liệu.

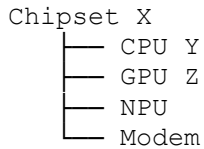
SpecHub mô hình hóa các thành phần chính thành các thực thể độc lập như:

- Chipset.
- CPU.
- GPU.
- NPU.
- Modem.
- Cảm biến camera.
- Mô-đun camera.
- Màn hình.
- Pin.
- Chuẩn bộ nhớ.
- Chuẩn lưu trữ.
- Hệ điều hành và phiên bản phần mềm.

Ví dụ, thay vì lưu:

Thiết bị A
Chipset = Chip X
CPU = CPU Y
GPU = GPU Z

và tiếp tục lặp lại toàn bộ thông tin đó trên thiết bị B, SpecHub tổ chức:



Sau đó:

Device Variant A —> Chipset X
Device Variant B —> Chipset X

Nhờ đó, khi người dùng truy cập thông tin của một chipset, hệ thống cũng có thể xác định các CPU, GPU, modem liên quan và những thiết bị đang sử dụng chipset đó.

2.4.5. Quan hệ giữa chipset và các thành phần xử lý

Một chipset trên thiết bị thông minh thường chứa hoặc liên quan đến nhiều đơn vị xử lý khác nhau như CPU, GPU, NPU và modem. Vì vậy, SpecHub không xem chipset như một chuỗi văn bản đơn thuần.

Các quan hệ được tổ chức thông qua những bảng liên kết như:

- Chipset – CPU.
- Chipset – GPU.
- Chipset – NPU.
- Chipset – Modem.

Tài liệu dữ liệu của SpecHub xác định chipset liên kết với CPU, GPU, NPU và modem thông qua các bảng nối thay vì sao chép toàn bộ thông tin sang từng thiết bị.

Cách tổ chức này cho phép xây dựng các truy vấn như:

- Chipset nào sử dụng CPU này?
- GPU này được tích hợp trên những chipset nào?
- Những thiết bị nào sử dụng cùng một chipset?
- Hai thiết bị sử dụng cùng CPU nhưng GPU có khác nhau hay không?
- Một kiến trúc CPU xuất hiện trên những thế hệ chipset nào?

Đây là một lợi thế quan trọng của thư viện dữ liệu có cấu trúc so với cách trình bày thông số dưới dạng văn bản độc lập.

2.4.6. Mô hình hóa hệ thống camera

Camera trên thiết bị thông minh cũng có cấu trúc phức tạp. Một camera không chỉ bao gồm giá trị độ phân giải mà còn liên quan đến cảm biến, vai trò của camera, khả năng quay video và nhiều tính năng xử lý hình ảnh.

Vì vậy, SpecHub phân biệt giữa:

- **Camera Sensor:** cảm biến hình ảnh.
- **Camera Module:** mô-đun camera được sử dụng trên thiết bị.
- **Camera Role:** vai trò như camera chính, góc siêu rộng, telephoto hoặc camera trước.
- **Camera Feature:** các khả năng và tính năng hỗ trợ.
- **Video Mode:** chế độ quay video.

Cách phân tách giữa cảm biến và mô-đun cho phép cùng một cảm biến được sử dụng trên nhiều thiết bị với cách triển khai khác nhau.

Có thể biểu diễn:

Ví dụ, hai thiết bị có thể sử dụng cùng một cảm biến camera nhưng khác ống kính, khả năng chống rung hoặc chế độ quay video. Nếu chỉ lưu tên cảm biến trực tiếp trên thiết bị thì những khác biệt này khó được mô hình hóa rõ ràng.

SpecHub vì vậy quản lý riêng cảm biến, mô-đun và các quan hệ giữa chúng.

2.4.7. Chuẩn hóa thuộc tính và từ điển công nghệ

Ngoài việc tách các thực thể phân cứng, dữ liệu còn cần được chuẩn hóa về thuật ngữ.

Trong thực tế, cùng một khái niệm có thể được viết dưới nhiều hình thức khác nhau. Nếu mỗi bản ghi tự lưu một chuỗi văn bản, hệ thống có thể xuất hiện các giá trị như:

4 nm
4nm
4-nm
4 nanometer

Mặc dù cùng biểu diễn một tiến trình sản xuất, cơ sở dữ liệu có thể coi đây là các giá trị khác nhau.

SpecHub sử dụng các nhóm dữ liệu từ điển cho những khái niệm thường được tái sử dụng như:

- Kiến trúc.
- Tiến trình sản xuất.
- Công nghệ màn hình.
- Hóa học pin.
- Vai trò camera.
- Hệ thống mạng.
- Đơn vị đo.
- Chuẩn kết nối.

Các bảng từ điển giúp giảm việc lặp chuỗi và tạo ra các điểm liên kết ổn định cho quá trình lọc, dịch thuật và phân tích dữ liệu.

Có thể xem quá trình chuẩn hóa:

Ví dụ:

```
"120 hz"  
"120Hz"  
"120 Hertz"  
↓  
Refresh rate = 120  
Unit = Hz
```

Dữ liệu sau khi chuẩn hóa trở nên phù hợp hơn với quá trình so sánh và tính toán.

2.4.8. Mô hình quan hệ nhiều - nhiều giữa thiết bị và linh kiện

Trong dữ liệu thiết bị, một linh kiện có thể được sử dụng trên nhiều sản phẩm và một biến thể cũng có thể liên kết với nhiều thành phần.

Đây là dạng quan hệ **Many-to-Many (N-N)**.

Ví dụ:

Một thiết bị có nhiều camera, trong khi một mô-đun camera cũng có thể được sử dụng trên nhiều thiết bị.

Tương tự:

SpecHub sử dụng các bảng liên kết cho chipset, CPU, GPU, NPU, modem, màn hình, pin, camera, bộ nhớ, hệ điều hành, băng tần và nhiều nhóm thông tin khác.

Cách tổ chức này giúp hệ thống có thể thực hiện các truy vấn chuyên sâu như:

- Tìm tất cả thiết bị hỗ trợ một băng tần cụ thể.
- Tìm các thiết bị sử dụng cùng GPU.
- Tìm thiết bị có cùng cảm biến camera.
- Tìm thiết bị hỗ trợ một chuẩn sạc nhất định.
- Tìm các sản phẩm sử dụng cùng loại bộ nhớ.

2.4.9. Chuẩn hóa dữ liệu phục vụ tìm kiếm và so sánh

Một trong những mục đích quan trọng của việc xây dựng dữ liệu có cấu trúc là hỗ trợ chức năng tìm kiếm, lọc và so sánh.

Khi dữ liệu đã được chuẩn hóa, hệ thống có thể thực hiện các điều kiện như:

```
RAM >= 8 GB  
Battery >= 5000 mAh  
Refresh rate >= 120 Hz  
Storage >= 256 GB
```

thay vì phải tìm các chuỗi văn bản chứa những giá trị tương ứng.

Đối với chức năng tìm kiếm, người dùng nhập từ khóa hoặc điều kiện lọc; hệ thống chuẩn hóa truy vấn và lựa chọn nguồn tìm kiếm phù hợp. Khi thực hiện so sánh, Backend tải các biến thể cùng quan hệ phân cứng và giao diện xây dựng ma trận thông số để làm nổi bật sự khác biệt.

Có thể biểu diễn mô hình so sánh:

với mỗi thuộc tính được so sánh theo cùng một loại dữ liệu và đơn vị.

Ví dụ:

Thuộc tính	Thiết bị A	Thiết bị B
RAM	8 GB	12 GB
Pin	5000 mAh	6000 mAh
Tần số quét	120 Hz	144 Hz

Nếu dữ liệu không được chuẩn hóa, những phép so sánh này sẽ khó thực hiện chính xác bằng chương trình.

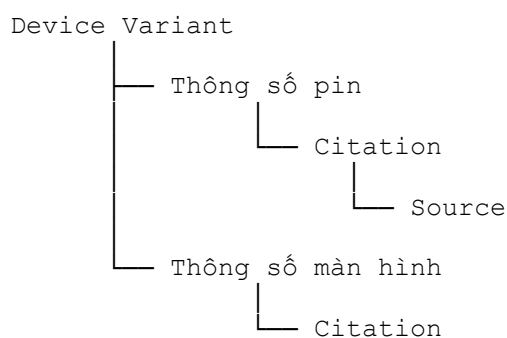
2.4.10. Quản lý nguồn và khả năng truy nguyên dữ liệu

Đối với dữ liệu thông số thiết bị, việc lưu giá trị chưa đủ để xác định độ tin cậy. Hệ thống cần biết thông tin đó được lấy từ đâu và có thể kiểm chứng lại khi cần.

SpecHub phân biệt hai khái niệm:

- **Nguồn (Source):** tài liệu hoặc trang dữ liệu gốc.
- **Trích dẫn (Citation):** mối liên hệ giữa nguồn và thông tin hoặc thực thể cụ thể.

Ví dụ:



Cách tổ chức này cho phép một thiết bị có thể sử dụng nhiều nguồn cho các nhóm dữ liệu khác nhau.

Việc quản lý nguồn mang lại các lợi ích:

- Có thể xác định nguồn gốc của thông tin.
- Hỗ trợ kiểm tra lại dữ liệu khi có sai lệch.
- Phân biệt dữ liệu đã xác minh với dữ liệu chưa được kiểm duyệt.

- Hỗ trợ Wiki và các nội dung biên tập.
- Cung cấp dữ liệu có căn cứ cho hệ thống AI.

Khả năng truy nguyên dữ liệu cũng phù hợp với mục tiêu chung của SpecHub là xây dựng một nguồn dữ liệu thống nhất có thể kiểm chứng và khai thác qua Web hoặc API.

2.4.11. Quy trình biên tập và kiểm duyệt dữ liệu

Thông tin thiết bị có thể được thu thập từ nhiều nguồn, do đó dữ liệu bên ngoài không nên được đưa trực tiếp vào thư viện công khai mà chưa qua quá trình kiểm tra.

SpecHub áp dụng nguyên tắc:

Dữ liệu thu thập được lưu vào khu vực chờ duyệt. Sau khi được người có quyền kiểm tra, dữ liệu phù hợp mới được chuẩn hóa và đưa vào danh mục công khai. Hệ thống không cho phép quá trình thu thập dữ liệu ghi trực tiếp vào catalog.

Quá trình này giúp hạn chế:

- Dữ liệu sai được xuất bản tự động.
- Thông số từ nguồn không đáng tin cậy đi vào hệ thống.
- Các giá trị không thống nhất về đơn vị.
- Trùng lặp thực thể.
- Thông tin chưa được xác minh xuất hiện trên giao diện người dùng.

2.4.12. Catalog Studio và quản lý phiên bản dữ liệu

Để hỗ trợ quy trình biên tập, SpecHub xây dựng **Catalog Studio** làm công cụ quản lý dữ liệu theo bản nháp.

Một thay đổi dữ liệu không nhất thiết được ghi trực tiếp vào thực thể đang hiển thị công khai. Thay vào đó, quá trình chỉnh sửa có thể được thực hiện thông qua bản nháp, kiểm tra tính hợp lệ và sau đó mới xuất bản.

Hệ thống sử dụng các nhóm dữ liệu:

- `catalog_drafts.`
- `catalog_draft_versions.`
- `catalog_entity_versions.`

Các nhóm này hỗ trợ:

- Lưu nội dung bản nháp.
- Theo dõi bước nhập liệu hiện tại.
- Lưu lịch sử phiên bản bản nháp.
- Khôi phục dữ liệu khi cần.
- Theo dõi phiên bản của thực thể.
- Kiểm tra dữ liệu trước khi xuất bản.

Quy trình cơ bản:

Tạo bản nháp



Nhập hoặc chỉnh sửa dữ liệu



Kiểm tra dữ liệu



Xử lý lỗi nếu có



Hoàn tất bản nháp



Xuất bản

Khi xuất bản, SpecHub sử dụng transaction để tránh trường hợp một phần dữ liệu đã được tạo trong khi phần còn lại thất bại, dẫn đến trạng thái catalog không hoàn chỉnh.

2.4.13. Khả năng mở rộng của mô hình dữ liệu

Một mục tiêu quan trọng của mô hình thư viện là không giới hạn hệ thống vào một loại thiết bị duy nhất.

Các thiết bị khác nhau có những nhóm thuộc tính khác nhau. Ví dụ:

Điện thoại thông minh:

- Chipset.
- Camera.
- Mạng di động.
- SIM.
- Pin và sạc.

Máy tính xách tay:

- CPU.
- GPU.
- RAM.
- Màn hình.
- Cổng kết nối.
- Pin.

Thiết bị đeo:

- Cảm biến.
- Kết nối.
- Pin.
- Hệ điều hành.
- Tính năng sức khỏe.

Nếu toàn bộ thông số được đưa vào một bảng duy nhất, số lượng cột sẽ tăng nhanh và nhiều trường không áp dụng cho từng loại thiết bị.

SpecHub vì vậy tổ chức các thành phần và cấu hình theo từng module, cho phép một biến thể có nhiều thành phần, tính năng và cấu hình mà vẫn duy trì khả năng truy vấn bằng cơ sở dữ liệu quan hệ.

Cách tiếp cận này tạo nền tảng để thư viện có thể được mở rộng thêm các nhóm sản phẩm mà không phải thay đổi toàn bộ mô hình dữ liệu cốt lõi.

2.4.14. Cơ sở hình thành mô hình thư viện SpecHub

Từ những yêu cầu về chuẩn hóa và quản lý dữ liệu thiết bị, mô hình thư viện của SpecHub được hình thành dựa trên sự kết hợp của nhiều lớp dữ liệu:

Trong đó:

- **Classification:** tổ chức thiết bị theo thương hiệu, loại và dòng sản phẩm.
- **Model – Variant:** phân biệt sản phẩm thương mại và cấu hình thực tế.
- **Hardware Modules:** quản lý các thành phần phần cứng dưới dạng thực thể tái sử dụng.
- **Technical Attributes:** chuẩn hóa thông số, khả năng và đơn vị.
- **Sources:** bảo đảm thông tin có khả năng truy nguyên.
- **Editorial Workflow:** kiểm soát dữ liệu trước khi được đưa vào thư viện công khai.

Từ cấu trúc dữ liệu này, các chức năng khác của SpecHub được xây dựng phía trên:

Thư viện dữ liệu chuẩn hóa



Tra cứu và lọc



So sánh thiết bị



Benchmark và chấm điểm



Khuyến nghị



Wiki và nội dung



AI/RAG

CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

1.1. Phân tích yêu cầu hệ thống

1.1.1. Yêu cầu chức năng

Yêu cầu chức năng mô tả những nghiệp vụ mà SpecHub phải thực hiện nhằm đáp ứng nhu cầu tra cứu, quản lý và khai thác dữ liệu thiết bị thông minh. Các chức năng được xác định dựa trên quyền và mục tiêu của từng nhóm tác nhân trong hệ thống.

3.1.1.1. Đối với khách truy cập và người dùng

1. Quản lý tài khoản và xác thực

Đối với người dùng đã đăng ký, hệ thống cần hỗ trợ:

- Đăng ký tài khoản bằng địa chỉ thư điện tử và mật khẩu.
- Đăng nhập vào hệ thống.
- Duy trì và làm mới phiên đăng nhập.
- Đăng xuất và thu hồi phiên hiện tại.
- Xem và cập nhật thông tin hồ sơ cá nhân.
- Thay đổi mật khẩu.
- Chỉ cho phép người dùng thay đổi những thông tin thuộc tài khoản của mình.

Hệ thống tạo access token có thời hạn ngắn và sử dụng phiên có khả năng thu hồi để quản lý trạng thái đăng nhập.

2. Tra cứu danh mục thiết bị

Khách truy cập và người dùng có thể khai thác thư viện thiết bị thông qua các chức năng:

- Xem danh sách thiết bị theo từng nhóm.
- Xem thông tin của nhà sản xuất và dòng sản phẩm.
- Xem thông tin chi tiết của một mẫu thiết bị.
- Xem các biến thể thuộc cùng một mẫu thiết bị.
- Xem thông số kỹ thuật theo từng nhóm.
- Xem hình ảnh và nội dung đa phương tiện.
- Xem điểm đánh giá và benchmark nếu có dữ liệu.
- Xem nguồn tham chiếu của thông tin.
- Tra cứu các thành phần phần cứng như chipset, CPU, GPU, NPU, modem, màn hình, camera, pin, bộ nhớ và hệ điều hành.

Thông tin được hiển thị dựa trên dữ liệu đã được chuẩn hóa và công khai trong hệ thống. Các thành phần phần cứng được quản lý thành những thực thể riêng nhằm hỗ trợ việc tra cứu ngược giữa linh kiện và thiết bị sử dụng linh kiện đó.

3. Tìm kiếm và lọc thiết bị

Hệ thống cần cho phép người dùng tìm kiếm dữ liệu mà không phải duyệt thủ công toàn bộ thư viện.

Các yêu cầu gồm:

- Tìm kiếm theo tên thiết bị.
- Tìm theo tên thay thế hoặc alias.
- Tìm kiếm linh kiện và dữ liệu liên quan.
- Lọc kết quả dựa trên các thuộc tính của thiết bị.
- Hỗ trợ phân trang đối với danh sách có nhiều kết quả.
- Đồng bộ trạng thái bộ lọc với URL để có thể quay lại hoặc chia sẻ truy vấn.
- Sử dụng công cụ tìm kiếm tăng cường khi được cấu hình.

4. So sánh thiết bị

Người dùng có thể lựa chọn nhiều thiết bị hoặc biến thể để đưa vào bảng so sánh.

Hệ thống cần:

- Hiển thị các thiết bị trên cùng hệ thống thuộc tính.

- Nhóm các thông số theo từng mô-đun như hiệu năng, màn hình, camera, pin và kết nối.
- Làm rõ những điểm khác nhau giữa các thiết bị.
- Phân biệt dữ liệu chưa xác định với giá trị bằng không.
- Cho phép chia sẻ danh sách thiết bị đang so sánh thông qua URL.

Chức năng này tận dụng cấu trúc dữ liệu đã được chuẩn hóa để các thông số cùng loại có thể được đối chiếu trực tiếp.

5. Khuyến nghị thiết bị theo nhu cầu

Hệ thống hỗ trợ người dùng lựa chọn thiết bị dựa trên nhu cầu thay vì chỉ tìm theo tên sản phẩm.

Người dùng có thể cung cấp các thông tin như:

- Nhu cầu sử dụng.
- Mức ngân sách.
- Nhóm tính năng được ưu tiên.
- Một số điều kiện mong muốn đối với thiết bị.

Từ các thông tin trên, hệ thống trả về danh sách thiết bị phù hợp kèm theo giải thích về lý do lựa chọn.

6. Vùng nghiên cứu

Đối với khách chưa đăng nhập, SpecHub vẫn hỗ trợ một vùng nghiên cứu cục bộ trên trình duyệt.

Vùng nghiên cứu có thể lưu:

- Danh sách thiết bị phục vụ so sánh.
- Các thiết bị đã xem gần đây.
- Một số truy vấn tìm kiếm gần đây.

Khi người dùng đăng nhập, dữ liệu phù hợp có thể được đồng bộ vào tài khoản theo quy tắc của hệ thống và không tạo dữ liệu trùng lặp.

7. Trợ lý AI và tìm kiếm ngữ nghĩa

SpecHub cung cấp chức năng trợ lý AI nhằm hỗ trợ người dùng khai thác kho dữ liệu bằng ngôn ngữ tự nhiên.

Các chức năng chính gồm:

- Đặt câu hỏi liên quan đến thiết bị và phần cứng.
- Nhận câu trả lời dựa trên dữ liệu được truy xuất từ hệ thống.
- Nhận câu trả lời theo luồng khi nội dung được sinh dần.
- Hiển thị nguồn tham chiếu cho các thông tin quan trọng.
- Tìm kiếm dữ liệu theo ngữ nghĩa.
- Nghiên cứu một thành phần phần cứng và các thiết bị có liên quan.
- Hỗ trợ khuyến nghị thiết bị dựa trên dữ liệu của hệ thống.

Các khẳng định định lượng quan trọng trong câu trả lời AI phải có dữ liệu hoặc nguồn phù hợp; thông tin không có căn cứ không được sử dụng như dữ liệu xác thực.

8. Wiki và nội dung kiến thức

Người dùng có thể đọc các bài Wiki đã được xuất bản trên hệ thống.

Đối với người dùng có quyền phù hợp, hệ thống còn hỗ trợ:

- Tạo nội dung Wiki.
- Chỉnh sửa bài viết.
- Tạo phiên bản mới khi nội dung thay đổi.
- Liên kết bài viết với nguồn và trích dẫn.
- Bình luận trên nội dung theo quy tắc kiểm duyệt.

Chỉ các phiên bản Wiki đã được xuất bản mới được hiển thị công khai.

9. Danh sách yêu thích

Người dùng đã đăng nhập có thể:

- Tạo nhiều danh sách yêu thích.
- Thêm thiết bị vào danh sách.
- Xóa thiết bị khỏi danh sách.
- Quản lý các thiết bị đã lưu theo nhu cầu cá nhân.

Chức năng này giúp người dùng lưu lại các sản phẩm đang quan tâm để tiếp tục nghiên cứu hoặc so sánh về sau.

10. Theo dõi giá và cảnh báo

Người dùng có thể theo dõi giá của một biến thể thiết bị và thiết lập điều kiện cảnh báo.

Hệ thống hỗ trợ:

- Thiết lập mức giá mong muốn.
- Chọn biến thể cần theo dõi.
- Xác định đơn vị tiền tệ.
- Bật hoặc tắt cảnh báo.
- Lưu lịch sử biến động giá.
- Tạo thông báo khi điều kiện được đáp ứng.
- Hiện thị thông báo trong ứng dụng.
- Đánh dấu thông báo đã đọc.

3.1.1.2. Đối với biên tập viên và kiểm duyệt viên

Biên tập viên và kiểm duyệt viên tham gia trực tiếp vào quy trình bảo đảm chất lượng dữ liệu của SpecHub.

1. Quản lý dữ liệu danh mục

Biên tập viên có thể thực hiện các thao tác được cấp quyền đối với:

- Loại thiết bị.
- Tổ chức và thương hiệu.
- Dòng sản phẩm.
- Mẫu thiết bị.
- Biến thể thiết bị.
- Thành phần phần cứng.
- Dữ liệu phần mềm.
- Media.
- Benchmark và các dữ liệu liên quan.

Các thành phần dùng chung như CPU, GPU, camera hoặc chipset được quản lý thành bản ghi gốc và liên kết tới thiết bị thay vì sao chép dữ liệu trên từng sản phẩm.

2. Quản lý nguồn và trích dẫn

Biên tập viên có thể:

- Thêm nguồn dữ liệu.

- Liên kết nguồn với nội dung hoặc thực thể.
- Quản lý các trích dẫn.
- Kiểm tra nguồn trước khi dữ liệu được sử dụng công khai.
- Theo dõi dữ liệu được thu thập từ bên ngoài.

Nguồn và trích dẫn được quản lý độc lập để bảo đảm khả năng truy nguyên thông tin.

3. Quản lý nội dung Wiki

Biên tập viên có thể tạo và chỉnh sửa Wiki, trong khi kiểm duyệt viên có quyền phù hợp có thể:

- Xem phiên bản đang chờ duyệt.
- Kiểm tra nội dung.
- Chấp nhận và xuất bản phiên bản.
- Từ chối phiên bản và ghi nhận lý do.
- Kiểm duyệt bình luận.
- Quản lý nguồn và trích dẫn của bài viết.

4. Kiểm duyệt dữ liệu thu thập

Dữ liệu được thu thập từ các nguồn bên ngoài không được đưa trực tiếp vào danh mục công khai.

Quy trình yêu cầu:

- Lưu dữ liệu thu thập ở trạng thái chờ duyệt.
- Biên tập viên kiểm tra nội dung.
- Xác minh nguồn.
- Chuẩn hóa thông tin.
- Chỉ dữ liệu đạt yêu cầu mới được sử dụng cho catalog hoặc kho tri thức.

3.1.1.3. Đối với quản trị viên

Quản trị viên là tác nhân có phạm vi quyền lớn nhất trong hệ thống.

1. Quản lý người dùng và phân quyền

Quản trị viên có thể:

- Xem danh sách người dùng.
- Xem thông tin tài khoản.

- Thay đổi vai trò theo quy định.
- Khóa tài khoản.
- Xóa tài khoản khi đáp ứng điều kiện nghiệp vụ.
- Quản lý quyền truy cập các chức năng quản trị.

2. Quản trị Catalog Studio

Catalog Studio hỗ trợ quản lý dữ liệu theo quy trình bản nháp trước khi xuất bản.

Quản trị viên và người có quyền có thể:

- Tạo bản nháp.
- Lưu dữ liệu theo từng bước.
- Tiếp tục chỉnh sửa bản nháp.
- Phát hiện xung đột phiên bản.
- Xem lịch sử bản nháp.
- Khôi phục một phiên bản trước.
- Kiểm tra dữ liệu trước khi xuất bản.
- Phát hiện lỗi về trường dữ liệu, quan hệ, nguồn và độ phủ.
- Xuất bản dữ liệu theo giao dịch.

3. Quản lý hệ thống điểm

Quản trị viên có thể cấu hình các hồ sơ điểm dùng để đánh giá thiết bị.

Một hồ sơ điểm bao gồm:

- Các mô-đun đánh giá.
- Trọng số mô-đun.
- Các chỉ số thành phần.
- Trọng số chỉ số.
- Chiều đánh giá tốt hoặc xấu.
- Quy tắc chuẩn hóa.

Hệ thống phải kiểm tra tổng trọng số trước khi cho phép xuất bản. Phiên bản hồ sơ đã xuất bản được giữ ổn định để kết quả điểm có thể được tái lập.

4. Quản lý kho tri thức AI

Người có quyền quản trị hoặc biên tập có thể yêu cầu xây dựng lại chỉ mục của kho tri thức trong phạm vi dữ liệu đã được phê duyệt.

Dữ liệu chưa duyệt, dữ liệu riêng tư hoặc thông tin xác thực không được đưa vào kho tri thức phục vụ câu trả lời công khai.

5. Quản lý chức năng thương mại và dịch vụ

Trong phạm vi các chức năng mở rộng, quản trị viên có thể:

- Quản lý đối tác liên kết.
- Quản lý liên kết mua hàng.
- Theo dõi dữ liệu giá.
- Quản lý các gói dịch vụ.
- Theo dõi trạng thái thuê bao.
- Quản lý các cấu hình tích hợp cần thiết.

6. Theo dõi tình trạng vận hành

Khu vực quản trị cung cấp các chức năng hỗ trợ vận hành như:

- Kiểm tra trạng thái API.
- Kiểm tra mức sẵn sàng của các phụ thuộc chính.
- Theo dõi một số chỉ số hệ thống.
- Tra cứu mã định danh của yêu cầu.
- Xem dữ liệu tổng hợp phục vụ quản trị.

3.1.1.4. Đối với đối tác B2B

SpecHub cho phép các đối tác được cấp quyền khai thác một phần dữ liệu thông qua API.

Các chức năng gồm:

- Tạo khóa API.
- Chỉ hiển thị giá trị bí mật của khóa tại thời điểm tạo.
- Luân chuyển khóa API.
- Thu hồi khóa API.
- Giới hạn phạm vi dữ liệu được phép truy cập.
- Kiểm soát số lượng Request theo khoảng thời gian.
- Quản lý hạn mức sử dụng.
- Ghi nhận số lần gọi API và kết quả sử dụng.

Đối tác không mặc nhiên có quyền truy cập toàn bộ dữ liệu của hệ thống mà chỉ có thể sử dụng các tài nguyên nằm trong phạm vi được cấp.

3.1.1.5. Đối với tiến trình nền

Một số nghiệp vụ được thực hiện tự động mà không cần tương tác trực tiếp của người dùng.

Tiến trình nền của SpecHub có các nhiệm vụ chính:

- Kiểm tra điều kiện cảnh báo giá.
- Thu thập nội dung từ các nguồn được cấu hình.
- Xử lý hàng đợi gửi thư thông báo.
- Thử lại các tác vụ thất bại theo cơ chế được quy định.
- Không chạy chồng cùng một tác vụ trong cùng tiến trình.

Các tiến trình nền giúp tách những công việc định kỳ hoặc kéo dài khỏi luồng Request/Response của người dùng.

3.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả những tiêu chí chất lượng mà hệ thống phải đáp ứng trong quá trình vận hành. Đối với SpecHub, các yêu cầu chính tập trung vào **an toàn thông tin, tính toàn vẹn dữ liệu, hiệu năng, khả năng phục hồi, khả năng bảo trì và trải nghiệm sử dụng**.

3.1.2.1. Yêu cầu về an toàn thông tin

Hệ thống cần bảo đảm:

- API từ chối các thuộc tính dữ liệu không được khai báo trong DTO.
- Các endpoint cần bảo vệ phải yêu cầu JWT theo mặc định.
- Endpoint công khai phải được khai báo rõ ràng.
- Quyền truy cập được kiểm tra theo vai trò và điều kiện nghiệp vụ.
- Mật khẩu người dùng phải được băm trước khi lưu.
- Khóa API chỉ lưu giá trị băm thay vì lưu bí mật gốc.
- Các thông tin bí mật không được đưa trực tiếp vào mã nguồn.
- CORS chỉ cho phép các nguồn giao diện đã được cấu hình.
- Sử dụng các HTTP Header bảo mật phù hợp.
- Webhook thanh toán phải kiểm tra chữ ký trước khi xử lý.
- Bộ thu thập dữ liệu phải kiểm soát địa chỉ và loại nội dung để hạn chế truy cập tới các tài nguyên không được phép.

3.1.2.2. Yêu cầu về tính đúng và toàn vẹn dữ liệu

Do SpecHub quản lý số lượng lớn thực thể có quan hệ với nhau, tính chính xác của dữ liệu là yêu cầu quan trọng.

Hệ thống cần:

- Sử dụng khóa chính và khóa ngoại phù hợp.
- Áp dụng ràng buộc duy nhất đối với các trường cần thiết.
- Sử dụng chỉ mục cho các trường được truy vấn thường xuyên.
- Kiểm soát migration trước khi áp dụng.
- Không đặt lại cơ sở dữ liệu khi chứa dữ liệu cần giữ.
- Sử dụng revision để phát hiện xung đột khi nhiều thao tác chỉnh sửa cùng một bản nháp.
- Ngăn việc một tác vụ hoặc webhook bị xử lý lặp ngoài ý muốn.
- Phân biệt rõ dữ liệu thô, dữ liệu bản nháp và dữ liệu đã xuất bản.

Đối với dữ liệu thiết bị, các trường chưa xác định không được tự động coi là giá trị bằng không vì điều này có thể làm sai kết quả lọc, so sánh hoặc chấm điểm.

3.1.2.3. Yêu cầu về hiệu năng và khả năng mở rộng

Hệ thống cần đáp ứng các nguyên tắc:

- Danh sách dữ liệu lớn phải được phân trang.
- Các trường tìm kiếm và lọc quan trọng cần có chỉ mục thích hợp.
- Quá trình so sánh nhiều thiết bị phải ưu tiên tải dữ liệu theo lô.
- Tránh thực hiện truy vấn lặp lại cho từng phần tử khi có thể tải cùng lúc.
- Các tác vụ dài hoặc định kỳ phải được xử lý ngoài luồng Request của người dùng.
- Redis có thể được sử dụng cho phiên và dữ liệu tạm.
- Meilisearch có thể được sử dụng để tăng cường chức năng tìm kiếm.

Hệ thống hiện chưa đặt ra một ngưỡng độ trễ cụ thể cho tất cả API khi chưa có bộ kiểm thử tải có khả năng tái lập. Vì vậy, không sử dụng những cam kết định lượng chưa được kiểm chứng trong yêu cầu hiệu năng.

3.1.2.4. Yêu cầu về khả dụng và khả năng phục hồi

SpecHub phụ thuộc vào nhiều thành phần khác nhau, do đó hệ thống phải xử lý được trường hợp một dịch vụ hỗ trợ tạm thời không khả dụng.

Các yêu cầu gồm:

- Phân biệt trạng thái tiến trình đang chạy với trạng thái các phụ thuộc đã sẵn sàng.
- Nếu dịch vụ AI bên ngoài không khả dụng, hệ thống sử dụng phương án cục bộ hoặc trả lời có kiểm soát.
- Nếu chức năng gửi thư bị tắt, thông báo trong ứng dụng vẫn được lưu.
- Tác vụ nền không được tự chạy chồng nhiều phiên của cùng công việc trong một tiến trình.
- Quy trình triển khai cần có cơ chế sao lưu, phục hồi và quay lui khi cần thiết.

3.1.2.5. Yêu cầu về khả năng bảo trì

Mã nguồn cần được tổ chức sao cho có thể tiếp tục phát triển khi số lượng chức năng và loại thiết bị tăng lên.

Các yêu cầu gồm:

- Phân chia mã nguồn theo từng ứng dụng và package dùng chung.
- Backend được chia thành các module theo miền nghiệp vụ.
- Áp dụng quy tắc đặt tên thống nhất cho tệp, lớp, kiểu dữ liệu, biến và hàm.
- Thay đổi mã nguồn phải vượt qua quá trình kiểm tra lint.
- Không được phát sinh lỗi TypeScript.
- Các kiểm thử liên quan phải đạt yêu cầu.
- Các thành phần cần build phải có khả năng biên dịch thành công.
- Tài liệu và đặc tả phải được cập nhật khi có những thay đổi đáng kể về cấu trúc hệ thống.

3.1.2.6. Yêu cầu về giao diện và trải nghiệm người dùng

Giao diện là thành phần trực tiếp phục vụ quá trình tra cứu và nghiên cứu thiết bị nên cần bảo đảm tính rõ ràng và nhất quán.

Hệ thống cần:

- Hiển thị phù hợp trên máy tính và thiết bị di động.
- Phân biệt rõ trạng thái đang tải, không có dữ liệu, lỗi và ngoại tuyến.
- Các thành phần điều khiển phải có nhãn phù hợp.
- Hình ảnh cần có nội dung mô tả thay thế khi cần thiết.
- Bảo đảm mức tương phản đủ để đọc nội dung.
- Không hiển thị dữ liệu chưa xác định như giá trị bằng không.
- Kết quả chấm điểm cần có thông tin giải thích hoặc phương pháp.
- Kết quả AI cần cung cấp nguồn hoặc thông tin căn cứ khi phù hợp.

1.2. Phân tích Use-case

3.2.1. Biểu đồ Use Case tổng quát

Hình 3.1. Use Case tổng quát hệ thống SpecHub

Biểu đồ Use Case tổng quát mô tả các nhóm tác nhân và những chức năng chính mà mỗi nhóm có thể thực hiện trên hệ thống.

- **Khách truy cập:** Có thể duyệt danh mục thiết bị, tìm kiếm, lọc, xem chi tiết thiết bị và phần cứng, so sánh, nhận khuyến nghị, đọc Wiki và sử dụng một số chức năng AI công khai. Hệ thống đồng thời cho phép lưu tạm vùng nghiên cứu trên trình duyệt mà không bắt buộc phải tạo tài khoản.
- **Người dùng:** Sau khi đăng nhập, ngoài các chức năng công khai, người dùng có thể quản lý hồ sơ cá nhân, danh sách yêu thích, cảnh báo giá, thông báo, dữ liệu nghiên cứu đã lưu và các chức năng cá nhân hóa khác.
- **Biên tập viên:** Có nhiệm vụ cập nhật và chuẩn hóa dữ liệu của thư viện. Biên tập viên được phép tạo và chỉnh sửa dữ liệu danh mục, quản lý nguồn và trích dẫn, làm việc với bản nháp Catalog Studio và cập nhật nội dung trong phạm vi được phân quyền.
- **Kiểm duyệt viên:** Thực hiện kiểm tra những nội dung cần phê duyệt, đặc biệt là phiên bản Wiki và các nội dung được ủy quyền. Kiểm duyệt viên có thể chấp nhận hoặc từ chối nội dung trước khi dữ liệu được công khai.
- **Quản trị viên:** Có quyền quản trị toàn bộ hệ thống, bao gồm người dùng, vai trò, danh mục, hồ sơ chấm điểm, cấu hình dịch vụ, quản trị nội dung và theo dõi trạng thái vận hành.
- **Đối tác B2B:** Khai thác dữ liệu SpecHub thông qua API Key. Quyền truy cập được giới hạn theo phạm vi, hạn mức và chính sách sử dụng được cấp cho từng khóa.

Các chức năng công khai như xem danh mục, tìm kiếm, xem chi tiết và đọc Wiki không bắt buộc người dùng đăng nhập. Những chức năng liên quan đến dữ liệu cá nhân, chỉnh sửa nội dung và quản trị yêu cầu quá trình xác thực và kiểm tra quyền tương ứng.

3.2.2. Biểu đồ phân rã Use Case

a. Biểu đồ phân rã Use Case đăng ký tài khoản

Hình 3.2. Use Case đăng ký tài khoản

Khách truy cập có thể tạo tài khoản để sử dụng các chức năng cá nhân hóa của SpecHub. Người dùng nhập các thông tin cần thiết như địa chỉ thư điện tử và mật khẩu. Hệ thống kiểm tra tính hợp lệ của dữ liệu và bảo đảm địa chỉ thư điện tử chưa được sử dụng trước khi tạo tài khoản.

Use Case đăng ký bao gồm các chức năng:

- Nhập thông tin đăng ký.
- Kiểm tra dữ liệu đầu vào.
- Kiểm tra tài khoản đã tồn tại.
- Bấm mật khẩu.
- Tạo tài khoản.
- Thông báo kết quả đăng ký.

Mật khẩu không được lưu trực tiếp mà phải được băm trước khi ghi vào cơ sở dữ liệu.

b. Biểu đồ phân rã Use Case đăng nhập và quản lý phiên

Hình 3.3. Use Case đăng nhập và quản lý phiên

Người dùng nhập thư điện tử và mật khẩu để truy cập tài khoản. Hệ thống xác thực thông tin và tạo phiên đăng nhập nếu thông tin hợp lệ.

Use Case bao gồm:

- Nhập thông tin đăng nhập.
- Kiểm tra thông tin xác thực.
- Tạo phiên đăng nhập.
- Cấp access token.
- Làm mới phiên đăng nhập.
- Đăng xuất.
- Thu hồi phiên.

Việc làm mới chỉ được thực hiện khi phiên tương ứng vẫn còn hiệu lực trong Redis. Khi người dùng đăng xuất, phiên hiện tại được thu hồi tại phía máy chủ.

c. Biểu đồ phân rã Use Case quản lý hồ sơ cá nhân

Hình 3.4. Use Case quản lý hồ sơ cá nhân

Sau khi đăng nhập, người dùng có thể truy cập khu vực hồ sơ để xem và quản lý thông tin cá nhân.

Các chức năng chính gồm:

- Xem thông tin tài khoản.

- Cập nhật hồ sơ.
- Thay đổi thông tin cá nhân được cho phép.
- Đổi mật khẩu.
- Xem các chức năng cá nhân hóa liên quan đến tài khoản.

Chỉ chủ sở hữu tài khoản mới được phép thay đổi hồ sơ cá nhân của mình.

d. Biểu đồ phân rã Use Case tra cứu và tìm kiếm thiết bị

Hình 3.5. Use Case tra cứu và tìm kiếm thiết bị

Đây là một trong những nhóm Use Case trọng tâm của SpecHub. Người dùng có thể duyệt thư viện hoặc tìm kiếm trực tiếp thiết bị và phần cứng cần nghiên cứu.

Các chức năng gồm:

- Xem danh mục thiết bị.
- Xem thiết bị theo thương hiệu hoặc dòng sản phẩm.
- Tìm kiếm theo từ khóa.
- Tìm kiếm theo tên hoặc alias.
- Lọc theo thuộc tính.
- Phân trang kết quả.
- Xem thông tin chi tiết của mẫu thiết bị.
- Xem các biến thể.
- Tra cứu phần cứng liên quan.

Kết quả tìm kiếm có thể sử dụng PostgreSQL hoặc công cụ tìm kiếm được cấu hình, trong khi bộ lọc được phản ánh lên URL để có thể tái lập trạng thái tìm kiếm.

e. Biểu đồ phân rã Use Case xem chi tiết thiết bị

Hình 3.6. Use Case xem chi tiết thiết bị

Sau khi lựa chọn một thiết bị, người dùng có thể xem toàn bộ dữ liệu đã được công khai của mẫu thiết bị đó.

Use Case xem chi tiết bao gồm:

- Xem thông tin tổng quan.
- Xem các biến thể.
- Xem cấu hình phần cứng.
- Xem màn hình.

- Xem camera.
- Xem pin và sạc.
- Xem bộ nhớ và lưu trữ.
- Xem phần mềm.
- Xem benchmark.
- Xem điểm đánh giá.
- Xem hình ảnh và media.
- Xem nguồn và trích dẫn.
- Xem thông tin giá nếu có.

Người dùng cũng có thể tiếp tục truy cập từ thiết bị sang các trang chi tiết của chipset, CPU, GPU hoặc các linh kiện liên quan.

f. Biểu đồ phân rã Use Case so sánh và khuyến nghị thiết bị

Hình 3.7. Use Case so sánh và khuyến nghị thiết bị

Người dùng có thể chọn nhiều biến thể để thực hiện so sánh trên cùng một hệ thống thuộc tính.

Nhóm chức năng so sánh bao gồm:

- Chọn thiết bị cần so sánh.
- Thêm hoặc loại bỏ thiết bị khỏi danh sách.
- Tải dữ liệu các biến thể.
- So sánh thông số kỹ thuật.
- Làm nổi bật điểm khác biệt.
- Phân biệt dữ liệu thiếu và giá trị bằng không.
- Chia sẻ bảng so sánh thông qua URL.

Bên cạnh so sánh trực tiếp, người dùng có thể nhập nhu cầu, ngân sách và các tiêu chí ưu tiên để hệ thống đưa ra danh sách thiết bị phù hợp kèm giải thích.

g. Biểu đồ phân rã Use Case trợ lý AI

Hình 3.8. Use Case trợ lý AI

Trợ lý AI hỗ trợ người dùng khai thác kho dữ liệu SpecHub bằng ngôn ngữ tự nhiên.

Các chức năng chính gồm:

- Nhập câu hỏi.

- Phân tích truy vấn.
- Truy xuất thông tin liên quan.
- Tìm kiếm ngữ nghĩa.
- Xây dựng ngữ cảnh.
- Sinh câu trả lời.
- Trả nội dung theo luồng.
- Kiểm tra dữ liệu định lượng.
- Gắn nguồn và trích dẫn.
- Nghiên cứu thông tin phân cứng.
- Đưa ra khuyến nghị khi phù hợp.

Hệ thống yêu cầu câu trả lời phải dựa trên dữ liệu được truy xuất. Các khẳng định định lượng quan trọng không có nguồn hoặc dữ liệu hỗ trợ phải được loại khỏi kết quả.

h. Biểu đồ phân rã Use Case danh sách yêu thích và cảnh báo giá

Hình 3.9. Use Case danh sách yêu thích và cảnh báo giá

Người dùng đã đăng nhập có thể lưu những thiết bị đang quan tâm để tiếp tục theo dõi.

Các chức năng gồm:

- Tạo danh sách yêu thích.
- Thêm thiết bị vào danh sách.
- Xóa thiết bị khỏi danh sách.
- Quản lý nhiều danh sách.
- Thiết lập cảnh báo giá.
- Xác định mức giá mong muốn.
- Bật hoặc tắt cảnh báo.
- Xem lịch sử giá.
- Nhận thông báo khi điều kiện được đáp ứng.
- Đánh dấu thông báo đã đọc.

Tiến trình nền định kỳ kiểm tra điều kiện cảnh báo và tạo thông báo khi mức giá đạt yêu cầu của người dùng.

i. Biểu đồ phân rã Use Case Wiki

Hình 3.10. Use Case quản lý Wiki

Hệ thống Wiki cung cấp khu vực nội dung kiến thức bên cạnh dữ liệu thông số có cấu trúc.

Đối với người dùng thông thường, các chức năng gồm:

- Xem danh sách bài Wiki.
- Đọc bài đã xuất bản.
- Xem nguồn và trích dẫn.
- Bình luận khi được phép.

Đối với người có quyền biên tập:

- Tạo bài viết.
- Chỉnh sửa nội dung.
- Tạo phiên bản sửa mới.
- Quản lý trích dẫn.

Đối với kiểm duyệt viên:

- Xem phiên bản chờ duyệt.
- Phê duyệt phiên bản.
- Từ chối phiên bản.
- Ghi nhận lý do từ chối.
- Kiểm duyệt nội dung liên quan.

Mỗi lần chỉnh sửa tạo ra phiên bản mới thay vì ghi đè trực tiếp lên lịch sử bài viết.

j. Biểu đồ phân rã Use Case quản lý Catalog Studio

Hình 3.11. Use Case quản lý Catalog Studio

Catalog Studio là nhóm chức năng phục vụ biên tập viên và quản trị viên trong quá trình xây dựng và chuẩn hóa thư viện thiết bị.

Các Use Case chính gồm:

- Tìm kiếm dữ liệu danh mục.
- Tạo bản nháp.
- Nhập và chỉnh sửa dữ liệu.
- Lưu bản nháp.
- Kiểm tra revision.
- Xem lịch sử chỉnh sửa.
- Khôi phục phiên bản.
- Kiểm tra dữ liệu trước xuất bản.
- Kiểm tra quan hệ giữa các thực thể.
- Kiểm tra nguồn.
- Quản lý media.

- Hoàn tất và xuất bản dữ liệu.

Việc xuất bản được thực hiện theo giao dịch nhằm tránh trường hợp chỉ một phần của dữ liệu được cập nhật thành công.

k. Biểu đồ phân rã Use Case quản lý hệ thống điểm

Hình 3.12. Use Case quản lý hệ thống điểm

Hệ thống điểm cho phép quản trị viên cấu hình phương pháp đánh giá thiết bị dựa trên các mô-đun và chỉ số.

Các chức năng gồm:

- Tạo hồ sơ điểm.
- Cấu hình mô-đun.
- Cấu hình chỉ số.
- Xác định trọng số.
- Xác định chiều đánh giá.
- Thiết lập quy tắc chuẩn hóa.
- Kiểm tra tổng trọng số.
- Xuất bản hồ sơ điểm.
- Tính điểm thiết bị.
- Lưu điểm mô-đun.
- Lưu điểm tổng.
- Lưu phiên bản hồ sơ và dữ liệu đầu vào.

Hồ sơ điểm đã xuất bản được lưu theo phiên bản để kết quả đã tính có thể được kiểm tra và tái lập về sau.

l. Biểu đồ phân rã Use Case quản trị hệ thống

Hình 3.13. Use Case quản trị hệ thống

Nhóm chức năng này dành cho quản trị viên và yêu cầu người dùng phải đăng nhập với quyền tương ứng.

Các chức năng chính gồm:

- Quản lý người dùng.
- Thay đổi vai trò.

- Khóa hoặc xử lý tài khoản.
- Quản lý danh mục thiết bị.
- Quản lý dữ liệu phần cứng.
- Quản lý Catalog Studio.
- Quản lý hồ sơ điểm.
- Quản lý gói dịch vụ.
- Quản lý các cấu hình liên quan.
- Xem thông tin vận hành.
- Kiểm tra trạng thái hệ thống.
- Theo dõi dữ liệu tổng hợp tại bảng điều khiển quản trị.

Các thao tác quản trị được kiểm soát thông qua cơ chế xác thực, vai trò và các điều kiện nghiệp vụ tại Backend.

m. Biểu đồ phân rã Use Case API dành cho đối tác B2B

Hình 3.14. Use Case khai thác API B2B

Đối tác B2B sử dụng API Key để khai thác dữ liệu được SpecHub cho phép.

Các chức năng liên quan gồm:

- Tạo API Key.
- Xem thông tin khóa.
- Luân chuyển khóa.
- Thu hồi khóa.
- Xác thực API Key.
- Kiểm tra phạm vi quyền.
- Kiểm tra giới hạn tần suất.
- Kiểm tra hạn mức sử dụng.
- Truy cập dữ liệu được cấp phép.
- Theo dõi mức sử dụng API.

Khóa bí mật chỉ được trả về khi tạo và cơ sở dữ liệu chỉ lưu giá trị băm. Mỗi khóa được giới hạn theo phạm vi và hạn mức nhằm bảo vệ tài nguyên của hệ thống.

1.3. Thiết kế sơ đồ quy trình nghiệp vụ

3.3.1. Quy trình nghiệp vụ đăng ký, đăng nhập

Hình 3.15. Sơ đồ nghiệp vụ đăng ký, đăng nhập

Quy trình đăng ký

Người dùng chưa có tài khoản có thể thực hiện đăng ký để sử dụng các chức năng cá nhân hóa của SpecHub.

Bước 1: Nhập thông tin đăng ký

Người dùng truy cập trang đăng ký và nhập các thông tin được yêu cầu, trong đó địa chỉ thư điện tử và mật khẩu là các thông tin chính phục vụ xác thực.

Bước 2: Kiểm tra dữ liệu

Giao diện gửi dữ liệu đăng ký đến Backend. Hệ thống kiểm tra:

- Định dạng các trường dữ liệu.
- Tính hợp lệ của địa chỉ thư điện tử.
- Các yêu cầu đối với mật khẩu.
- Sự tồn tại của tài khoản sử dụng cùng địa chỉ thư điện tử.

Nếu dữ liệu không hợp lệ hoặc tài khoản đã tồn tại, hệ thống trả về thông báo lỗi và không tạo tài khoản mới.

Bước 3: Tạo tài khoản

Nếu dữ liệu hợp lệ, mật khẩu được băm trước khi lưu vào cơ sở dữ liệu. Hệ thống tạo tài khoản và gán các thông tin mặc định theo chính sách hiện hành.

Bước 4: Hoàn tất đăng ký

Sau khi tài khoản được tạo thành công, hệ thống thông báo kết quả để người dùng có thể thực hiện đăng nhập. Yêu cầu của SpecHub xác định tài khoản phải có địa chỉ thư điện tử duy nhất và mật khẩu phải được lưu dưới dạng đã băm.

Quy trình đăng nhập

Bước 1: Gửi thông tin xác thực

Người dùng nhập địa chỉ thư điện tử và mật khẩu. Web gửi yêu cầu đăng nhập đến Auth API.

Bước 2: Xác thực tài khoản

Backend tìm tài khoản trong PostgreSQL và đối chiếu mật khẩu người dùng nhập với mật khẩu đã được băm trong cơ sở dữ liệu.

Nếu thông tin không chính xác hoặc tài khoản không đáp ứng điều kiện truy cập, hệ thống từ chối đăng nhập.

Bước 3: Khởi tạo phiên

Khi xác thực thành công, Backend lấy hồ sơ và vai trò của người dùng, sau đó tạo một phiên có thời hạn trong Redis.

Bước 4: Cấp thông tin xác thực

Backend trả access token cho Web và thiết lập thông tin phục vụ quá trình làm mới phiên.

Bước 5: Làm mới phiên

Khi access token hết hạn, Web gửi yêu cầu làm mới. Backend kiểm tra phiên tương ứng trong Redis. Nếu phiên còn hợp lệ, một access token mới được cấp.

Bước 6: Đăng xuất

Khi người dùng đăng xuất, phiên hiện tại được thu hồi tại phía máy chủ để không thể tiếp tục sử dụng cho quá trình làm mới.

Luồng chính của quy trình có thể biểu diễn:

Người dùng → Web → Auth API → PostgreSQL → Redis → Web → Người dùng

3.3.2. Quy trình nghiệp vụ tìm kiếm, tra cứu và so sánh thiết bị

Hình 3.16. Sơ đồ nghiệp vụ tìm kiếm và so sánh thiết bị

Đây là một trong những quy trình chính của SpecHub, cho phép người dùng khai thác thư viện thiết bị từ thao tác tìm kiếm đến việc đối chiếu các sản phẩm.

Giai đoạn 1: Tìm kiếm thiết bị

Bước 1: Nhập điều kiện tìm kiếm

Người dùng nhập từ khóa hoặc lựa chọn các điều kiện lọc trên giao diện Web.

Điều kiện có thể liên quan đến:

- Tên thiết bị.
- Thương hiệu.
- Dòng sản phẩm.
- Biến thể.
- Thành phần phần cứng.
- Các thông số kỹ thuật.

Bước 2: Gửi yêu cầu tìm kiếm

Web chuyển từ khóa và các bộ lọc đến API.

Bước 3: Chuẩn hóa truy vấn

Backend kiểm tra và chuẩn hóa các tham số tìm kiếm trước khi thực hiện truy vấn dữ liệu.

Tùy cấu hình của hệ thống, quá trình tìm kiếm có thể được thực hiện bằng PostgreSQL hoặc sử dụng Meilisearch để tăng cường khả năng tìm kiếm toàn văn.

Bước 4: Trả danh sách kết quả

Kết quả tìm kiếm được trả về cùng thông tin phân trang. Web hiển thị danh sách để người dùng lựa chọn thiết bị phù hợp.

Giai đoạn 2: Xem chi tiết thiết bị

Khi người dùng chọn một thiết bị, Web gửi yêu cầu lấy thông tin chi tiết.

Backend truy xuất:

- Thông tin mẫu thiết bị.
- Các biến thể.
- Chipset, CPU, GPU và các thành phần xử lý.
- Màn hình.
- Camera.
- Pin và sạc.
- Bộ nhớ và lưu trữ.
- Hệ điều hành.
- Benchmark và điểm đánh giá.
- Media.
- Nguồn và trích dẫn.

Các dữ liệu được tổng hợp và trả về giao diện theo từng nhóm thông số.

Giai đoạn 3: So sánh thiết bị

Bước 1: Người dùng thêm các biến thể cần so sánh vào danh sách.

Bước 2: Web gửi danh sách định danh biến thể đến Backend.

Bước 3: API tải theo lô dữ liệu của các biến thể cùng những quan hệ phần cứng liên quan.

Bước 4: Dữ liệu được chuẩn hóa về cùng hệ thuộc tính.

Bước 5: Web xây dựng ma trận so sánh và đánh dấu các điểm khác biệt.

Bước 6: Những trường chưa có thông tin được thể hiện ở trạng thái chưa xác định thay vì được xem là giá trị bằng không.

Luồng tổng quát:

Người dùng → Nhập từ khóa → Tìm kiếm → Xem thiết bị → Chọn biến thể → So sánh → Hiển thị kết quả

3.3.3. Quy trình nghiệp vụ quản lý và xuất bản dữ liệu bằng Catalog Studio

Hình 3.17. Sơ đồ nghiệp vụ quản lý dữ liệu Catalog Studio

Catalog Studio là quy trình nghiệp vụ trọng tâm đối với việc xây dựng và duy trì thư viện dữ liệu SpecHub. Dữ liệu mới không được đưa trực tiếp vào danh mục công khai mà được xử lý theo cơ chế bản nháp, kiểm tra và xuất bản có kiểm soát.

Giai đoạn 1: Kiểm tra dữ liệu hiện có

Biên tập viên hoặc quản trị viên trước tiên tìm kiếm các thực thể đang tồn tại trong hệ thống.

Mục đích của bước này là hạn chế việc tạo trùng:

- Thương hiệu.
- Dòng sản phẩm.
- Mẫu thiết bị.
- Biến thể.
- Chipset.
- CPU.
- GPU.
- Camera.
- Màn hình và các linh kiện khác.

Nếu thực thể đã tồn tại, người biên tập sử dụng hoặc cập nhật thực thể hiện có thay vì tạo bản ghi mới.

Giai đoạn 2: Tạo bản nháp

Người có quyền tạo một bản nháp mới hoặc tiếp tục chỉnh sửa bản nháp đã lưu.

Bản nháp lưu:

- Nội dung đang nhập.
- Bước hiện tại.
- Phiên bản chỉnh sửa.
- Thông tin phục vụ phát hiện xung đột.

Giai đoạn 3: Nhập và chuẩn hóa dữ liệu

Người biên tập lần lượt nhập các nhóm dữ liệu:

- Thông tin chung.
- Mẫu thiết bị.
- Biến thể.
- Chipset và bộ xử lý.
- Màn hình.
- Camera.
- Pin.
- Bộ nhớ.
- Phần mềm.
- Media.
- Nguồn tham chiếu.

Các thành phần đã tồn tại được liên kết với thiết bị thay vì sao chép dữ liệu mới.

Giai đoạn 4: Lưu bản nháp

Trong quá trình chỉnh sửa, hệ thống lưu dữ liệu cùng *revision*.

Nếu một yêu cầu cập nhật sử dụng *revision* cũ, hệ thống có thể từ chối thao tác để tránh một phiên chỉnh sửa vô tình ghi đè lên dữ liệu mới hơn.

Giai đoạn 5: Kiểm tra trước khi xuất bản

Trước khi cho phép xuất bản, hệ thống kiểm tra:

- Trường dữ liệu bắt buộc.
- Quan hệ giữa các thực thể.
- Nguồn và trích dẫn.
- Media.
- Độ phủ thông tin.
- Dữ liệu liên quan đến hệ thống điểm.

Nếu còn lỗi, hệ thống trả danh sách các nội dung cần bổ sung hoặc chỉnh sửa.

Giai đoạn 6: Xuất bản

Khi dữ liệu đạt yêu cầu và người có quyền xác nhận, Backend thực hiện quá trình xuất bản trong một *transaction*.

Hệ thống:

- Tạo hoặc cập nhật các thực thể cần thiết.
- Ghi nhận lịch sử.
- Cập nhật dữ liệu công khai.
- Hoàn tất trạng thái bản nháp.

Việc sử dụng transaction giúp tránh tình trạng chỉ một phần dữ liệu được cập nhật khi quá trình xuất bản gặp lỗi.

Giai đoạn 7: Cập nhật chỉ mục

Nếu dữ liệu vừa xuất bản thuộc phạm vi tìm kiếm hoặc kho tri thức AI, hệ thống thực hiện cập nhật lại chỉ mục tương ứng.

Quy trình tổng quát:

Tìm dữ liệu hiện có → Tạo bản nháp → Nhập dữ liệu → Lưu → Kiểm tra → Xác nhận → Xuất bản → Cập nhật chỉ mục

3.3.4. Quy trình nghiệp vụ thu thập và kiểm duyệt dữ liệu

Hình 3.18. Sơ đồ nghiệp vụ thu thập và kiểm duyệt dữ liệu

Thông tin thiết bị có thể đến từ nhiều nguồn bên ngoài. Tuy nhiên, dữ liệu thu thập tự động không được đưa trực tiếp vào thư viện công khai. SpecHub áp dụng một quy trình trung gian để kiểm tra và chuẩn hóa dữ liệu trước khi xuất bản.

Giai đoạn 1: Thu thập dữ liệu

Tiến trình nền truy cập những nguồn đã được cấu hình và cho phép.

Trước khi tiếp nhận nội dung, hệ thống kiểm tra:

- Địa chỉ URL.
- Nguồn dữ liệu.
- Loại nội dung.
- Kích thước dữ liệu.

- Các điều kiện an toàn liên quan.

Giai đoạn 2: Lưu dữ liệu thô

Nội dung sau khi thu thập được lưu dưới dạng **dữ liệu thô** với trạng thái chờ duyệt.

Dữ liệu này chưa được xem là dữ liệu chính thức của SpecHub và không được tự động xuất hiện trong danh mục công khai.

Giai đoạn 3: Biên tập viên đánh giá

Biên tập viên mở dữ liệu đang chờ và kiểm tra:

- Nội dung có phù hợp hay không.
- Nguồn có đáng tin cậy hay không.
- Thực thể tương ứng đã tồn tại hay chưa.
- Thông tin có xung đột với dữ liệu hiện tại hay không.
- Dữ liệu cần chuẩn hóa những trường nào.

Giai đoạn 4: Quyết định kiểm duyệt

Nếu dữ liệu không phù hợp, biên tập viên từ chối và lưu lý do.

Nếu dữ liệu được chấp nhận, nội dung được chuyển thành dữ liệu phục vụ quá trình biên tập hoặc bản nháp.

Giai đoạn 5: Kiểm tra và xuất bản

Dữ liệu tiếp tục đi qua quy trình kiểm tra của Catalog Studio trước khi có thể trở thành dữ liệu công khai.

Giai đoạn 6: Lập chỉ mục

Sau khi dữ liệu được xuất bản, hệ thống có thể cập nhật chỉ mục tìm kiếm và kho tri thức AI nếu dữ liệu thuộc phạm vi tương ứng.

Quy trình được mô tả:

Nguồn được duyệt → Thu thập → Kiểm tra → Dữ liệu thô → Biên tập viên → Bản nháp → Kiểm tra → Xuất bản → Lập chỉ mục

Nguyên tắc quan trọng của quy trình là:

Dữ liệu thu thập ≠ Dữ liệu công khai

Chỉ dữ liệu đã trải qua quá trình kiểm tra và xuất bản mới trở thành một phần chính thức của thư viện SpecHub.

3.3.5. Quy trình nghiệp vụ hỏi đáp và nghiên cứu thiết bị bằng AI

Hình 3.19. Sơ đồ nghiệp vụ hỏi đáp với trợ lý AI

Trợ lý AI cho phép người dùng khai thác dữ liệu thiết bị bằng ngôn ngữ tự nhiên. Khác với quá trình tìm kiếm thông thường, hệ thống cần xác định ý định, truy xuất dữ liệu liên quan và kiểm tra căn cứ trước khi trả lời.

Giai đoạn 1: Tiếp nhận yêu cầu

Người dùng nhập câu hỏi hoặc cung cấp nhu cầu thiết bị.

Ví dụ:

- Tra cứu một thông số.
- So sánh hai thiết bị.
- Tìm thiết bị phù hợp với ngân sách.
- Nghiên cứu một chipset hoặc GPU.
- Yêu cầu giải thích sự khác biệt giữa các linh kiện.

Giai đoạn 2: Xác định ý định

Backend phân loại yêu cầu thành các nhóm như:

- Tra cứu.
- So sánh.
- Xếp hạng.
- Khuyến nghị.

- Nghiên cứu phân cứng.

Việc phân loại giúp hệ thống xác định loại dữ liệu cần truy xuất.

Giai đoạn 3: Truy xuất dữ liệu

Hệ thống tìm dữ liệu trong phạm vi được phép từ:

- Danh mục thiết bị.
- Thành phần phần cứng.
- Benchmark.
- Hệ thống điểm.
- Wiki đã xuất bản.
- Nguồn và trích dẫn.
- Dữ liệu đã được duyệt.

Đối với tìm kiếm ngữ nghĩa, hệ thống có thể sử dụng embedding và dữ liệu véc-tơ để xác định nội dung liên quan.

Giai đoạn 4: Xây dựng ngữ cảnh

Các dữ liệu được lựa chọn được tổ chức thành ngữ cảnh phục vụ việc sinh câu trả lời.

Đối với yêu cầu khuyến nghị, ngữ cảnh còn chứa các tiêu chí, mức độ phù hợp và những điểm đánh đổi giữa các lựa chọn.

Giai đoạn 5: Sinh câu trả lời

Ngữ cảnh và câu hỏi được gửi đến bộ sinh câu trả lời đã được cấu hình.

Trong triển khai cục bộ của đề án, mô hình được vận hành thông qua Ollama.

Giai đoạn 6: Kiểm tra căn cứ

Kết quả không được trả trực tiếp cho người dùng ngay sau khi mô hình sinh nội dung.

Hệ thống kiểm tra:

- Nội dung có được hỗ trợ bởi ngữ cảnh hay không.
- Các giá trị định lượng có nguồn hay không.
- Trích dẫn có tham chiếu tới dữ liệu tồn tại hay không.

- Có nội dung nào vượt khỏi dữ liệu truy xuất hay không.

Phần không có căn cứ được loại bỏ hoặc được xử lý lại theo quy tắc của hệ thống.

Giai đoạn 7: Trả kết quả

Web nhận:

- Nội dung câu trả lời.
- Trích dẫn.
- Các giới hạn hoặc lưu ý cần thiết.

Kết quả sau đó được hiển thị cho người dùng.

Luồng tổng quát:

Câu hỏi → Xác định ý định → Truy xuất dữ liệu → Xây dựng ngữ cảnh → Sinh câu trả lời → Kiểm tra căn cứ → Gắn trích dẫn → Hiển thị

3.3.6. Quy trình nghiệp vụ cảnh báo giá thiết bị

Hình 3.20. Sơ đồ nghiệp vụ cảnh báo giá

Chức năng cảnh báo giá cho phép người dùng theo dõi biến động giá của một biến thể thiết bị và nhận thông báo khi giá đạt mức mong muốn.

Giai đoạn 1: Tạo cảnh báo

Người dùng đăng nhập, lựa chọn một biến thể thiết bị và thiết lập:

- Mức giá mục tiêu.
- Loại tiền tệ.
- Trạng thái kích hoạt cảnh báo.

Hệ thống kiểm tra dữ liệu và lưu cảnh báo vào cơ sở dữ liệu.

Giai đoạn 2: Cập nhật dữ liệu giá

Khi hệ thống đồng bộ được dữ liệu giá mới, giá hiện hành của thiết bị được cập nhật.

Nếu giá thay đổi, hệ thống đồng thời lưu lại lịch sử để phục vụ việc theo dõi xu hướng.

Giai đoạn 3: Kiểm tra điều kiện

Tiến trình nền định kỳ lấy danh sách các cảnh báo đang hoạt động và so sánh mức giá hiện tại với điều kiện do người dùng đặt.

Có hai trường hợp:

Giá	chưa	đạt	điều	kiện:
Không tạo thông báo và tiếp tục theo dõi trong chu kỳ sau.				

Giá	đạt	điều	kiện:
Hệ thống kiểm tra cảnh báo đã được gửi trước đó hay chưa để tránh tạo thông báo trùng.			

Giai đoạn 4: Tạo thông báo

Nếu điều kiện đạt và chưa có thông báo tương ứng:

- Thông báo trong ứng dụng được tạo và lưu ngay.
- Yêu cầu gửi thư được đưa vào hàng đợi nếu chức năng thư điện tử được bật.

Giai đoạn 5: Gửi thông báo

Tiến trình gửi thư xử lý các bản ghi theo lô.

Đối với lỗi tạm thời, hệ thống có thể thử lại. Nếu xác định lỗi vĩnh viễn, quá trình gửi tương ứng được dừng và trạng thái được ghi nhận.

Luồng nghiệp vụ:

Người dùng tạo cảnh báo → Hệ thống lưu điều kiện → Giá được cập nhật → Worker kiểm tra → So sánh ngưỡng → Tạo thông báo → Gửi đến người dùng

1.4. Thiết kế cơ sở dữ liệu

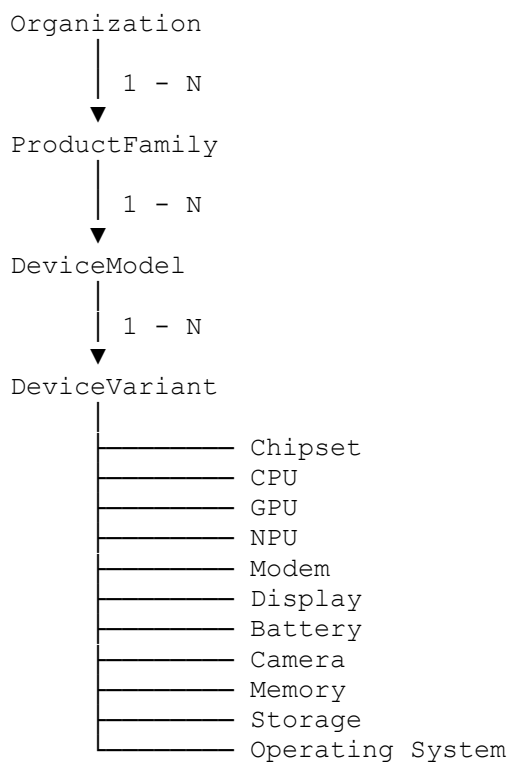
3.4.1. Sơ đồ thực thể mối quan hệ (ERD)

Sơ đồ thực thể mối quan hệ mô tả các thực thể quan trọng và mối liên kết giữa chúng trong cơ sở dữ liệu SpecHub.

Do phạm vi dữ liệu của hệ thống tương đối lớn, ERD tổng thể bao gồm nhiều nhóm thực thể. Trong đó, nhóm danh mục thiết bị và phần cứng đóng vai trò trung tâm; những nhóm dữ liệu như benchmark, nguồn, Wiki, AI, người dùng và thương mại được liên kết xung quanh các thực thể này.

Hình 3.21. Sơ đồ thực thể mối quan hệ của hệ thống SpecHub

Quan hệ cốt lõi của danh mục thiết bị có thể biểu diễn:



Trong mô hình hiện tại, một tổ chức có thể sở hữu nhiều dòng sản phẩm, một dòng sản phẩm chứa nhiều mẫu thiết bị và mỗi mẫu thiết bị có thể có nhiều biến thể. Biến thể tiếp tục liên kết với các nhóm thông số vật lý, I/O, tản nhiệt, phần cứng và phần mềm.

a) Nhóm tổ chức và danh mục thiết bị

Đây là nhóm thực thể trung tâm của cơ sở dữ liệu, bao gồm:

- organizations
- device_categories
- regions
- product_families
- device_models
- device_variants
- device_model_aliases
- device_editorial_sections
- variant_price_history
- variant_physical_specs
- variant_io_specs
- variant_thermal_specs
- model_lineage
- model_similarity

Bảng organizations

Lưu thông tin về tổ chức hoặc thương hiệu có liên quan đến sản phẩm.

Một tổ chức có thể sở hữu nhiều dòng sản phẩm:

Bảng device_categories

Dùng để phân loại thiết bị thành những nhóm khác nhau. Việc tách loại thiết bị khỏi mẫu sản phẩm giúp SpecHub có khả năng mở rộng sang nhiều nhóm thiết bị thông minh.

Một loại thiết bị có thể chứa nhiều dòng sản phẩm:

Bảng product_families

Đại diện cho một dòng sản phẩm và đóng vai trò trung gian giữa tổ chức, loại thiết bị và các mẫu thiết bị.

Bảng device_models

Đại diện cho thiết bị ở cấp tên thương mại chính. Đây là thực thể được sử dụng để lưu các thông tin chung của một mẫu sản phẩm.

Một mẫu thiết bị có thể:

- Có nhiều biến thể.
- Có nhiều tên thay thế.
- Có nhiều nội dung biên tập.

Quan hệ chính:

Bảng `device_variants`

Đại diện cho cấu hình cụ thể của một mẫu thiết bị. Đây là cấp dữ liệu quan trọng đối với các chức năng so sánh, benchmark, chấm điểm, giá và cấu hình phần cứng.

Việc tách `device_models` và `device_variants` giúp hệ thống tránh coi mỗi cấu hình RAM, lưu trữ hoặc khu vực phát hành là một mẫu sản phẩm hoàn toàn độc lập.

b) Nhóm thành phần phần cứng và phần mềm

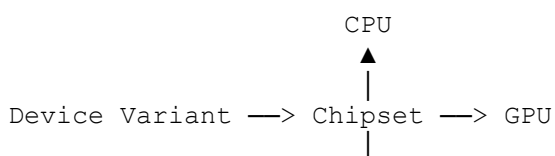
Các thành phần phần cứng không được lưu trực tiếp dưới dạng chuỗi trong từng thiết bị mà được xây dựng thành các thực thể độc lập.

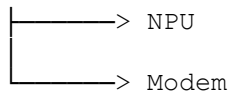
Nhóm này bao gồm:

- `chipsets`
- `cpus`
- `cpu_clusters`
- `gpus`
- `npus`
- `modems`
- `camera_sensors`
- `camera_modules`
- `display_units`
- `battery_units`
- `memory_standards`
- `storage_standards`
- `operating_systems`
- `os_versions`
- `os_ui_layers`
- `os_ui_layer_versions`

Cách tổ chức này cho phép cùng một linh kiện được liên kết đến nhiều sản phẩm khác nhau.

Ví dụ:





Chipset được liên kết với CPU, GPU, NPU và modem thông qua các bảng trung gian thay vì sao chép thông số của những thành phần này vào từng thiết bị. Camera cũng được tách giữa cảm biến và mô-đun; hệ điều hành được tách thành nền tảng, phiên bản và lớp giao diện.

c) Nhóm bảng liên kết phần cứng

Để biểu diễn các quan hệ nhiều - nhiều, SpecHub sử dụng nhiều bảng trung gian như:

- `chipset_cpu_links`
- `chipset_gpu_links`
- `chipset_npu_links`
- `chipset_modem_links`
- `camera_module_sensor_links`

Ngoài ra còn có các bảng biểu diễn khả năng và tiêu chuẩn phần cứng:

- `cpu_capabilities`
- `gpu_apis`
- `gpu_api_support`
- `camera_features`
- `camera_video_modes`
- `hdr_standards`
- `color_gamuts`
- `charging_protocols`
- `connectivity_features`
- `cellular_bands`
- `wifi_bands`
- `certifications`

Ví dụ, quan hệ giữa chipset và GPU có thể biểu diễn:

thông qua:

Việc sử dụng bảng trung gian cho phép hệ thống biểu diễn linh hoạt những trường hợp một thành phần có thể xuất hiện trong nhiều cấu hình kỹ thuật khác nhau.

d) Nhóm cấu hình biến thể thiết bị

Một `DeviceVariant` có thể liên kết với nhiều thành phần và thông tin cấu hình. SpecHub sử dụng các bảng:

- `variant_chipsets`
- `variant_cpus`
- `variant_gpus`
- `variant_npus`
- `variant_modems`
- `variant_displays`
- `variant_batteries`
- `variant_camera_systems`
- `variant_camera_modules`
- `variant_memory_configs`
- `variant_storage_configs`
- `variant_wifi_bands`
- `variant_operating_systems`
- `variant_software_profiles`
- `variant_connectivity_support`
- `variant_cellular_band_support`
- `variant_certifications`
- `variant_region_availability`

Cấu trúc này giúp một biến thể có thể chứa nhiều thành phần mà vẫn giữ được khả năng tìm kiếm và lọc bằng cơ sở dữ liệu quan hệ.

Ví dụ:

```
DeviceVariant
├── variant_chipsets ———> Chipset
├── variant_displays ———> Display
├── variant_batteries ———> Battery
├── variant_camera_modules -> CameraModule
├── variant_memory_configs
└── variant_storage_configs
```

Thiết kế này cũng giúp xử lý những trường hợp một mẫu thiết bị có cấu hình khác nhau tùy theo khu vực hoặc phiên bản.

e) Nhóm từ điển và dữ liệu chuẩn hóa

SpecHub sử dụng một số bảng đóng vai trò từ điển dùng chung:

- `technology_families`
- `architectures`

- `process_nodes`
- `camera_roles`
- `display_technologies`
- `battery_chemistries`
- `network_generations`

Mục đích của các bảng này là hạn chế việc lặp lại cùng một giá trị dưới nhiều dạng văn bản khác nhau.

Ví dụ, thay vì lưu trực tiếp tên kiến trúc hoặc công nghệ màn hình trên mọi thiết bị, dữ liệu được liên kết đến một thực thể chuẩn.

Điều này mang lại các lợi ích:

- Giảm dữ liệu trùng lặp.
- Hạn chế sai khác trong cách viết.
- Hỗ trợ lọc dữ liệu.
- Hỗ trợ dịch thuật.
- Thuận lợi cho phân tích và thống kê.

f) Nhóm nguồn, trích dẫn, ngôn ngữ và media

Một trong những yêu cầu quan trọng của SpecHub là khả năng xác định nguồn gốc của thông tin.

Các bảng chính gồm:

- `languages`
- `translations`
- `release_statuses`
- `currencies`
- `sources`
- `citations`
- `media_assets`
- `entity_media`
- `module_field_coverage`
- `tags`
- `entity_tags`
- `units`

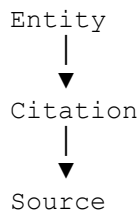
Bảng `sources`

Lưu thông tin nguồn được sử dụng để kiểm chứng dữ liệu.

Bảng citations

Biểu diễn mối liên hệ giữa nguồn và dữ liệu hoặc thực thể tương ứng.

Quan hệ tổng quát:



Nguồn và trích dẫn được lưu độc lập với nội dung, giúp cùng một nguồn có thể được sử dụng cho nhiều đối tượng khác nhau.

Bảng media_assets và entity_media

Media được quản lý thành tài nguyên riêng và liên kết tới nhiều loại thực thể thông qua bảng liên kết. Cách tổ chức này hạn chế việc nhúng trực tiếp dữ liệu tệp vào từng bảng thiết bị.

g) Nhóm benchmark và hệ thống điểm

SpecHub không chỉ lưu thông số mà còn hỗ trợ benchmark và hệ thống chấm điểm.

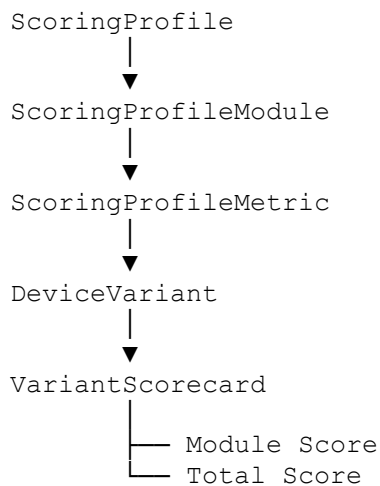
Các bảng chính gồm:

- benchmarks
- benchmark_runs
- device_variant_benchmarks
- chipset_benchmarks
- cpu_benchmarks
- gpu_benchmarks
- npu_benchmarks
- variant_module_scores
- variant_score_metric_inputs
- variant_scorecards
- variant_scorecard_modules
- scoring_profiles
- scoring_profile_modules
- scoring_profile_metrics

Mô hình tính điểm tổng quát:

Tổng trọng số của chỉ số trong mỗi mô-đun và tổng trọng số của các mô-đun trong một hồ sơ phải bằng 1 trong sai số cho phép. Kết quả được lưu cùng phiên bản hồ sơ và đầu vào để có thể kiểm tra lại cách điểm đã được tính.

Có thể biểu diễn quan hệ:



h) Nhóm Catalog Studio và quản lý phiên bản

Các bảng:

- catalog_drafts
- catalog_draft_versions
- catalog_entity_versions

được sử dụng để hỗ trợ quá trình biên tập dữ liệu.

Bảng catalog_drafts

Lưu trạng thái bản nháp, dữ liệu đang nhập và revision.

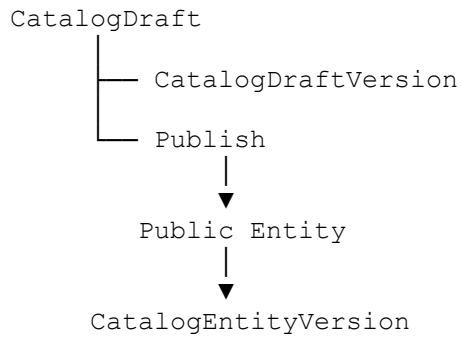
Bảng catalog_draft_versions

Lưu các phiên bản trước của bản nháp nhằm hỗ trợ lịch sử và khôi phục.

Bảng catalog_entity_versions

Theo dõi trạng thái của các thực thể sau những thay đổi quan trọng.

Cấu trúc:



Khi xuất bản, hệ thống sử dụng transaction để tránh tạo dữ liệu ở trạng thái không hoàn chỉnh.

ii) Nhóm Wiki, AI và dữ liệu thu thập

Nhóm này bao gồm:

- wiki_articles
- wiki_revisions
- wiki_article_citations
- comments
- embeddings
- ai_query_cache
- search_logs
- data_sources
- raw_pages

Wiki

Một bài Wiki và các phiên bản chỉnh sửa được tách thành các thực thể riêng.

Quan hệ có thể mô tả:

Nhờ đó, một lần chỉnh sửa không ghi đè trực tiếp lên nội dung trước đó.

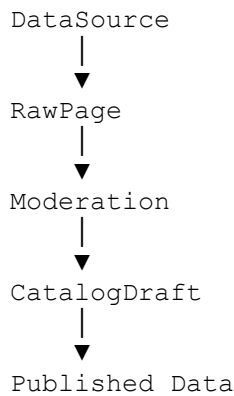
AI

`embeddings` lưu dữ liệu biểu diễn véc-tơ phục vụ tìm kiếm ngữ nghĩa và RAG. Việc tạo embedding chỉ được áp dụng đối với dữ liệu nằm trong phạm vi đã được phê duyệt.

Dữ liệu thu thập

`data_sources` và `raw_pages` phục vụ quá trình thu thập dữ liệu từ nguồn bên ngoài.

Dữ liệu thô được tách biệt khỏi catalog chính:



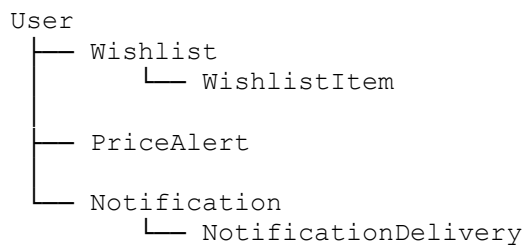
Điều này bảo đảm dữ liệu được thu thập không tự động trở thành thông tin công khai.

j) Nhóm người dùng và dữ liệu tương tác

Nhóm dữ liệu này phục vụ các chức năng tài khoản và cá nhân hóa, trong đó có:

- users
- wishlists
- wishlist_items
- price_alerts
- notifications
- notification_deliveries

Quan hệ tổng quát:



Một người dùng có thể tạo nhiều danh sách yêu thích và mỗi danh sách có thể chứa nhiều thiết bị. Cảnh báo giá được liên kết với nhu cầu theo dõi của từng tài khoản; thông báo logic và quá trình gửi thông báo được quản lý tách biệt.

k) Nhóm giá, thương mại và thuê bao

Các bảng liên quan gồm:

- affiliate_partners
- affiliate_links
- affiliate_price_history
- affiliate_clicks
- subscription_plans
- subscriptions
- billing_audit_logs
- billing_webhook_events

Thiết kế phân biệt:

- Liên kết mua hàng hiện tại và lịch sử giá.
- Gói dịch vụ và thuê bao.
- Sự kiện webhook gốc và nhật ký xử lý.

Việc tách dữ liệu giá hiện tại khỏi lịch sử giá giúp hệ thống có thể theo dõi biến động và phục vụ chức năng cảnh báo.

l) Nhóm API B2B

SpecHub sử dụng:

- api_keys
- api_key_usage

để quản lý việc truy cập API dành cho đối tác.

Thông tin khóa được tách thành:

- Định danh khóa.
- Giá trị băm của bí mật.
- Trạng thái.
- Mức sử dụng.

Khóa bí mật không cần được lưu dưới dạng văn bản thuần trong cơ sở dữ liệu.

3.4.2. Các mối quan hệ chính trong cơ sở dữ liệu

Dựa trên mô hình dữ liệu, các quan hệ quan trọng có thể tổng hợp như sau:

Quan hệ	Kiểu	Ý nghĩa
Organization –	1:N	Một tổ chức có nhiều dòng sản phẩm

Quan hệ	Kiểu	Ý nghĩa
ProductFamily		
DeviceCategory – ProductFamily	1:N	Một loại thiết bị có nhiều dòng sản phẩm
ProductFamily – DeviceModel	1:N	Một dòng sản phẩm có nhiều mẫu
DeviceModel – DeviceVariant	1:N	Một mẫu có nhiều biến thể
DeviceModel – Alias	1:N	Một mẫu có nhiều tên thay thế
DeviceVariant – Hardware	N:N	Một biến thể liên kết nhiều linh kiện và linh kiện có thể được tái sử dụng
Chipset – CPU/GPU/NPU/Modem	N:N	Biểu diễn cấu thành và quan hệ phần cứng
CameraModule – CameraSensor	N:N	Một mô-đun liên kết với cảm biến
DeviceVariant – Benchmark	N:N	Một biến thể có thể có nhiều kết quả benchmark
ScoringProfile – Module – Metric	1:N	Hồ sơ điểm chứa nhiều mô-đun và chỉ số
WikiArticle – WikiRevision	1:N	Một bài Wiki có lịch sử nhiều phiên bản
User – Wishlist	1:N	Một người dùng có nhiều danh sách
Wishlist – WishlistItem	1:N	Một danh sách chứa nhiều mục
DeviceVariant – PriceHistory	1:N	Một biến thể có nhiều bản ghi giá theo thời gian

Những quan hệ cốt lõi giữa tổ chức, danh mục, mẫu và biến thể được triển khai trực tiếp trong cấu trúc dữ liệu hiện tại của SpecHub.

3.4.3. Quy tắc toàn vẹn dữ liệu

Để bảo đảm dữ liệu của thư viện không bị sai lệch trong quá trình cập nhật, cơ sở dữ liệu và tầng nghiệp vụ áp dụng các nguyên tắc sau:

a) Khóa chính và khóa ngoại

Các thực thể cần có định danh riêng, trong khi những quan hệ giữa thiết bị, biến thể và linh kiện được ràng buộc thông qua các khóa ngoại phù hợp.

Điều này hạn chế việc xuất hiện các bản ghi liên kết tới thực thể không tồn tại.

b) Ràng buộc duy nhất

Các thuộc tính mang tính định danh cần được kiểm soát để hạn chế tạo dữ liệu trùng.

Đây là yêu cầu đặc biệt quan trọng với những thực thể như:

- Người dùng.
- Slug.
- Mẫu thiết bị.
- Biến thể.
- Các thực thể phân cứng dùng chung.

c) Kiểm soát dữ liệu trùng lặp

Trước khi tạo một thành phần mới trong Catalog Studio, hệ thống thực hiện tìm kiếm thực thể hiện có.

Các linh kiện dùng chung được liên kết tới nhiều thiết bị thay vì tạo bản sao cho từng thiết bị.

d) Transaction

Những nghiệp vụ thay đổi nhiều thực thể, đặc biệt là quá trình xuất bản Catalog Studio, được xử lý theo transaction.

Nếu một bước thất bại, toàn bộ thao tác có thể được hủy để tránh trạng thái dữ liệu chỉ được cập nhật một phần. Đặc tả SpecHub xác định quá trình xuất bản phải sử dụng giao dịch dữ liệu.

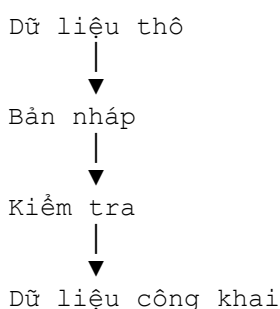
e) Kiểm soát xung đột bản nháp

Bản nháp sử dụng `revision` để phát hiện trường hợp một phiên chỉnh sửa cũ cố ghi đè lên dữ liệu đã được cập nhật bởi phiên mới hơn.

Cơ chế này hỗ trợ bảo vệ dữ liệu trong quy trình biên tập.

f) Tách trạng thái dữ liệu

SpecHub phân biệt rõ:



Dữ liệu thu thập từ bên ngoài không được ghi trực tiếp vào catalog và bản nháp chưa duyệt cũng không được xem là dữ liệu công khai.

g) Lưu phiên bản dữ liệu

Đối với các loại dữ liệu có yêu cầu truy nguyên, hệ thống lưu lịch sử phiên bản.

Ví dụ:

- Phiên bản Wiki.
- Phiên bản bản nháp.
- Phiên bản thực thể catalog.
- Phiên bản hồ sơ điểm.

Đặc biệt, kết quả điểm phải gắn với phiên bản hồ sơ tương ứng để có thể tái lập cách tính ở thời điểm trước đó.

CHƯƠNG 4: PHÁT TRIỂN PHẦN MỀM

4.1. Xây dựng ứng dụng

Quá trình phát triển được thực hiện theo kiến trúc Monorepo kết hợp Modular Monolith. Giao diện Web được xây dựng bằng Next.js và React, Backend sử dụng NestJS kết hợp Fastify, PostgreSQL được sử dụng làm cơ sở dữ liệu chính và Prisma đảm nhiệm việc truy cập dữ liệu.

4.1.1. Hiện thực hóa kiến trúc hệ thống và quy trình phát triển

Dựa trên kiến trúc đã thiết kế ở Chương 2, SpecHub được triển khai thành các thành phần chính:

- **Frontend:** Sử dụng Next.js và React để xây dựng giao diện người dùng. Frontend đảm nhiệm hiển thị danh mục thiết bị, thông tin chi tiết, tìm kiếm, so sánh, Wiki, trợ lý AI và các giao diện quản trị.
- **Backend:** Sử dụng NestJS kết hợp Fastify để xây dựng RESTful API và xử lý logic nghiệp vụ. Backend được tổ chức thành các module theo từng miền chức năng như xác thực, thiết bị, phần cứng, tìm kiếm, Wiki, AI, cảnh báo và quản trị.
- **Worker:** Thực hiện các tác vụ nền như kiểm tra cảnh báo giá, thu thập dữ liệu nguồn và gửi thông báo.
- **Packages dùng chung:** Chứa các thành phần về cơ sở dữ liệu, kiểu dữ liệu, xác thực, chấm điểm, AI và giao diện dùng chung.

Các thành phần trên được quản lý trong cùng một Monorepo, giúp thống nhất cấu hình và tái sử dụng mã nguồn giữa Web, API và các chức năng hỗ trợ. Cấu trúc hiện tại của dự án bao gồm `web`, `api`, `worker`, `ai-service` cùng các package chuyên biệt như `database`, `scoring-core`, `ai-core`, `auth`, `types` và `ui`.

4.1.2. Hiện thực hóa hệ thống thư viện và chuẩn hóa dữ liệu thiết bị

Chức năng cốt lõi của SpecHub là xây dựng thư viện dữ liệu thiết bị có cấu trúc. Thay vì lưu toàn bộ thông số dưới dạng văn bản trực tiếp trên từng sản phẩm, hệ thống phân tách dữ liệu thành các thực thể độc lập và thiết lập quan hệ giữa chúng.

Dữ liệu thiết bị được tổ chức theo cấu trúc:

Loại thiết bị → Dòng sản phẩm → Mẫu thiết bị → Biến thể thiết bị

Trong đó, mỗi biến thể có thể được liên kết với các thành phần phần cứng và phần mềm như:

- Chipset, CPU, GPU, NPU và modem.
- Màn hình.
- Camera và cảm biến camera.
- Pin và chuẩn sạc.
- Bộ nhớ và lưu trữ.
- Hệ điều hành và phiên bản phần mềm.
- Chuẩn kết nối và băng tần.

Các thành phần phần cứng được lưu thành các thực thể dùng chung thay vì sao chép dữ liệu vào từng thiết bị. Ví dụ, chipset được liên kết với CPU, GPU, NPU và modem thông qua các bảng liên kết; camera được tách thành cảm biến và mô-đun.

Cách tổ chức này giúp hạn chế trùng lặp dữ liệu và tạo điều kiện cho các chức năng tra cứu, lọc và so sánh. Khi một thông tin linh kiện được cập nhật, các thiết bị đang liên kết với linh kiện đó có thể sử dụng lại dữ liệu đã được chuẩn hóa.

Đối với quá trình nhập và chỉnh sửa dữ liệu, hệ thống sử dụng **Catalog Studio**. Người biên tập tạo bản nháp, bổ sung thông tin, kiểm tra dữ liệu và chỉ xuất bản khi các điều kiện cần thiết được đáp ứng. Bản nháp và lịch sử thay đổi được quản lý riêng, còn quá trình xuất bản sử dụng transaction để hạn chế tình trạng dữ liệu được cập nhật không đầy đủ.

4.1.3. Triển khai cơ chế bảo mật và xác thực

Hệ thống triển khai cơ chế xác thực dựa trên **JWT kết hợp Redis**.

Sau khi người dùng đăng nhập thành công, Backend cấp access token có thời hạn ngắn và tạo phiên tương ứng trong Redis. Refresh token được liên kết với phiên này để phục vụ quá trình cấp lại access token. Khi người dùng đăng xuất, phiên được thu hồi tại phía máy chủ.

Các yêu cầu gửi tới Backend được kiểm soát theo chuỗi:

- **ThrottlerGuard:** giới hạn tần suất gửi yêu cầu.
- **JwtAuthGuard:** kiểm tra trạng thái xác thực.
- **RolesGuard:** kiểm tra vai trò người dùng.

Những endpoint công khai được khai báo riêng, trong khi các chức năng quản trị yêu cầu người dùng có quyền phù hợp. Ngoài kiểm tra vai trò, Backend tiếp tục kiểm tra quyền sở hữu tài nguyên tại tầng Service đối với các dữ liệu cá nhân.

Đối với API dành cho đối tác B2B, hệ thống sử dụng API Key riêng và kiểm tra phạm vi truy cập, giới hạn tần suất cũng như hạn mức sử dụng trước khi cung cấp dữ liệu.

4.1.4. Giải pháp lưu trữ và quản lý dữ liệu

SpecHub triển khai dữ liệu theo nhiều hình thức phù hợp với đặc điểm của từng loại thông tin:

- **PostgreSQL:** Lưu trữ dữ liệu quan hệ chính của hệ thống như thiết bị, biến thể, linh kiện, benchmark, nguồn, Wiki, người dùng và dữ liệu quản trị.
- **Prisma ORM:** Quản lý mô hình dữ liệu và cung cấp lớp truy cập giữa Backend với PostgreSQL.
- **Redis:** Lưu phiên đăng nhập và các dữ liệu tạm thời.
- **pgvector:** Lưu dữ liệu véc-tơ phục vụ tìm kiếm ngữ nghĩa và chức năng AI.
- **Meilisearch:** Hỗ trợ tăng cường khả năng tìm kiếm danh mục thiết bị.
- **Kho lưu trữ tương thích S3:** Được sử dụng cho dữ liệu media; trong môi trường phát triển hệ thống có thể sử dụng phương án lưu trữ cục bộ.

4.2. Các kỹ thuật được áp dụng trong xử lý và tối ưu hệ thống

4.2.1. Thuật toán chuẩn hóa và tính điểm thiết bị

Một trong những chức năng đặc trưng của SpecHub là hệ thống đánh giá thiết bị dựa trên nhiều nhóm thông số kỹ thuật. Việc tính điểm không thể sử dụng trực tiếp các giá trị ban đầu do mỗi thông số có đơn vị, khoảng giá trị và ý nghĩa khác nhau. Ví dụ, điểm

benchmark có thể có giá trị hàng nghìn hoặc hàng triệu, trong khi dung lượng pin thường được biểu diễn bằng mAh và các thông số khác lại sử dụng Hz, nit hoặc Mbps.

Để xử lý vấn đề này, hệ thống sử dụng cơ chế **chuẩn hóa dữ liệu kết hợp trọng số**, được triển khai trong package `scoring-core`. Cấu trúc hệ thống điểm gồm ba thành phần chính:

- Hồ sơ điểm (Scoring Profile).
- Mô-đun đánh giá (Scoring Module).
- Chỉ số đánh giá (Scoring Metric).

Mỗi hồ sơ điểm xác định cách một nhóm thiết bị được đánh giá. Trong hồ sơ có nhiều mô-đun, mỗi mô-đun tiếp tục chứa các chỉ số thành phần và trọng số tương ứng. Cơ sở dữ liệu của SpecHub lưu riêng hồ sơ, mô-đun, chỉ số, đầu vào đo được và kết quả để quá trình tính điểm có thể được kiểm tra lại.

a) Chuẩn hóa giá trị đầu vào

Giá trị thực tế của từng chỉ số được chuyển về một thang điểm thống nhất trước khi đưa vào quá trình tổng hợp.

Công thức tổng quát:

Trong đó:

- : giá trị thực tế của chỉ số thứ .
- : quy tắc chuẩn hóa.
- : chiều đánh giá của chỉ số.
- : giới hạn được cấu hình.
- : điểm sau chuẩn hóa.

Chiều đánh giá cho phép hệ thống xử lý các loại thông số khác nhau. Với một số chỉ số, giá trị lớn hơn thể hiện kết quả tốt hơn; trong khi một số chỉ số khác có thể được đánh giá theo chiều ngược lại.

Việc chuẩn hóa giúp các giá trị khác đơn vị có thể được đưa về cùng một miền điểm trước khi tính trọng số.

b) Tính điểm mô-đun

Sau khi các chỉ số được chuẩn hóa, điểm của một mô-đun được tính từ tổng các điểm chỉ số nhân với trọng số tương ứng:

Trong đó:

- : điểm chuẩn hóa của chỉ số.
- : trọng số của chỉ số.
- : số chỉ số thuộc mô-đun.

Tổng trọng số của các chỉ số trong một mô-đun phải bằng 1 trong phạm vi sai số được hệ thống cho phép.

Ví dụ, mô-đun hiệu năng có thể tổng hợp nhiều kết quả đo khác nhau. Thay vì sử dụng duy nhất một benchmark, từng chỉ số được chuẩn hóa và đóng góp vào điểm mô-đun theo trọng số được cấu hình.

c) Tính điểm tổng của thiết bị

Sau khi hoàn thành điểm của từng mô-đun, hệ thống tiếp tục tổng hợp thành điểm cuối:

Trong đó:

- : điểm của mô-đun thứ .
- : trọng số của mô-đun.
- : tổng số mô-đun trong hồ sơ điểm.

Tương tự cấp chỉ số, tổng trọng số của các mô-đun trong một hồ sơ phải bằng 1.

d) Phiên bản hóa hồ sơ điểm

Một vấn đề quan trọng của hệ thống điểm là công thức đánh giá có thể thay đổi theo thời gian. Nếu chỉ lưu điểm cuối cùng, khi trọng số hoặc quy tắc chuẩn hóa được chỉnh sửa sẽ không thể xác định một điểm cũ đã được tính bằng phương pháp nào.

Vì vậy, SpecHub lưu:

- Phiên bản hồ sơ điểm.

- Điểm từng mô-đun.
- Điểm tổng.
- Các giá trị đầu vào đã sử dụng.

Hồ sơ đã được xuất bản được giữ ổn định; khi phương pháp đánh giá thay đổi, hệ thống có thể sử dụng một phiên bản mới thay vì làm thay đổi ý nghĩa của kết quả đã tồn tại. Các yêu cầu của Catalog Studio cũng quy định phiên bản hồ sơ đã xuất bản là bất biến và kết quả bảng điểm phải lưu phiên bản hồ sơ cùng dữ liệu đầu vào.

Quy trình tính điểm có thể mô tả:

Dữ liệu thiết bị → Lựa chọn hồ sơ điểm → Chuẩn hóa chỉ số → Tính điểm mô-đun → Tính điểm tổng → Lưu bảng điểm và phiên bản

Cách tổ chức này giúp kết quả đánh giá không chỉ là một con số cuối cùng mà còn có khả năng giải thích và truy nguyên phương pháp tính.

4.2.2. Cơ chế kiểm soát xung đột và xuất bản dữ liệu trong Catalog Studio

SpecHub cho phép biên tập viên và quản trị viên chỉnh sửa lượng lớn thông tin của một thiết bị, bao gồm mẫu sản phẩm, biến thể, chipset, màn hình, camera, pin, phần mềm, media và nguồn tham chiếu. Quá trình này có thể kéo dài và một bản nháp có khả năng được cập nhật nhiều lần trước khi xuất bản.

Nếu nhiều thao tác chỉnh sửa được thực hiện trên cùng một bản nháp mà không có cơ chế kiểm soát, một thay đổi cũ có thể ghi đè dữ liệu mới hơn. Vì vậy, Catalog Studio sử dụng cơ chế **revision** để phát hiện xung đột khi cập nhật.

a) Cơ chế Revision

Mỗi bản nháp lưu:

- Dữ liệu hiện tại.
- Bước nhập liệu hiện tại.
- Số revision.

Khi Client cập nhật bản nháp, yêu cầu phải gửi kèm revision tương ứng với phiên bản mà người dùng đang chỉnh sửa.

Quy trình:

Mở bản nháp → Nhận revision hiện tại → Chỉnh sửa → Gửi dữ liệu + revision → Backend kiểm tra revision

Có hai trường hợp:

Revision **hợp** **lệ:**

Dữ liệu được cập nhật và revision được thay đổi sang phiên bản mới.

Revision không hợp lệ:

Backend từ chối thao tác để tránh ghi đè lên thay đổi mới hơn.

Có thể biểu diễn:

```
Client A đọc Revision = 5
Client B đọc Revision = 5
```

Client A cập nhật

Revision mới = 6

Client B gửi Revision = 5

Phát hiện xung đột

Từ chối cập nhật

SpecHub sử dụng mã trạng thái 409 `Conflict` cho các trường hợp xung đột về duy nhất, revision hoặc trạng thái nghiệp vụ.

Cơ chế này tương tự phương pháp **Optimistic Concurrency Control**, phù hợp với quá trình biên tập vì không cần khóa bản ghi trong toàn bộ thời gian người dùng đang chỉnh sửa.

b) Quản lý lịch sử bản nháp

Ngoài revision hiện tại, hệ thống còn lưu lịch sử thông qua:

- catalog_drafts.
- catalog_draft_versions.
- catalog_entity_versions.

`catalog_drafts` lưu trạng thái hiện hành của bản nháp, trong khi `catalog_draft_versions` giữ các phiên bản trước nhằm phục vụ xem lịch sử và khôi phục. `catalog_entity_versions` ghi nhận trạng thái thực thể sau các thay đổi quan trọng.

Cơ chế này giúp hạn chế việc mất dữ liệu khi biên tập viên cần quay lại phiên bản trước.

c) Kiểm tra trước khi xuất bản

Dữ liệu trong Catalog Studio không được đưa trực tiếp vào danh mục công khai sau khi nhập.

Trước khi xuất bản, Backend thực hiện kiểm tra:

- Các trường bắt buộc.
- Quan hệ giữa các thực thể.
- Nguồn và trích dẫn.
- Media.
- Độ phủ dữ liệu.
- Dữ liệu phục vụ hệ thống điểm.

Nếu tồn tại lỗi, hệ thống trả danh sách vấn đề để người biên tập chỉnh sửa trước khi tiếp tục.

Quá trình có thể biểu diễn:

Bản nháp → Validate → Có lỗi?

- Có → trả danh sách lỗi → tiếp tục chỉnh sửa.
- Không → cho phép xác nhận xuất bản.

d) Transaction khi xuất bản

Một thiết bị trong SpecHub có thể liên quan đến nhiều bảng khác nhau. Trong một lần xuất bản, hệ thống có thể phải tạo hoặc cập nhật mẫu thiết bị, biến thể và hàng loạt quan hệ phân cứng.

Nếu một thao tác thành công nhưng thao tác tiếp theo thất bại, cơ sở dữ liệu có thể rơi vào trạng thái không hoàn chỉnh.

Do đó, quá trình xuất bản được thực hiện trong **transaction**:

BEGIN TRANSACTION

Tạo/Cập nhật Device Model
Tạo/Cập nhật Device Variant
Liên kết Chipset
Liên kết Display
Liên kết Camera
Liên kết Battery
Cập nhật Source
Ghi lịch sử

Nếu tất cả thành công

↓
COMMIT

Nếu một bước thất bại

↓
ROLLBACK

Đặc tả của hệ thống yêu cầu xuất bản phải tạo hoặc cập nhật các thực thể liên quan trong cùng một giao dịch dữ liệu nhằm tránh trạng thái dữ liệu được tạo một phần.

e) Cập nhật dữ liệu sau khi xuất bản

Sau khi transaction thành công:

- Bản ghi công khai được cập nhật.
- Lịch sử thực thể được tạo.
- Nếu thực thể thuộc phạm vi tìm kiếm, chỉ mục tìm kiếm được cập nhật.
- Nếu thực thể thuộc kho tri thức AI, dữ liệu có thể được lập chỉ mục lại.

Quy trình hoàn chỉnh của Catalog Studio:

Tìm thực thể → Tạo/Mở bản nháp → Nhập dữ liệu → Lưu kèm revision → Kiểm tra → Xác nhận → Transaction xuất bản → Ghi lịch sử → Cập nhật chỉ mục.

Đây là cơ chế quan trọng để duy trì tính toàn vẹn của thư viện khi số lượng bảng và quan hệ dữ liệu lớn.

4.2.3. Tối ưu hóa truy vấn, tìm kiếm và so sánh dữ liệu

Một thiết bị trong SpecHub không chỉ chứa thông tin tại một bảng mà có thể liên kết với nhiều thực thể phân cứng khác nhau. Vì vậy, nếu mỗi thành phần được truy vấn độc lập, số lượng truy vấn sẽ tăng nhanh khi hiển thị hoặc so sánh nhiều thiết bị.

Hệ thống áp dụng các kỹ thuật xử lý sau:

a) Phân trang danh sách dữ liệu

Đối với những danh sách có số lượng bản ghi lớn, hệ thống sử dụng phân trang thay vì trả toàn bộ dữ liệu trong một lần yêu cầu.

Các nhóm dữ liệu như:

- Mẫu thiết bị.
- Biên thể.
- Thành phần phần cứng.
- Kết quả tìm kiếm.

được giới hạn theo từng trang.

Việc này giúp kiểm soát kích thước dữ liệu được truyền giữa Backend và Frontend, đồng thời giới hạn số bản ghi cần xử lý trong từng truy vấn.

b) Sử dụng chỉ mục cho các trường tìm kiếm và lọc

Những trường thường được sử dụng trong điều kiện tìm kiếm, lọc hoặc xác định duy nhất được tổ chức với các chỉ mục và ràng buộc phù hợp.

Yêu cầu hiệu năng của SpecHub xác định các danh sách lớn phải được phân trang và sử dụng chỉ mục đối với những trường thường xuyên tìm kiếm hoặc lọc.

c) Tải dữ liệu theo lô khi so sánh

Chức năng so sánh là trường hợp dễ phát sinh nhiều truy vấn do người dùng có thể lựa chọn đồng thời nhiều biến thể.

Thay vì thực hiện:

Variant A → truy vấn quan hệ
Variant B → truy vấn quan hệ
Variant C → truy vấn quan hệ

Backend thực hiện theo hướng:

[A, B, C]
↓
Truy vấn theo tập định danh
↓
Tải các quan hệ phần cứng cần thiết
↓
Nhóm dữ liệu theo Variant
↓
Trả kết quả so sánh

Đây là kỹ thuật được sử dụng để hạn chế dạng truy vấn lặp theo từng phần tử. Yêu cầu hiệu năng của hệ thống xác định truy vấn so sánh phải tải dữ liệu theo lô.

d) Kết hợp PostgreSQL và Meilisearch

SpecHub sử dụng PostgreSQL làm nguồn dữ liệu chính. Đối với chức năng tìm kiếm, dịch vụ có thể lựa chọn PostgreSQL hoặc Meilisearch tùy theo cấu hình.

Quy trình:

- Người dùng nhập từ khóa và bộ lọc.
- Backend chuẩn hóa truy vấn.
- Chọn nguồn tìm kiếm.
- Nhận danh sách kết quả.
- Trả định danh, loại thực thể, slug, tóm tắt và thông tin phân trang.

Meilisearch đóng vai trò tăng cường tìm kiếm danh mục, không thay thế PostgreSQL làm nguồn dữ liệu chính.

e) Chuẩn hóa dữ liệu trước khi so sánh

Sau khi các biến thể được truy xuất, Web xây dựng dữ liệu theo cùng một hệ thuộc tính.

Đối với mỗi thuộc tính, hệ thống cần phân biệt:

- Có giá trị.
- Chưa có dữ liệu.
- Giá trị thực sự bằng 0.

Ví dụ:

Thiết bị A: Không có dữ liệu
Thiết bị B: 0

hai trường hợp trên không được coi là tương đương.

Việc giữ trạng thái thiếu dữ liệu là **không xác định** giúp tránh sai lệch trong bảng so sánh và các phép đánh giá tiếp theo. Quy trình tìm kiếm và so sánh hiện hành của SpecHub cũng xác định API tải dữ liệu theo lô, còn Web chịu trách nhiệm xây dựng ma trận thuộc tính và giữ trường thiếu ở trạng thái không xác định.

4.2.4. Kỹ thuật Retrieval-Augmented Generation và kiểm soát độ tin cậy của AI

SpecHub sử dụng trí tuệ nhân tạo để hỗ trợ người dùng hỏi đáp, nghiên cứu phân cứng và đưa ra khuyến nghị. Tuy nhiên, dữ liệu kỹ thuật của thiết bị có nhiều giá trị định lượng nên việc chỉ gửi câu hỏi trực tiếp cho mô hình ngôn ngữ có thể tạo ra thông tin không có căn cứ.

Để giải quyết vấn đề này, hệ thống triển khai phương pháp **Retrieval-Augmented Generation (RAG)**, trong đó quá trình truy xuất dữ liệu được thực hiện trước quá trình sinh câu trả lời.

a) Phạm vi kho tri thức

Không phải toàn bộ dữ liệu trong SpecHub đều được phép đưa vào AI.

Các dữ liệu được phép lập chỉ mục gồm:

- Mẫu và biến thể thiết bị đã xuất bản.
- Thành phần phần cứng và các quan hệ.
- Benchmark và điểm đánh giá.
- Nội dung Wiki đã được xuất bản.
- Trang dữ liệu thô đã được duyệt.
- Nguồn và trích dẫn công khai.

Ngược lại, các dữ liệu sau bị loại khỏi ngữ cảnh AI công khai:

- Thông tin tài khoản.
- Mật khẩu và phiên.
- API Key.
- Dữ liệu thanh toán riêng tư.
- Nhật ký nội bộ chứa dữ liệu nhạy cảm.
- Bản nháp chưa duyệt.
- Dữ liệu tương tác cá nhân.

Việc giới hạn này tạo ra ranh giới rõ giữa **kho dữ liệu vận hành** và **kho tri thức AI**.

b) Embedding và tìm kiếm ngữ nghĩa

Đối với dữ liệu phù hợp, hệ thống có thể tạo embedding và lưu biểu diễn véc-tơ trong PostgreSQL với sự hỗ trợ của `pgvector`.

Khi người dùng gửi câu hỏi:

Embedding của truy vấn được sử dụng để tìm các nội dung có mức tương đồng phù hợp trong kho tri thức.

Quy trình:

Câu hỏi → Embedding → Vector Search → Top-k dữ liệu liên quan

Kết quả truy xuất không phải câu trả lời cuối cùng mà đóng vai trò làm ngữ cảnh cho mô hình.

c) Xây dựng ngữ cảnh

Backend tổng hợp:

- Dữ liệu danh mục.
- Thông số phần cứng.
- Benchmark.
- Điểm đánh giá.
- Wiki.
- Nguồn và trích dẫn.

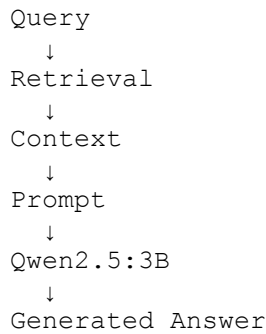
thành phần ngữ cảnh phù hợp với câu hỏi.

Quy trình hỏi đáp hiện tại của hệ thống gồm xác định ý định, truy xuất thực thể danh mục và nguồn liên quan, xây dựng thông tin về tiêu chí và đánh đổi, sau đó mới sinh câu trả lời.

d) Sinh câu trả lời

Trong triển khai của đồ án, Ollama được sử dụng để vận hành mô hình **Qwen2.5:3B** tại môi trường cục bộ.

Luồng xử lý:



Backend API là thành phần điều phối quá trình sinh câu trả lời. `apps/ai-service` hiện đảm nhiệm phần liên quan đến chuẩn đoán embedding cục bộ thay vì trực tiếp trở thành nơi xử lý toàn bộ câu trả lời AI.

e) Kiểm tra căn cứ – Grounding

Nội dung do mô hình sinh ra không được coi ngay là kết quả hợp lệ.

SpecHub áp dụng các nguyên tắc:

- Câu trả lời phải dựa trên ngữ cảnh đã truy xuất.
- Citation phải tham chiếu tới thực thể hoặc nguồn thực sự tồn tại.
- Nội dung định lượng không có dữ liệu hỗ trợ phải bị loại bỏ hoặc được yêu cầu sửa lại.
- Khuyến nghị phải trình bày điều kiện phù hợp và các điểm đánh đổi.
- Mô hình không được phép tự truy cập SQL tùy ý.

Ví dụ, nếu mô hình sinh:

Thiết bị A có pin 6.000 mAh.

nhưng dữ liệu truy xuất không chứa giá trị 6.000 mAh, hệ thống không nên xem thông tin trên là dữ kiện hợp lệ.

Có thể biểu diễn nguyên tắc:

Mỗi khẳng định quan trọng cần có dữ liệu tương ứng trong ngữ cảnh.

f) Quy trình hoàn chỉnh

Quy trình RAG của SpecHub được tổ chức:

Người dùng nhập câu hỏi



Phân tích ý định



Truy xuất dữ liệu



Tìm kiếm ngữ nghĩa



Xây dựng ngữ cảnh



Sinh câu trả lời



Kiểm tra Grounding



Liên kết Citation



Trả kết quả

Cơ chế này giúp AI trở thành lớp khai thác dữ liệu của SpecHub thay vì một nguồn dữ liệu độc lập.

4.2.5. Cơ chế xử lý tác vụ nền và dữ liệu bất đồng bộ

Một số nghiệp vụ của SpecHub không cần hoàn thành ngay trong thời gian người dùng chờ một HTTP Request. Những công việc như kiểm tra giá, thu thập nguồn hoặc gửi thư có thời gian xử lý phụ thuộc vào dịch vụ bên ngoài và thường cần chạy theo chu kỳ.

Nếu các công việc này được thực hiện trực tiếp trong luồng Request, thời gian phản hồi của Backend có thể phụ thuộc vào quá trình xử lý bên ngoài.

Do đó, SpecHub tách các công việc định kỳ sang `apps/worker`. Đây là một ứng dụng độc lập trong Monorepo.

a) Kiểm tra cảnh báo giá

Worker thực hiện kiểm tra cảnh báo giá theo chu kỳ mặc định **60 phút**.

Quy trình:

Lấy Price Alert đang hoạt động

↓

Lấy dữ liệu giá mới

↓

So sánh với giá mục tiêu

↓

Nếu đạt điều kiện → tạo Notification

↓

Đưa yêu cầu gửi thư vào luồng tương ứng

Việc này cho phép người dùng đóng trình duyệt sau khi tạo cảnh báo mà hệ thống vẫn tiếp tục theo dõi điều kiện.

b) Thu thập nguồn dữ liệu

Worker thu thập dữ liệu nguồn theo chu kỳ mặc định **360 phút**.

Nội dung thu thập không được ghi trực tiếp vào catalog.

Quy trình:

Nguồn đã cấu hình → Thu thập → Kiểm tra → Raw Page → Chờ duyệt → Biên tập viên xử lý

Sau khi được chấp nhận, dữ liệu mới có thể đi tiếp vào quy trình Catalog Studio.

Cách xử lý này bảo đảm dữ liệu thu thập tự động không trở thành dữ liệu công khai nếu chưa được kiểm tra.

c) Xử lý thông báo thư điện tử

Quá trình gửi thư được thực hiện theo chu kỳ riêng, mặc định **5 phút** khi chức năng gửi thư được bật.

Việc tách quá trình tạo thông báo và gửi thư giúp nghiệp vụ chính không phụ thuộc vào thời gian phản hồi của dịch vụ email.

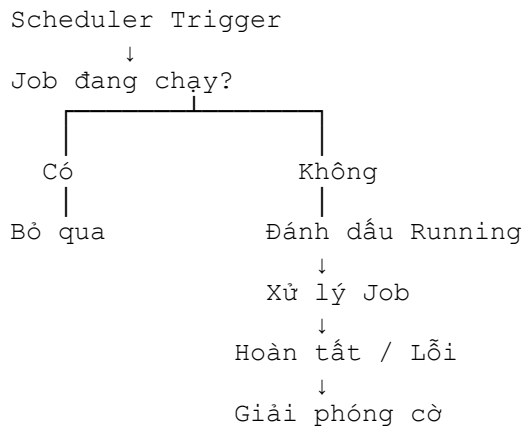
Trong trường hợp dịch vụ gửi thư không được bật, hệ thống vẫn có thể lưu thông báo trong ứng dụng theo yêu cầu khả dụng của SpecHub.

d) Cơ chế chống chạy chồng

Một công việc định kỳ có thể chưa kết thúc khi chu kỳ kế tiếp bắt đầu. Nếu hai phiên cùng xử lý một nhóm dữ liệu, hệ thống có nguy cơ tạo thông báo hoặc cập nhật trạng thái nhiều lần.

Worker sử dụng cờ nội bộ để tránh cùng một công việc chạy chồng trong một tiến trình.

Quá trình có thể biểu diễn:



Bên cạnh đó, các webhook, thao tác gửi thư và tác vụ nền được yêu cầu có tính lũy đẳng hoặc có cơ chế chống xử lý lặp để hạn chế việc một sự kiện được thực hiện nhiều lần.

e) Xử lý theo lô

Đối với những công việc có nhiều bản ghi, hệ thống ưu tiên xử lý theo lô thay vì tải toàn bộ dữ liệu vào một lần.

Ví dụ, quá trình đồng bộ giá lựa chọn các liên kết đã quá hạn, giới hạn kích thước lô và số lượng yêu cầu đồng thời trước khi cập nhật lịch sử giá và đánh giá cảnh báo.

Cách triển khai này giúp kiểm soát lượng công việc tại từng chu kỳ và hạn chế một tác vụ nền sử dụng quá nhiều tài nguyên trong cùng thời điểm.

Quy trình tổng quát của nhóm tác vụ nền:

Scheduler → Chọn tác vụ → Kiểm tra trạng thái chạy → Lấy dữ liệu theo lô → Xử lý → Lưu kết quả → Hoàn tất → Chờ chu kỳ tiếp theo

Việc tách các nghiệp vụ định kỳ khỏi API đồng thời phù hợp với yêu cầu hiệu năng của SpecHub: các tác vụ dài hoặc định kỳ phải được xử lý ngoài luồng yêu cầu của người dùng.

4.3. Kết quả triển khai

Giao diện trang chủ

Hình 4.x. Giao diện trang chủ SpecHub

Trang chủ là giao diện đầu tiên người dùng tiếp cận khi truy cập hệ thống. Giao diện cung cấp khu vực tìm kiếm nhanh và danh sách các thiết bị nổi bật, giúp người dùng có thể bắt đầu quá trình tra cứu mà không cần truy cập trực tiếp vào danh mục.

Từ trang chủ, người dùng có thể chuyển đến danh mục thiết bị, tìm kiếm, so sánh, khuyến nghị, Wiki hoặc các chức năng khác thông qua hệ thống điều hướng chung của ứng dụng.

Giao diện đăng ký, đăng nhập

Hình 4.x. Giao diện đăng ký tài khoản

Trang đăng ký cho phép người dùng tạo tài khoản mới bằng địa chỉ thư điện tử và mật khẩu. Các thông tin được kiểm tra trước khi gửi đến hệ thống nhằm hạn chế dữ liệu không hợp lệ.

Hình 4.x. Giao diện đăng nhập

Trang đăng nhập cho phép người dùng sử dụng thông tin tài khoản đã đăng ký để truy cập các chức năng yêu cầu xác thực. Sau khi đăng nhập thành công, trạng thái người dùng được quản lý thông qua `AuthProvider` và phiên có thể được làm mới khi cần thiết.

Giao diện danh mục thiết bị

Hình 4.x. Giao diện danh mục thiết bị

Trang danh mục hiển thị các thiết bị hiện có trong thư viện dưới dạng danh sách hoặc các thành phần trực quan. Người dùng có thể sử dụng bộ lọc và phân trang để thu hẹp phạm vi thiết bị cần tìm.

Thông tin danh mục được lấy từ dữ liệu đã được chuẩn hóa trong hệ thống. Mỗi mẫu thiết bị có thể tiếp tục chứa nhiều biến thể và các thành phần phần cứng liên quan. Tuyến `/devices` được triển khai riêng cho chức năng danh mục, lọc và phân trang.

Giao diện tìm kiếm

Hình 4.x. Giao diện tìm kiếm thiết bị và phần cứng

Trang tìm kiếm cho phép người dùng nhập từ khóa để tra cứu các đối tượng trong thư viện. Kết quả có thể bao gồm thiết bị và các thành phần phần cứng tương ứng.

Hệ thống hỗ trợ tìm theo tên, slug, alias và các dữ liệu liên quan. Phía Backend chuẩn hóa truy vấn và lựa chọn PostgreSQL hoặc Meilisearch tùy theo cấu hình trước khi trả kết quả cho giao diện.

Giao diện chi tiết thiết bị

Hình 4.x. Giao diện thông tin chi tiết thiết bị

Trang chi tiết là một trong những giao diện quan trọng nhất của SpecHub. Khi lựa chọn một mẫu thiết bị, người dùng có thể xem các thông tin tổng quan, biến thể và thông số kỹ thuật được chia thành từng nhóm.

Các nội dung được trình bày gồm:

- Thông tin chung của thiết bị.
- Các biến thể.
- Chipset, CPU, GPU và các thành phần xử lý.
- Màn hình.
- Camera.
- Pin và sạc.
- Bộ nhớ và lưu trữ.
- Kết nối.
- Phần mềm.
- Benchmark.
- Điểm đánh giá.
- Media và các dữ liệu liên quan.

Giao diện sử dụng các thành phần `device-spec-modules` để tổ chức thông số, `device-scorecard` để hiển thị kết quả đánh giá và `device-media-gallery` để trình bày hình ảnh hoặc video của sản phẩm.

Giao diện điểm đánh giá thiết bị

Hình 4.x. Giao diện bảng điểm thiết bị

Bên cạnh thông số gốc, hệ thống hiển thị kết quả đánh giá của thiết bị theo từng mô-đun. Người dùng có thể theo dõi điểm tổng và điểm của từng nhóm chỉ số thay vì chỉ nhận một giá trị đánh giá duy nhất.

Điểm số được gắn với hồ sơ chấm điểm và phương pháp tính tương ứng, giúp kết quả có khả năng giải thích thay vì trở thành một giá trị đánh giá không xác định nguồn gốc. Thành phần `device-scorecard` và `catalog-score` được sử dụng để trình bày điểm, mô-đun và phần giải thích liên quan.

Giao diện chi tiết phần cứng

Hình 4.x. Giao diện chi tiết thành phần phần cứng

SpecHub không chỉ cung cấp thông tin thiết bị mà còn cho phép truy cập trực tiếp các thành phần phần cứng. Người dùng có thể xem thông tin của chipset, CPU, GPU, NPU, modem, màn hình, camera, pin và các thành phần khác.

Việc xây dựng trang phần cứng riêng giúp người dùng nghiên cứu một linh kiện độc lập, đồng thời có thể xác định mối liên hệ giữa linh kiện và các thiết bị sử dụng linh kiện đó. Tuyến `/hardware/[kind]/[slug]` được sử dụng cho nhóm giao diện này.

Giao diện so sánh thiết bị

Hình 4.x. Giao diện so sánh thiết bị

Trang so sánh cho phép người dùng lựa chọn nhiều biến thể và đặt các thông số trên cùng một bảng đối chiếu.

Dữ liệu được chia theo các nhóm thuộc tính như hiệu năng, màn hình, camera, pin, bộ nhớ và kết nối. Những giá trị khác nhau được thể hiện rõ để người dùng dễ nhận biết ưu, nhược điểm của từng sản phẩm.

Đối với dữ liệu chưa được xác định, giao diện giữ trạng thái thiếu dữ liệu thay vì hiển thị thành giá trị bằng 0. Backend đồng thời tải dữ liệu các biến thể và quan hệ phân cứng theo lô để phục vụ quá trình so sánh.

Giao diện khuyến nghị thiết bị

Hình 4.x. Giao diện khuyến nghị thiết bị theo nhu cầu

Trang khuyến nghị hỗ trợ người dùng lựa chọn thiết bị dựa trên nhu cầu thay vì yêu cầu phải biết trước tên sản phẩm.

Người dùng cung cấp các tiêu chí như mục đích sử dụng, ngân sách và những yếu tố được ưu tiên. Hệ thống tiếp nhận các điều kiện này và trả về những thiết bị phù hợp cùng thông tin giải thích. Thành phần `device-recommender` được sử dụng để thu thập yêu cầu và trình bày kết quả gợi ý.

Giao diện trợ lý AI

Hình 4.x. Giao diện trợ lý AI SpecHub

Trang trợ lý AI cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên để tra cứu và nghiên cứu dữ liệu thiết bị.

Hệ thống có thể xử lý các yêu cầu như:

- Tra cứu thông số.
- So sánh thiết bị.
- Giải thích thành phần phần cứng.
- Nghiên cứu một chipset, CPU hoặc GPU.
- Khuyến nghị thiết bị theo điều kiện cụ thể.

Câu trả lời được truyền về giao diện theo luồng và có thể kèm theo các nguồn tham chiếu. Thành phần `ai-stream-answer` đảm nhiệm việc xử lý dữ liệu NDJSON của quá trình trả lời AI.

Hình 4.x. Kết quả hỏi đáp và trích dẫn của trợ lý AI

Kết quả AI được xây dựng dựa trên dữ liệu đã truy xuất từ catalog, phần cứng, Wiki và các nguồn được phép sử dụng. Các thông tin không được ngữ cảnh hỗ trợ được loại bỏ hoặc xử lý lại trước khi trả cho người dùng.

Giao diện Wiki

Hình 4.x. Giao diện danh sách Wiki

Wiki cung cấp khu vực nội dung kiến thức liên quan đến thiết bị, phần cứng và công nghệ. Người dùng có thể duyệt danh sách các bài viết đã được xuất bản và truy cập nội dung chi tiết.

Hình 4.x. Giao diện đọc bài Wiki

Trang chi tiết Wiki hiển thị nội dung bài viết và các nguồn trích dẫn đi kèm. Chỉ các nội dung đã được xuất bản mới được hiển thị cho người dùng công khai.

Hình 4.x. Giao diện biên tập Wiki

Đối với tài khoản có quyền phù hợp, hệ thống cung cấp giao diện tạo bài mới và chỉnh sửa nội dung. Mỗi lần chỉnh sửa tạo ra phiên bản mới thay vì ghi đè trực tiếp lên lịch sử bài viết. SpecHub triển khai riêng các tuyến đọc, tạo mới và chỉnh sửa Wiki với cơ chế kiểm soát quyền tương ứng.

Giao diện khu vực cá nhân

Hình 4.x. Giao diện bảng điều khiển cá nhân

Sau khi đăng nhập, người dùng có thể truy cập Dashboard để sử dụng các chức năng cá nhân hóa của tài khoản.

Hình 4.x. Giao diện danh sách yêu thích

Trang danh sách yêu thích cho phép người dùng lưu các thiết bị đang quan tâm để tiếp tục nghiên cứu hoặc so sánh sau đó.

Hình 4.x. Giao diện cảnh báo giá

Người dùng có thể tạo điều kiện cảnh báo đối với các biến thể đang theo dõi. Khi dữ liệu giá thỏa mãn điều kiện, hệ thống tạo thông báo cho người dùng.

Hình 4.x. Giao diện trung tâm thông báo

Trang thông báo tập hợp các thông tin được gửi đến tài khoản và hỗ trợ quản lý trạng thái đã đọc hoặc chưa đọc. Các tuyến Dashboard, Wishlist, Alerts và Notifications đều đã được triển khai riêng trong ứng dụng Web.

Giao diện Catalog Studio

Hình 4.x. Giao diện Catalog Studio

Catalog Studio là giao diện phục vụ biên tập viên và quản trị viên trong quá trình xây dựng thư viện thiết bị. Đây là một trong những giao diện quản trị quan trọng nhất của SpecHub.

Thay vì nhập toàn bộ thông tin thiết bị trong một biểu mẫu lớn, `catalog-studio-wizard` tổ chức quá trình nhập dữ liệu thành nhiều bước. Người biên tập có thể khai báo lần lượt thông tin chung, biến thể, phần cứng, màn hình, camera, pin, phần mềm, media và các dữ liệu liên quan.

Hình 4.x. Giao diện nhập và chỉnh sửa dữ liệu thiết bị

Trong quá trình biên tập, dữ liệu được lưu dưới dạng bản nháp và kiểm tra trước khi xuất bản. Hệ thống hỗ trợ lịch sử bản nháp, kiểm tra dữ liệu và phát hiện xung đột revision nhằm tránh ghi đè lên phiên bản mới hơn. Quy trình Catalog Studio hiện thực việc tạo/mở bản nháp, lưu kèm revision, kiểm tra trường và quan hệ, xác nhận xuất bản và cập nhật chỉ mục khi cần.

Giao diện quản lý hồ sơ chấm điểm

Hình 4.x. Giao diện quản lý hồ sơ chấm điểm

Giao diện `scoring-profile-manager` cho phép quản trị viên cấu hình phương pháp đánh giá thiết bị.

Người quản trị có thể quản lý:

- Các mô-đun đánh giá.
- Các chỉ số trong từng mô-đun.
- Trọng số.
- Chiều đánh giá.
- Quy tắc chuẩn hóa.
- Trạng thái của hồ sơ điểm.

Trước khi một hồ sơ được xuất bản, hệ thống kiểm tra tính hợp lệ của các trọng số. Phiên bản đã xuất bản được giữ ổn định để bảo đảm kết quả điểm đã tính có thể được kiểm tra lại.

Giao diện quản trị hệ thống

Hình 4.x. Giao diện bảng điều khiển quản trị

Khu vực quản trị được xây dựng dưới dạng `admin-console`, cung cấp giao diện dành cho các tài khoản có quyền quản trị.

Thông qua khu vực này, quản trị viên có thể truy cập các nhóm chức năng liên quan đến:

- Quản lý người dùng.
- Quản lý danh mục.
- Catalog Studio.
- Quản lý dữ liệu liên quan.
- Quản lý đối tác.
- Quản lý gói dịch vụ.
- Theo dõi các thông tin vận hành và nhật ký phù hợp với quyền truy cập.

Hình 4.x. Giao diện quản lý người dùng

Giao diện quản lý người dùng hỗ trợ quản trị viên theo dõi tài khoản và thực hiện các thao tác quản trị như thay đổi vai trò hoặc xử lý trạng thái tài khoản theo quy tắc của hệ thống.

Giao diện quản lý API dành cho đối tác

Hình 4.x. Giao diện quản lý API Key

Trang API Access cho phép người dùng hoặc đối tác có quyền quản lý khóa API và theo dõi mức sử dụng. Hệ thống hỗ trợ tạo, luân chuyển và thu hồi khóa, đồng thời áp dụng phạm vi truy cập và hạn mức phù hợp.

Tuyến `/api-access` được triển khai riêng và yêu cầu người dùng phải được xác thực.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Sau quá trình nghiên cứu, phân tích, thiết kế và phát triển, đồ án “**XÂY DỰNG NỀN TẢNG WEB THƯ VIỆN DÀNH CHO THIẾT BỊ THÔNG MINH**” đã xây dựng được hệ thống SpecHub theo mục tiêu đề ra. Hệ thống không chỉ dừng lại ở việc hiển thị thông số sản phẩm mà hướng đến xây dựng một kho dữ liệu thiết bị có cấu trúc, trong đó thông tin về mẫu thiết bị, biến thể, thành phần phần cứng, benchmark, nguồn tham chiếu và nội dung kiến thức được tổ chức và liên kết trên cùng một nền tảng.

Các kết quả chính đạt được bao gồm:

- **Về mặt kỹ thuật:** Xây dựng hệ thống Web theo kiến trúc Monorepo kết hợp Modular Monolith, với Frontend sử dụng Next.js và React, Backend sử dụng NestJS kết hợp Fastify, PostgreSQL và Prisma ORM phục vụ lưu trữ và truy cập dữ liệu. Các thành phần Web, API và tiến trình nền được phân chia rõ ràng, đồng thời sử dụng các package dùng chung nhằm thuận tiện cho việc bảo trì và mở rộng.
- **Về mặt dữ liệu:** Xây dựng được mô hình dữ liệu có cấu trúc, phân biệt rõ giữa dòng sản phẩm, mẫu thiết bị và biến thể. Các thành phần như chipset, CPU, GPU, NPU, modem, màn hình, camera, pin, bộ nhớ và hệ điều hành được quản lý thành các thực thể riêng và liên kết tới thiết bị thay vì lưu lặp lại trên từng sản phẩm. Cấu trúc dữ liệu hiện tại gồm 139 mô hình Prisma, cho phép biểu diễn nhiều nhóm dữ liệu và quan hệ khác nhau trong nền tảng.
- **Về mặt nghiệp vụ:** Hệ thống đã triển khai các chức năng chính gồm tra cứu danh mục, tìm kiếm và lọc thiết bị, xem thông tin chi tiết, tra cứu phần cứng, so sánh

thiết bị, khuyến nghị theo nhu cầu, quản lý Wiki, danh sách yêu thích, cảnh báo giá và thông báo. Quy trình Catalog Studio hỗ trợ biên tập dữ liệu dưới dạng bản nháp, kiểm tra tính hợp lệ, quản lý lịch sử và xuất bản dữ liệu có kiểm soát.

- **Về hệ thống đánh giá:** SpecHub xây dựng cơ chế chuẩn hóa và chấm điểm thiết bị dựa trên hồ sơ điểm, mô-đun và các chỉ số thành phần. Kết quả được lưu cùng phiên bản hồ sơ và dữ liệu đầu vào, giúp việc đánh giá có khả năng giải thích và kiểm tra lại thay vì chỉ cung cấp một điểm số cố định.
- **Về trí tuệ nhân tạo:** Hệ thống tích hợp trợ lý AI theo hướng Retrieval-Augmented Generation, trong đó câu trả lời được xây dựng dựa trên dữ liệu danh mục, phần cứng, benchmark, Wiki và nguồn đã được kiểm duyệt. Các thông tin định lượng quan trọng phải có căn cứ trong dữ liệu truy xuất; nội dung không được ngữ cảnh hỗ trợ được loại bỏ hoặc xử lý lại. AI vì vậy đóng vai trò hỗ trợ khai thác kho dữ liệu của SpecHub thay vì trở thành nguồn thông tin độc lập.
- **Về chất lượng mã nguồn:** Tại thời điểm rà soát, hệ thống đã vượt qua kiểm tra quy tắc mã và kiểm tra kiểu TypeScript; 36 bộ kiểm thử API với 189 ca kiểm thử đều đạt, các ca kiểm thử chấm điểm dữ liệu đạt yêu cầu, Prisma Schema hợp lệ và toàn bộ các workspace có yêu cầu build đều biên dịch thành công.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế. SpecHub chưa có kết quả kiểm thử tải chuẩn hóa để xác định khả năng phục vụ đồng thời ở quy mô lớn; độ phủ dữ liệu hiện tại vẫn phụ thuộc vào dữ liệu được nhập và các nguồn đã qua kiểm duyệt. Nội dung Wiki giữa các chủ đề chưa đồng đều và hệ thống AI tuy đã có cơ chế kiểm tra căn cứ nhưng chưa được đánh giá định lượng trên một bộ câu hỏi chuẩn. Một số dịch vụ tích hợp bên ngoài cũng chỉ có thể hoạt động đầy đủ khi được cung cấp thông tin xác thực của nhà cung cấp.

Nhìn chung, kết quả của đề án đã hình thành được nền tảng ban đầu cho một **thư viện dữ liệu thiết bị thông minh có cấu trúc và khả năng truy nguyên**, đồng thời tạo cơ sở để tiếp tục mở rộng về quy mô dữ liệu, chất lượng thông tin và các công cụ hỗ trợ nghiên cứu thiết bị trong tương lai.

2. Hướng phát triển

Trong tương lai, để nâng cao chất lượng dữ liệu, khả năng vận hành và phạm vi ứng dụng của SpecHub, hệ thống có thể tiếp tục được phát triển theo các hướng sau:

- **Mở rộng phạm vi thư viện thiết bị:** Tiếp tục hoàn thiện mô hình dữ liệu để mở rộng danh mục sang laptop, máy tính bảng, thiết bị đeo, TV và các nhóm thiết bị gia dụng thông minh trên cùng cấu trúc Model – Variant – Hardware Module. Việc mở rộng cần duy trì nguyên tắc chuẩn hóa thành phần dùng chung để hạn chế dữ liệu trùng lặp.

- **Nâng cao chất lượng dữ liệu Catalog:** Xây dựng bảng điều khiển theo dõi chất lượng dữ liệu, bao gồm tỷ lệ hoàn thiện trường thông tin, độ phủ nguồn, media, dữ liệu cũ và các trường chưa được xác minh. Đồng thời phát triển cơ chế phát hiện xung đột giữa nhiều nguồn để hỗ trợ biên tập viên trong quá trình kiểm duyệt.
- **Hoàn thiện hệ thống tìm kiếm và khai thác dữ liệu:** Tiếp tục tối ưu quá trình lập chỉ mục, tìm kiếm theo thuộc tính và tìm kiếm ngữ nghĩa khi số lượng thiết bị tăng lên. Khi quy mô thực tế yêu cầu, thành phần tìm kiếm có thể được tách thành dịch vụ riêng để mở rộng độc lập; việc phân tách chỉ nên được thực hiện khi có nhu cầu vận hành thực tế thay vì dựa đơn thuần vào số lượng module.
- **Nâng cao chất lượng trợ lý AI:** Xây dựng bộ dữ liệu đánh giá gồm câu hỏi chuẩn, đáp án tham chiếu và nguồn trích dẫn để đo lường độ chính xác, khả năng sử dụng nguồn và mức độ hữu ích của câu trả lời. Bên cạnh đó, tiếp tục cải thiện chiến lược Retrieval, xử lý ngữ cảnh và lựa chọn mô hình phù hợp nhằm nâng cao chất lượng hỏi đáp và khuyến nghị thiết bị.
- **Hoàn thiện kiểm thử và đánh giá hiệu năng:** Bổ sung kiểm thử End-to-End cho các hành trình chính như đăng nhập, tìm kiếm, so sánh, AI, Wiki, cảnh báo và Catalog Studio; thực hiện kiểm thử tích hợp với PostgreSQL và Redis thực tế, kiểm thử tải và đánh giá các chỉ số độ trễ tại những tuyến quan trọng. Đồng thời tiến hành kiểm thử bảo mật và rà soát hệ thống theo các tiêu chuẩn phù hợp.
- **Cải thiện khả năng vận hành tác vụ nền:** Hoàn thiện cơ chế khóa phân tán hoặc hệ thống hàng đợi để các công việc định kỳ như đồng bộ giá, thu thập nguồn và gửi thông báo có thể hoạt động an toàn khi triển khai nhiều bản sao của hệ thống.
- **Phát triển API dành cho đối tác:** Chuẩn hóa tài liệu OpenAPI, hỗ trợ sinh API Client tự động và mở rộng cơ chế cung cấp dữ liệu cho đối tác B2B. Trong giai đoạn tiếp theo, hệ thống có thể cung cấp webhook hoặc delta feed để đối tác nhận các thay đổi dữ liệu mà không cần liên tục truy vấn toàn bộ danh mục.
- **Tăng cường khả năng sao lưu và phục hồi:** Xây dựng quy trình sao lưu, khôi phục và thử nghiệm phục hồi định kỳ; xác định các chỉ số RPO và RTO phù hợp nhằm giảm rủi ro mất dữ liệu khi hệ thống được triển khai trong môi trường thực tế.
- **Hỗ trợ bản địa hóa và mở rộng theo khu vực:** Phát triển cơ chế quản lý dữ liệu theo nhiều khu vực, ngôn ngữ, đơn vị tiền tệ và chính sách bản địa hóa. Hướng phát triển này đặc biệt cần thiết do cùng một mẫu thiết bị có thể có tên thương mại, cấu hình phần cứng và biến thể khác nhau giữa các thị trường.

DANH MỤC THAM KHẢO

- [1] Microsoft, “The TypeScript Handbook,” *TypeScript Documentation*, [Online]. Available: <https://www.typescriptlang.org/docs/handbook/intro.html>. [Accessed: 09-Aug-2026].
- [2] Meta Platforms, Inc., “React Documentation – The library for web and native user interfaces,” *React*, [Online]. Available: <https://react.dev/>. [Accessed: 09-Aug-2026].
- [3] Vercel, “Next.js Documentation – App Router,” *Next.js*, [Online]. Available: <https://nextjs.org/docs/app>. [Accessed: 09-Aug-2026].
- [4] NestJS, “NestJS Documentation,” *NestJS – A progressive Node.js framework*, [Online]. Available: <https://docs.nestjs.com/>. [Accessed: 09-Aug-2026].
- [5] Fastify, “Fastify Documentation,” *Fastify*, [Online]. Available: <https://fastify.dev/docs/latest/>. [Accessed: 09-Aug-2026].
- [6] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation – Indexes,” *PostgreSQL Documentation*, [Online]. Available: <https://www.postgresql.org/docs/16/indexes.html>. [Accessed: 09-Aug-2026].
- [7] Prisma Data, Inc., “Prisma ORM v6 Documentation,” *Prisma Documentation*, [Online]. Available: <https://www.prisma.io/docs/orm/v6>. [Accessed: 09-Aug-2026].

- [8] Redis, “Redis Data Types,” *Redis Documentation*, [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>. [Accessed: 09-Aug-2026].
- [9] Meilisearch, “Meilisearch Documentation – Full-text Search and Typo Tolerance,” *Meilisearch Documentation*, [Online]. Available: <https://www.meilisearch.com/docs/>. [Accessed: 09-Aug-2026].
- [10] pgvector, “pgvector – Open-source vector similarity search for Postgres,” *GitHub*, [Online]. Available: <https://github.com/pgvector/pgvector>. [Accessed: 09-Aug-2026].
- [11] Ollama, “Ollama Documentation – Embeddings,” *Ollama Documentation*, [Online]. Available: <https://docs.ollama.com/capabilities/embeddings>. [Accessed: 09-Aug-2026].
- [12] Ollama, “Generate a Chat Message,” *Ollama API Documentation*, [Online]. Available: <https://docs.ollama.com/api/chat>. [Accessed: 09-Aug-2026].
- [13] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>. [Accessed: 09-Aug-2026].
- [14] Ollama, “Qwen2.5 Model Library,” *Ollama*, [Online]. Available: <https://ollama.com/library/qwen2.5>. [Accessed: 09-Aug-2026].
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>. [Accessed: 09-Aug-2026].
- [16] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Doctoral dissertation, University of California, Irvine, 2000. [Online]. Available: <https://roy.gbiv.com/pubs/dissertation/top.htm>. [Accessed: 09-Aug-2026].

- [17] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” *RFC 9110*, *Internet Engineering Task Force (IETF)*, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110.html>. [Accessed: 09-Aug-2026].
- [18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” *RFC 7519*, *Internet Engineering Task Force (IETF)*, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519.html>. [Accessed: 09-Aug-2026].
- [19] Microsoft, “Visual Studio Code Documentation,” *Visual Studio Code*, [Online]. Available: <https://code.visualstudio.com/docs>. [Accessed: 09-Aug-2026].
- [20] Tailwind Labs, “Tailwind CSS Documentation,” *Tailwind CSS*, [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 09-Aug-2026].
- [21] TanStack, “TanStack Query Documentation,” *TanStack Query*, [Online]. Available: <https://tanstack.com/query/latest/docs/framework/react>. [Accessed: 09-Aug-2026].
- [22] Vercel, “Turborepo Documentation,” *Turborepo*, [Online]. Available: <https://turborepo.dev/docs>. [Accessed: 09-Aug-2026].
- [23] pnpm, “pnpm Documentation,” *pnpm*, [Online]. Available: <https://pnpm.io/>. [Accessed: 09-Aug-2026].

